

Darstellung von Kaustiken mit Hilfe von Photon Mapping

Studienarbeit

vorgelegt von
Marco Thum



Institut für Computervisualistik
Arbeitsgruppe Computergrafik

Betreuer: Dipl.-Inf. Markus Geimer
Prüfer: Prof. Dr. Stefan Müller

Dezember 2003

Inhaltsverzeichnis

1. Motivation	2
2. Grundlagen photorealistischer Computergraphik	3
2.1 Globale Beleuchtung	3
2.1.1 Die Idee des Raytracings	3
2.2 Radio- und Photometrie	5
2.3 Lokale Beleuchtungsmodelle	7
2.3.1 BRDF	7
2.3.2 Phong-Shading	7
2.3.3 Schlick	9
2.4 Monte-Carlo-Raytracing	12
2.4.1 Distribution Raytracing	12
2.4.2 Path Tracing	13
3. Photon Mapping.....	14
3.2 Einbindung der Rendering Equation.....	15
3.3 First Pass: Photon Tracing	16
3.3.1 Emission.....	16
3.3.2 Scattering.....	20
3.3.3 Abspeicherung in der Photon Map.....	23
3.1 Architektur der Photon Map	24
3.4 Second Pass: Rendering	25
3.4.1 Direkte Beleuchtung.....	25
3.4.2 Indirekte diffuse Beleuchtung.....	26
3.4.3 Indirekte spekulare Beleuchtung und Glanzreflexion	26
3.4.4 Kaustiken.....	27
3.4.5 Radiance Estimate – Abschätzung der Leuchtdichte	27
4. Dokumentation des Raytracers	30
4.1 Modularisierung	30
4.2 Spezifikation.....	33
4.2.1 Auslesen der Kommandozeile.....	34
4.2.2 Konstruktion von Szene und Raytracer.....	35
4.2.3 Verfolgung von Sehstrahlen.....	36
4.2.4 Shading.....	38
5. Integration	45
5.1 Basiskonfiguration	45
5.2 Klassen und Objekte	47
5.2.1 PhotonMap	47
5.2.2 Light	52
5.2.3 AreaLight	55
5.2.4 Spotlight	58
5.2.5 Sample.....	58
5.2.6 Scene	60
5.2.7 Raytracer	61
5.2.8 PhotonMaterial	62

6. Resultate	69
6.1 Cornell-Box	69
7. Optimierungsmöglichkeiten	74
7.1 Beschleunigung	74
7.1.1 Parallelisierung	74
7.1.2 Irradiance Caching	74
7.1.3 Shadow Photons	75
7.1.4 Projection Maps	75
Fazit	76
Literatur	77
Anhang	79

1. Motivation

Eine der wesentlichen Motivationen in der Geschichte der Computergraphik besteht darin, Graphiken möglichst photorealistisch darzustellen. Um dieses Ziel zu erreichen, werden mehrere Verfahren angewandt, die sich hinsichtlich Effizienz und Realismus deutlich unterscheiden. Eines der gängigsten Verfahren zur Simulation photorealistischer Bilder ist das *Raytracing*. Es ist ein Verfahren, das versucht, möglichst viele Informationen, die von sich in der Szene befindlichen Lichtquellen und Objekten ausgehen, mit in die Berechnung einzubeziehen. Allerdings wird die Effizienz des herkömmlichen *Raytracing* einerseits durch seine geringe Performance, andererseits durch geringe photometrische Konsistenz eingeschränkt. Diverse Beschleunigungstechniken und physikalisch komplexe, lokale Beleuchtungsmodelle grenzen diesen Punkt ein.

Striktes *Raytracing* arbeitet zudem eher auf lokaler denn auf globaler Ebene. Beispielsweise ist es mit herkömmlichen *Raytracing* nicht möglich, die indirekte Beleuchtung auf *diffusen* Oberflächen zu ermitteln. Globale Beleuchtungsinformationen werden erst mittels zusätzlicher Verfahren besser evaluiert. Eines dieser Verfahren ist das *Photon Mapping*, welches von Henrik Wann Jensen als Teil seiner Dissertation zwischen den Jahren 1993-1994 entwickelt und zwischen 1995-1996 erstmals veröffentlicht wurde. Die Informationen über die globalen Beleuchtungsverhältnisse werden gewonnen, indem vor dem Rendering Energiewerte von den Lichtquellen der Szene ausgesandt und über spekulare Flächen hinweg bis zu den diffusen Flächen verfolgt werden. An dieser Stelle werden die von den Photonen, den Energieträgern, bis hierhin verteilten und durch Reflexion und Refraktion veränderten Energiewerte gespeichert.

Dass globale Beleuchtung mittels *Photon Mapping* simuliert werden kann, führt dazu, dass sich auch Effekte, die eigentlich komplexe physikalische Berechnungen benötigen, darstellen lassen. Derjenige Effekt, der entsteht, wenn Lichtpartikel durch transmittierende Materialien dringen, schließlich konzentriert auf einer diffusen Fläche liegen bleiben und somit eine Art Brennpunkt bilden, nennt man *Kaustik*.



Abb. 1.1: Kaustik eines Cognac-Glas auf Tischplatte (Jensen , 1995)

Ziel dieser Studienarbeit ist es nun, die Idee des *Photon Mapping* in ein bestehendes Raytracing-System zu integrieren, so dass sich neben allgemeiner globaler Beleuchtung auch *Kaustiken* darstellen lassen. Auf dem Weg dorthin sollen die Features des Raytracing-Systems und die Theorie des Photon Mapping beschrieben und erklärt werden. Zuletzt sollen einige Vergleichstests die Effizienz des Verfahrens verdeutlichen und ein Ausblick auf Optimierungen des *Photon Mapping* gegeben werden.

2. Grundlagen photorealistischer Computergraphik

In diesem Kapitel sollen die wichtigsten theoretischen und mathematischen Grundlagen zum Thema *Photorealistische Computergraphik* erläutert werden. Angefangen von der Idee des *Raytracings*, über diverse Beleuchtungsmodelle, bis hin zu verbesserten *Raytracing*-Algorithmen, soll das Basiswissen vermittelt werden, um die Funktionsweise des implementierten Systems zu verstehen.

2.1 Globale Beleuchtung

Unter globaler Beleuchtung (*global illumination*) versteht man den Vorgang, wie die Lichtquellen auf die Objekte einer dreidimensional modellierten Welt einwirken. Währenddessen sollen alle Interaktionen, die das Licht mit seiner Umgebung eingeht, mit in das zu rendernde Bild einfließen. Wichtigstes Kriterium ist hierbei die *indirekte* Interaktion des Lichts mit den Objekten. Werden Objekte lediglich *direkt* beleuchtet, ist der Eindruck weniger photorealistisch. Um dies zu realisieren, existieren in der Forschung der Computergraphik mehrere Ansätze:

- Punktesampling (Raytracing)
- Finite Elemente (Radiosity)

Auf die Finiten Elemente Techniken soll hier jedoch nicht näher eingegangen werden. Für weitere Informationen kann [Goral84] gesichtet werden.

2.1.1 Die Idee des Raytracings

Da heutzutage die Visionen *virtueller Realität* zwar recht nahe gerückt sind, aber die Vision holographischer 3D-Displays vor den Augen eines jeden Computerbesitzers noch weit entfernt liegt, ist man immer noch daran interessiert, dreidimensionale Daten möglichst photorealistisch auf zweidimensionaler Ebene abzubilden. Diese Transformation gestaltet sich sehr schwierig, weil viele Informationen der dreidimensionalen Welt mit in die zweidimensionale Abbildung mit einfließen müssen. Der wichtigste Faktor ist hierbei das *Licht* und sein Einfluss auf die dreidimensionalen Körper.

Herkömmlicherweise sendet das Licht Strahlung aus, die von der Geometrie der Szene reflektiert, refraktiert oder absorbiert wird und schließlich das Auge des Betrachters erreicht. Dieser Weg der Berechnung ist allerdings nicht sehr praktikabel, weshalb vom Augpunkt aus gestartet wird und die Informationen der Lichtquellen rekursiv eingeholt wird. In der Notation für Lichttransporte [Heckbert90] berechnet *Raytracing* folgende Lichtpfade:

$$LD?S^*A$$

Ausgehend von einer statischen dreidimensionalen Szene mit Informationen über geometrische Körper und deren Materialeigenschaften, Lichtquellen mit ihrem Lichtstrom, und der Position und Blickrichtung des Betrachters, werden zu Beginn des *Backward-Raytracing* Strahlen von der Betrachterposition in seine Blickrichtung „verschossen“ (sog. Augstrahlen). Diese Strahlen haben allerdings nicht nur eine einzige Richtung, sondern tasten ein Gitterfeld ab, das dem gesamten Blickfeld des Betrachters auf die Szene entspricht, sozusagen ist dies das zweidimensionale „Fenster“

zur dreidimensionalen Welt. Das Gitterfeld ist zugleich das zweidimensionale Abbild der dreidimensionalen Szene und jeder Abtastpunkt mit Koordinate (x, y) entspricht einem Pixel des Abbildes an der Stelle (x, y) .

Um die zu dem Pixel an der Stelle (x, y) zugehörigen Farbinformationen zu gewinnen, wird der vom Auge (oder auch Kamera) ausgesandte Sehstrahl soweit in die Szene „verfolgt“, bis er Geometrie trifft. Allerdings ist dieser Algorithmus ohne Beschleunigungsmethoden mit hohem Aufwand verbunden, da jedes geometrische Objekt der Szene auf einen Treffer hin getestet werden muss, bis das an vorderster Stelle liegende ermittelt ist. Ebenso stoppt die Verfolgung, wenn Objekte weiter entfernt liegen, als der Strahl lang ist.

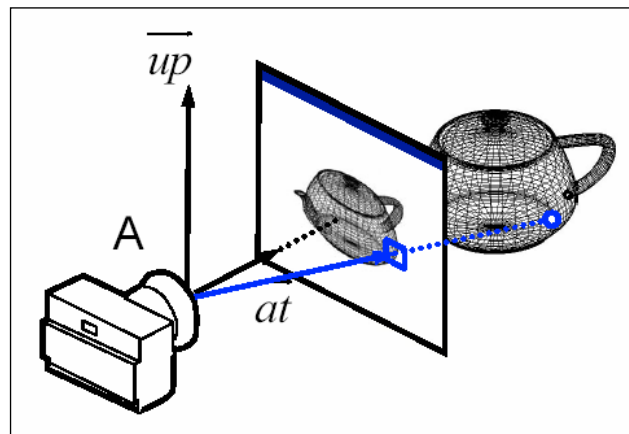


Abb. 2.1: Abtastung des Rastergitters mittels Sehstrahlen
[Müller03]

Ist ein Schnittpunkt mit einem geometrischen Objekt ermittelt, werden die ersten Farbinformationen gesammelt, indem die Materialparameter des Objektes untersucht werden. Des Weiteren wird die Beleuchtung des Schnittpunkts durch die in der Szene vorhandenen Lichtquellen überprüft. Je nachdem, welche Intensität, Position und Richtung im Verhältnis zum Schnittpunkt sie besitzen, werden ihre Parameter mit in den finalen Farbwert des Schnittpunkts einberechnet. Der Sehstrahl wird abhängig von der Beschaffenheit des Materials absorbiert, reflektiert oder refraktiert und rekursiv weiterverfolgt. Rekursiv eingesammelte Informationen, die von der Geometrie der restlichen Szene auf den Schnittpunkt einwirken, werden aufgerechnet. Bei Absorption stoppt die Verfolgung und die resultierende Farbinformation wird an Position (x, y) gesetzt. Zu dem Zweck, ob ein Schnittpunkt überhaupt von der Lichtquelle direkt erreicht wird, wird vorher ein *Schattenfühler* zu ihrer Position gesandt.

```

void render() {
    ImagePlane *ip = new ImagePlane(pic_width, pic_height);
    Camera cam;
    for(each pixel of image plane){
        Ray r = cam.computeRay(pixel, ip);
        Color c = raytrace(ray, recursion_depth);
        setPixel(pixel, c);
    }
}
Color raytrace(Ray ray, int recursion_depth){
    LocalGeometry lg = Scene.intersect(ray);
    Color c = lightshade(lg.getIntersectionInfo(), recursion_depth);
    return c;
}
Color lightshade(IntersectionInfo, int recursion_depth){
    for(each lightsource in scene)
        if(shadowIntersect(ray_from_intersection_to_source)==false)
            color = color + illumination;
    if(intersected_object_with_specular_material)
        color = color + raytrace(secondary_ray, --recursion_depth);
    return color;
}

```

Algorithmus 2.1: Pseudocode des *Raytracing*

2.2 Radio- und Photometrie

Eine kurzer Überblick zu radio- und photometrischen Größen soll dazu verhelfen, die Arbeitsweise des später vorgestellten Raytracers und des Photon Mappings zu verstehen.

Photonenenergie

Wenn man photorealistische Computergraphik darstellen möchte, so ist die Basis hierzu generell die Simulation von Licht. Die Grundeinheit des Lichts ist das *Photon*, ein Partikel des Lichtflusses. Die Energie e_λ eines Photons mit der Wellenlänge λ ist

$$e_\lambda = \frac{hc}{\lambda}, \quad (2.1)$$

mit $h \approx 6.63 \cdot 10^{-34} \text{ J} \cdot \text{s}$ als die Planck'sche Konstante, und c als die Geschwindigkeit des Lichts (in einem Vakuum ist $c = c_0 = 299.792.458 \text{ m/s}$).

(Spektrale) Strahlungsmenge $Q(\lambda)$ und Lichtmenge Q

Die Strahlungsmenge $Q(\lambda)$ in n_λ Photonen mit Wellenlänge λ ist definiert durch

$$Q(\lambda) = n_\lambda e_\lambda = n_\lambda \frac{hc}{\lambda} \quad (2.2)$$

Die Lichtmenge Q ist die Energie einer Ansammlung von Photonen und wird definiert durch das Integral der (spektralen) Strahlungsmenge über alle möglichen Wellenlängen:

$$Q = \int_0^\infty Q_\lambda d\lambda \quad (2.3)$$

Strahlungsfluss $\phi(\lambda)$ und Lichtstrom ϕ

Um den Einfluss der Lichtquellen und somit der Photonen auf die globale Szene zu ermitteln, muss man den Strahlungsfluss $\phi(\lambda)$ kennen, den die Lichtquelle in den Raum abgibt. $\phi(\lambda)$ ist der Quotient aus der Strahlungsmenge $Q(\lambda)$ und der Zeit t :

$$\phi = \frac{dQ}{dt} \quad (2.4)$$

Strahlstärke $I(\lambda)$ und Lichtstärke I

Unter der Strahlstärke $I(\lambda)$ versteht man den Quotienten aus dem von einer Lichtquelle in einer bestimmten Richtung ausgesandten Strahlungsfluss und dem durchstrahlten Raumwinkel.

$$I = \frac{d\phi}{d\omega} \quad (2.5)$$

Strahldichte $L(\lambda)$ und Leuchtdichte L

Unter der Leuchtdichte L versteht man die Menge des Lichtstroms ϕ , die bei der Betrachtung der Fläche A der Lichtquelle aus dem Winkel θ und dem durchstrahlten Raumwinkel ω gesehen wird. Je kleiner die Lichtquelle ist, desto intensiver wird sie gesehen.

$$L = \frac{d^2\phi}{dA \cdot \cos\theta \cdot d\omega} \quad (2.6)$$

Die Strahldichte ist invariant und somit verlustfrei auf der Strecke vom Sender zum Empfänger, solange die Strahlen nicht umgelenkt und ihre Werte verändert werden.

Bestrahlungsstärke $E(\lambda)$ und Beleuchtungsstärke E

Unter der Beleuchtungsstärke E versteht man den ankommenden Anteil des auf einer Fläche A von einer Lichtquelle ausgestrahlten Strahlungsflusses ϕ .

$$E = \frac{d\phi}{dA} \quad (2.7)$$

In dieser Studienarbeit werden überwiegend die Bezeichner und Größen der Photometrie verwendet, da für die Radiometrie das Wissen über zusätzliche physikalische Größen (z.B. Spektren, Wellenlängen, LVK) benötigt wird. Nähere Informationen bietet [Müller03].

2.3 Lokale Beleuchtungsmodelle

In einem *Raytracing*-System geht es darum, die Verhältnisse eingehender und ausgehender Strahlung an einer bestimmten Pixelposition zu berechnen. Lokal nennt man die Beleuchtungsmodelle deshalb, weil sie die Berechnung nur für eine Position im Raum durchführen und nicht global agieren. Die Unterschiedlichkeit von Strahlungen und Reflexionen macht es notwendig, dass diese – soweit wie möglich – physikalisch korrekt bestimmt werden.

2.3.1 BRDF

Die bidirektionale Reflexionsverteilungsfunktion (*Bidirectional Reflectance Distribution Function*) ist eine sechsdimensionale Funktion, die das grundlegende Verhältnis von eingehender zu ausgehender Strahlung beschreibt. Sie wird definiert als:

$$f_r(p_x, \bar{w}_i, \bar{w}_o) = \frac{dL_r(p_x, \bar{w}_o)}{dE_i(p_x, \bar{w}_i)} \quad (2.8)$$

mit

- p_x beleuchteter Schnittpunkt
- $\bar{w}_i$ Einfallrichtung
- $\bar{w}_o$ Ausfallrichtung
- L_r reflektierte Leuchtdichte
- E_i eingehende Beleuchtungsstärke

Ferner kann Gleichung (2.8) unter Berücksichtigung des Quotienten $\frac{d\phi}{dA}$ der Beleuchtungsstärke vereinfacht werden zu:

$$f_r(p_x, \bar{w}_i, \bar{w}_o) = \frac{dL_r(p_x, \bar{w}_o)}{L_i(p_x, \bar{w}_i)(\bar{w}_i \cdot \bar{n})d\bar{w}_i} \quad (2.9)$$

mit

- L_i eingehende Leuchtdichte
- $\bar{n}$ Normale am Schnittpunkt

Die Schwierigkeit liegt nun darin, die BRDF an die unterschiedlich reflektierenden Materialien anzupassen. Für einfache Lambert-Reflexionen und perfekte Spiegelreflexion sind die Umformungen nicht kompliziert, allerdings erhöht sich die Komplexität mit der Vielzahl an Parametern, von denen ein Material bestimmt wird. Nähere Erläuterungen hierzu finden sich in späteren Kapiteln. Im folgenden werden zwei Modelle vorgestellt, die sich dieser Aufgabe annehmen.

2.3.2 Phong-Shading

Um es vorwegzunehmen: Das *Phong-Modell* ist physikalisch nicht korrekt. Es wird hauptsächlich dazu verwendet, spekulare Reflexionen mit besonderen *HotSpots* deutlich sichtbarer zu machen. Allerdings bindet das vorliegende Raytracing-System dieses Modell mit ein, weshalb es an dieser Stelle kurz angerissen wird.

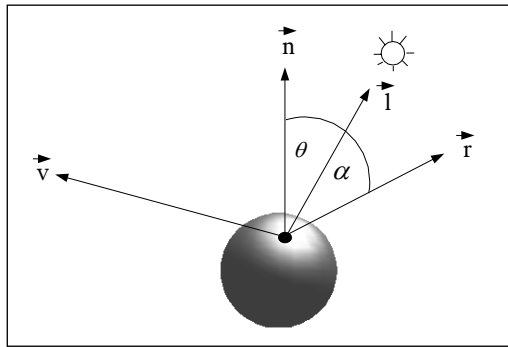


Abb. 2.2: Beleuchtungsmodell von Phong
mit $\vec{v}$ Sehstrahl, $\vec{r}$ Reflexionsstrahl,
 $\vec{n}$ Normale, $\vec{l}$ Lichtvektor

Um eine Verwechslung mit photometrischen Größen auszuschließen, wird im folgenden – aber nur für diesen Abschnitt – die Bezeichnung *Intensität* I als Pseudonym für die reflektierte Leuchtdichte verwendet. Dann ergibt sich die Gesamtintensität eines Punktes p_x durch

$$I_{ges} = I_a + I_d + I_s \quad (2.10)$$

mit

- I_a ambienter Term / Anteil
- I_d diffuser Term / Anteil
- I_s spekulärer Term / Anteil

Der ambiante Anteil ist die allgemeine globale Beleuchtung, die sich homogen über die Szene ausbreitet. Welchen Anteil ein Punkt mitnimmt, bestimmt folgende Gleichung:

$$I_a = L_a \cdot I_{Global_a} \quad (2.11)$$

mit

- L_a ambienter Anteil der Leuchtdichte des Materials
- I_{Global_a} globale ambiante Beleuchtung

Der diffuse Anteil setzt sich zusammen aus

$$I_d = \cos \theta \cdot k_d \cdot L_d \quad (2.12)$$

mit

- $\cos \theta = (\vec{l} \cdot \vec{n})$ Reflexionswinkel der Lichtquelle
- k_d diffuser Reflexionsgrad des Materials
- L_d Intensität der Lichtquelle

Für den spekularen Anteil kommt der sogenannte *Phong-Koeffizient* ins Spiel. Dieser potenziert den Cosinus zwischen reflektiertem Sehstrahl und einfallendem Licht. Je größer der Koeffizient, desto intensiver wirkt der Glanz und desto spiegelnder ist das Material:

$$I_s = \cos^p \alpha \cdot k_s \cdot L_s \quad (2.13)$$

mit

- p Phong-Koeffizient
- α Cosinus zwischen reflektiertem Sehstrahl und einfallendem Licht
- k_s spekulärer Reflexionsgrad des Materials
- L_s Intensität der Lichtquelle

2.3.3 Schlick

Ein intuitiveres und plausibleres Beleuchtungsmodell stammt von Schlick [Schlick93]. Es ist empirisch nach einigen theoretischen Ansätzen und physikalischer Betrachtung von Lichtverhalten gewachsen und kommt den Ansprüchen photorealistischer Darstellung nahe.

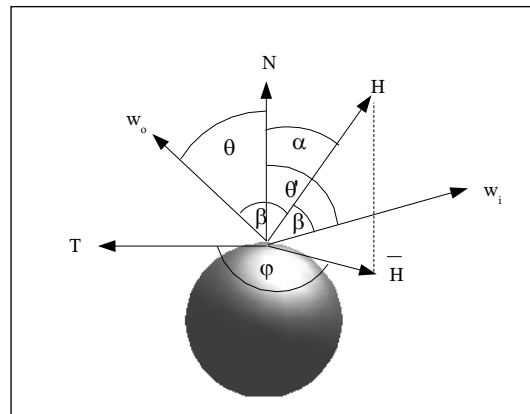


Abb. 2.3: Vektoren und Winkel des Schlick-Modells
(Erläuterung in Tab. 2.1)

Folgende Gleichung bestimmt die BRDF von Schlick:

$$f_r(p_x, \bar{w}_i, \bar{w}_o) = S(u) \left\{ \frac{d}{\pi} + g \cdot D(t, v, v', w) + s \cdot f_{r,s}(p_x, \bar{w}_i, \bar{w}_o) \right\} \quad (2.14)$$

mit

- $S(u)$ Spektraler Faktor, approximiert nach Fresnel
- d diffuser Reflexionsgrad des Materials
- g Glanzgrad des Materials
- s spekularer Reflexionsgrad des Materials
- D richtungsweisender Faktor
- $f_{r,s}$ spekulare BRDF

Die BRDF wird hauptsächlich von der Fresnel-Approximation $S(u)$ bestimmt. Zu ihrer Herleitung folgt an dieser Stelle ein kurzer Exkurs.

Exkurs: Fresnel-Terme

Fresnel hat eine physikalisch orientierte Theorie entwickelt, die Reflexion von Licht anhand des Refraktionsindex eines Materials und dem Winkel des Lichteinfalls zu bestimmen. Er unterscheidet bei seiner Untersuchung die Polarisation der Lichtquelle. Daher hat er zwei Terme für zwei Arten elektromagnetischer Felder aufgestellt: $p_{\parallel}$ steht für ein Feld, das parallel zum Lichteinfall ist, und $p_{\perp}$ für ein Feld, das orthogonal zum Lichteinfall ist. Beide Terme sind Reflexionskoeffizienten für polarisiertes und unpolarisiertes Licht. Die Gleichungen lauten im einzelnen:

$$\begin{aligned}
 p_{\parallel} &= \frac{\eta_2 \cos \theta_1 - \eta_1 \cos \theta_2}{\eta_2 \cos \theta_1 + \eta_1 \cos \theta_2} \\
 p_{\perp} &= \frac{\eta_1 \cos \theta_1 - \eta_2 \cos \theta_2}{\eta_1 \cos \theta_1 + \eta_2 \cos \theta_2}
 \end{aligned}
 \tag{2.15}$$

mit

- η_1 Refraktionsindex des Eintrittsmaterials
- η_2 Refraktionsindex des Austrittsmaterials
- θ_1 Eintrittswinkel
- θ_2 Austrittswinkel (im Material)

In der Computergraphik wird die Polarisation allerdings nur selten betrachtet. Deswegen sollte das Mittel der beiden Terme genommen werden. Dies führt zu folgender Gleichung:

$$F_r(\theta) = \frac{1}{2}(p_{\parallel}^2 + p_{\perp}^2) \tag{2.16}$$

Gleichung (2.16) beschreibt die Relation von ausgehendem zu eingehendem Strahlungsfluss der Lichtquellen und ist definiert durch den Quotienten

$$\frac{d\phi_r}{d\phi_i} \tag{2.17}$$

Approximation

Die von Fresnel definierten Terme können allerdings auch effizient angenähert werden. Schlick schlägt dazu Gleichung (2.18) vor:

$$F_r(\theta) = F_0 + (1 - F_0)(1 - \cos \theta)^5 \tag{2.18}$$

mit

- F_0 Fresnel-Reflexionskoeffizient bei Senkrechteinfall (Normale)

$S(u)$ ist folglich äquivalent zu $F_r(\theta)$, mit $u = \cos \theta$.

Faktorberechnungen

Für den Term D , der den Grad der Glanzreflexion bestimmt, sind weitere Berechnungen von Winkeln und Vektoren nötig wie sie in Abbildung 2.3 abzulesen sind:

Größe	Bedeutung
$\vec{H} = \frac{\vec{w}_i + \vec{w}_o}{\ \vec{w}_i + \vec{w}_o\ }$	Halbvektor zwischen reflektierter und eingehender Beleuchtung
$\vec{u} = \vec{w}_o \cdot \vec{H}$	$\cos \beta$

$\vec{t} = \vec{n} \cdot \vec{H}$	$\cos \alpha$
$v = \vec{w}_o \cdot \vec{n}$	$\cos \theta$
$v' = \vec{w}_i \cdot \vec{n}$	$\cos \theta'$
$\vec{T} = \vec{n}_\perp$	Tangentialvektor der Oberfläche zur Normalen (Orthogonale zur Normalen)
$w = \vec{T} \cdot \frac{\vec{H} - (\vec{n} \cdot \vec{H})\vec{n}}{ \vec{H} - (\vec{n} \cdot \vec{H})\vec{n} }$	Cosinus des Winkels von orthogonaler Projektion $\vec{H}$ von H auf N und dem Tangentialvektor T ($\cos \varphi$)

Tab. 2.1: Größen zur Bestimmung des *direktionalen* Faktors

Des Weiteren berücksichtigt Schlick anisotropische und isotropische Oberflächen mittels zweier Terme $Z(t)$ und $A(w)$. Denjenigen Anteil einfallenden und reflektierten Lichts, der *nicht* von der Oberfläche selbst abgeschattet wird, beschreibt $G(v)$ respektive $G(v')$. Zur Bestimmung dieser Terme werden die Materialparameter σ für die Feinheit und ψ für den Isotropiegrad benötigt. Die Terme lauten:

$$Z(t) = \frac{\sigma}{(1 + \sigma t^2 - t^2)^2} \quad (2.19)$$

$$A(w) = \sqrt{\frac{\psi}{\psi^2 - \psi^2 w^2 + w^2}} \quad (2.20)$$

$$G(v) = \frac{v}{\sigma - \sigma v + v} \quad (2.21)$$

Damit lässt sich der *direktionale* Term D , der den Weg des Lichts auf der Oberfläche des Materials beschreibt, nun zusammensetzen aus:

$$D(t, v, v', w) = \frac{G(v)G(v')Z(t)A(w)}{4\pi vv'} + \frac{1 - G(v)G(v')}{\pi} A(w) \quad (2.22)$$

Zur Auflösung der Schlick-BRDF sind nun noch die Reflexionsgrade des Materials festzulegen. Diese können für das Material explizit gesetzt werden oder lassen sich implizit anhand der Feinheit der Materialoberfläche bestimmen:

$$\begin{aligned} g &= 4\sigma(1 - \sigma) \\ d &= \begin{cases} 0 & \text{wenn } \sigma < 0.5 \\ 1 - g & \text{sonst} \end{cases} \\ s &= \begin{cases} 1 - g & \text{wenn } \sigma < 0.5 \\ 0 & \text{sonst} \end{cases} \end{aligned} \quad (2.23)$$

2.4 Monte-Carlo-Raytracing

Unter Monte-Carlo-Raytracing versteht man diejenigen Verfahren, die mittels stochastischer Funktionswerte versuchen, die Integrale der *Rendering Equation* zu lösen. Diesen Vorgang nennt man *Monte-Carlo-Integration*. Die Werte der Integrale werden als Erwartungswerte der Zufallsvariablen angesehen und durch Stichprobennahmen (sog. *Samples*) angenähert. Die hier vorgestellten Methoden haben zwei Erscheinungen gemeinsam: Sie erzeugen *Punktesamples* und verursachen *Varianz (Noise)*.

Unter *Varianz* versteht man die durchschnittliche quadratische Abweichung einer Zufallsvariable x zum Erwartungswert $E(X)$:

$$Var = \frac{1}{n} \cdot \sum_{i=1}^n (x_i - E(X))^2 \quad (2.24)$$

mit

- n Anzahl der Zufallszahlen

Es ist eine statistisch bewiesene Tatsache, dass die *Varianz* umso mehr reduziert wird, je mehr *Samples* erzeugt werden. Strategien zur Reduktion des Fehlers sind beispielsweise *Stratified Sampling*, *Jittering* und *Importance Sampling*.

Rendering Equation

Die zu lösende Rendering Equation [Kajiya86] ist die ausgehende Leuchtdichte L_o am Punkt p_x , definiert als die Summe von emittierter (L_e) und reflektierter (L_r) Leuchtdichte:

$$L_o(p_x, \bar{w}_o) = L_e(p_x, \bar{w}_o) + L_r(p_x, \bar{w}_o) \quad (2.25)$$

Das zu lösende Integral ist das der reflektierten Leuchtdichte über alle Richtungen in der Hemisphäre des Punktes p_x und kann aufgelöst werden als:

$$L_r(p_x, \bar{w}_o) = \int_{\Omega} f_r(p_x, \bar{w}_i, \bar{w}_o) L_i(p_x, \bar{w}_i) (\bar{w}_i \cdot \bar{n}) d\bar{w}_i \quad (2.26)$$

2.4.1 Distribution Raytracing

Ein früher Ansatz zur Lösung der Rendering Equation wurde von Cook et. al. [Cook84] entwickelt. Zur Bestimmung der reflektierten Leuchtdichte werden von diffusen Flächen aus mehrere stochastisch ermittelte Sehstrahlen losgeschickt. Für indirekte Effekte müssen mehr Strahlen geschossen werden, wodurch die ungewünschte Varianz entsteht.

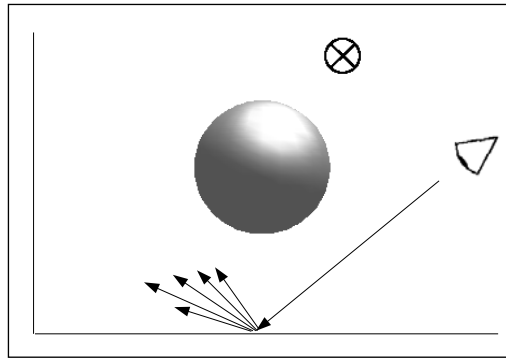


Abb. 2.4: Tracing mehrerer Sekundärstrahlen bei diffuser Reflexion (*Distribution*)

2.4.2 Path Tracing

Eine Erweiterung des Cook-Modells ist das Path Tracing [Kajiya86]. Anstatt bei der Reflexion von Sehstrahlen alle Pfade zu verfolgen, wird nur ein Pfad anhand einer Zufallsvariable und den Reflexionsgraden des Materials ausgewählt. Um die Varianz zu reduzieren, werden mehrere Strahlen durch ein Pixel des Abtastgitters geschickt. Je höher die Anzahl der Strahlen, desto näher kommt das Zufallsexperiment seinem Erwartungswert. Die Art der Reflexion wird mittels *Russischem Roulette* ermittelt, welches in Kapitel 3.3.2.1 vorgestellt wird.

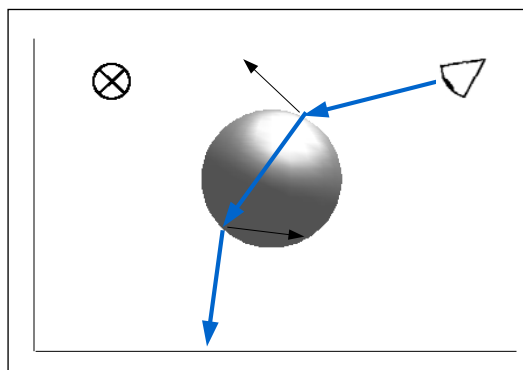


Abb. 2.4: Nur ein Pfad wird verfolgt (*Path*)

3. Photon Mapping

Die Motivation für Photon Mapping entstand durch zwei Überlegungen: Zum einen sollte es möglich sein, jede Art von Geometrie effizient und photorealistisch darstellen zu können, und zum anderen sollte jede Art von BRDF während des Renderns in Betracht gezogen werden können. Die Vorteile herkömmlicher Monte-Carlo basierter Verfahren sollten für ein neues Verfahren ausgenutzt und gleichzeitig die Nachteile ausgemerzt werden:

Vorteile	Nachteile
Darstellung beliebiger, komplexer Geometrie (ohne Vorberechnung des Lichtaustausches / Triangulierung / Meshing)	
Darstellung globaler Beleuchtung und deren Erscheinungsformen (Color Bleeding / Kaustiken / Halbschatten / Motion Blur)	Lange Rechenzeiten für brauchbare Resultate (insbesondere für Kaustiken)
Photometrisch korrekte Resultate	Artefakte durch Rauschen (Beseitigung sehr teuer)
Geringer Speicherbedarf	

Tab. 3.1: Vor- und Nachteile von Monte-Carlo basiertem Raytracing

Des Weiteren müssen für die korrekte Darstellung globaler Beleuchtung zwei Pfade betrachtet werden: Der des Lichtes zu den Objekten der Szene und der des Betrachters zu den Objekten der Szene. Beide Pfade sollen dabei unabhängig voneinander berechnet werden. Wenn man den Weg des Lichtes verfolgt, fällt auf, dass größere, diffuse Flächen oftmals sehr weich und homogen beleuchtet werden. Auf diesen Flächen kommt also ein großer Anteil des Lichtes an. Diesen Sachverhalt macht sich *Photon Mapping* zu nutze: Die Beleuchtung der Szene soll von der Geometrie entkoppelt und separat gespeichert werden, um jederzeit Zugriff auf diese Daten zu haben, ohne sie nochmals berechnen zu müssen. Die Beleuchtungsinformationen werden folglich nur auf Lambertschen Flächen festgehalten. Wie ein Sehstrahl auch wird Licht von spekularen Flächen reflektiert oder refraktiert, oder – und dies von allen Arten von Flächen – absorbiert.

Eine weitere Überlegung geht hin zur Repräsentation des Lichtstroms. Ein Lichtstrom besteht aus unendlich vielen, kleinen Partikeln, den Photonen. Jedes Photon trägt einen gewissen Prozentsatz des Lichtstroms in die Szene und verteilt seine Energie. Anders als ein Sehstrahl, der eine bestimmte Länge hat, kann der Weg eines Photons unendlich lang sein. Was sich verändert, ist der Energiegehalt, der von Reflexion zu Reflexion abnimmt, wenn das Photon einen Teil seiner Energie abgibt. Zur geometrischen Repräsentation eines Photons eignen sich Punkte. Um den Pfad, den jedes Photon verfolgt hat, nachvollziehen zu können und auch für Nicht-Lambertsche Oberflächen nützlich zu machen, wird jedem Photon die Richtung mitgegeben, von der es gekommen ist.

Der große Nachteil herkömmlicher Monte-Carlo basierten Techniken ist die Entstehung von Varianz – das Rauschen. Um dies zu verhindern, werden zu einem Schnittpunkt mehr Informationen benötigt als die direkte Beleuchtung und die Beleuchtung durch andere Flächen. Ebenso leistet ein einzelnes Photonenpartikel keinen wesentlichen Beitrag zur ausgehenden Leuchtdichte. Daher müssen an diesem Punkt mehr Photonen aus der Umgebung in die Berechnung der finalen Leuchtdichte mit einbezogen werden. Innerhalb eines bestimmten Radius ist daher die *Dichte* an Photonen abzuschätzen und dem Schnittpunkt zuzuordnen. Der Nachteil dieses Vorgehens liegt aber darin, dass das Ergebnis

photometrisch an Korrektheit verliert. Je mehr Photonen allerdings verschossen werden und in der Photon Map gespeichert sind, desto korrekter werden die Ergebnisse.

3.2 Einbindung der Rendering Equation

Derjenige Parameter, der an jedem Schnittpunkt p zu bestimmen ist, ist der der ausgehenden Leuchtdichte. Die ausgehende Leuchtdichte L_o eines Punktes p setzt sich aus der an diesem Punkt emittierten Leuchtdichte L_e und der von diesem Punkt reflektierten Leuchtdichte L_r zusammen und ergibt:

$$L_o(p, \bar{w}_o) = L_e(p, \bar{w}_o) + L_r(p, \bar{w}_o) \quad (3.1)$$

mit

- $\bar{w}_o$ Ausfallsrichtung

Die reflektierte Leuchtdichte ist wiederum ein Integral von allen eingehenden Leuchtdichten, den zugehörigen BRDF und den Einfallswinkeln über alle Einfallsrichtungen:

$$L_r(p, \bar{w}_o) = \int_{\Omega} f_r(p, \bar{w}_i, \bar{w}_o) L_i(p, \bar{w}_i) (\bar{w}_i \cdot \bar{n}) d\bar{w}_i \quad (3.2)$$

mit

- f_r BRDF
- L_i eingehende Leuchtdichte
- $\bar{w}_i$ Einfallsrichtung
- $\bar{n}$ Normale

Die BRDF kann nochmals in einen diffusen und einen spekularen Term aufgeteilt werden:

$$f_r(p, w_i, w_o) = f_{r,S}(p, w_i, w_o) + f_{r,D}(p, w_i, w_o) \quad (3.3)$$

mit

- $f_{r,S}$ spekulärer Anteil
- $f_{r,D}$ diffuser Anteil

Die eingehende Leuchtdichte besteht ebenso aus mehreren Komponenten:

$$L_i(p, \bar{w}_i) = L_{i,l}(p, \bar{w}_i) + L_{i,c}(p, \bar{w}_i) + L_{i,d}(p, \bar{w}_i) \quad (3.4)$$

mit

- $L_{i,l}(p, \bar{w}_i)$ direkte Beleuchtung
- $L_{i,c}(p, \bar{w}_i)$ indirekte Beleuchtung durch spekulare Reflexion oder Transmission
- $L_{i,d}(p, \bar{w}_i)$ indirekte Beleuchtung durch diffuse Reflexion

Die Komponente $L_{i,c}$ ist dabei direkt verantwortlich für die Entstehung von Kaustiken und benötigt mindestens eine Reflexion oder Refraktion, bevor sie in die Gleichung mit aufgenommen wird.

Die einzelnen Anteile der Rendering Equation, insbesondere die der reflektierten Leuchtdichte, werden bei der Analyse des *Second Pass* noch einmal genauer betrachtet, um festzustellen, in welchem Abschnitt des Algorithmus welcher Teil berechnet wird. Daher wird Gleichung (3.2) nochmals in die einzelnen Beleuchtungsterme aufgeteilt und es entsteht Gleichung (3.5):

$$\begin{aligned}
 L_r(p, w_o) = & \int_{\Omega} f_r(p, \bar{w}_i, \bar{w}_o) L_{i,l}(p, \bar{w}_i) (\bar{w}_i \cdot \bar{n}) d\bar{w}_i + \\
 & \int_{\Omega} f_{r,s}(p, \bar{w}_i, \bar{w}_o) (L_{i,c}(p, \bar{w}_i) + L_{i,d}(p, \bar{w}_i)) (\bar{w}_i \cdot \bar{n}) d\bar{w}_i + \\
 & \int_{\Omega} f_{r,D}(p, \bar{w}_i, \bar{w}_o) L_{i,c}(p, \bar{w}_i) (\bar{w}_i \cdot \bar{n}) d\bar{w}_i + \\
 & \int_{\Omega} f_{r,D}(p, \bar{w}_i, \bar{w}_o) L_{i,d}(p, \bar{w}_i) (\bar{w}_i \cdot \bar{n}) d\bar{w}_i
 \end{aligned} \tag{3.5}$$

3.3 First Pass: Photon Tracing

Als *First Pass* bezeichnet man das Aussenden und Verfolgen der Photonen durch die Szene von der Lichtquelle aus. Der Weg der Photonen dauert so lange, bis sie die von der Lichtquelle abgegebene Energie in der Szene verteilt haben und schließlich absorbiert werden.

3.3.1 Emission

Bevor die Photonen ihre Energie an die Szene abgeben können, müssen sie von den Lichtquellen generiert werden. Jedes Photon trägt dabei einen Bruchteil der Energie. Anders als in realen Beleuchtungsverhältnissen ist die Anzahl der Photonen, die eine Lichtquelle aussenden kann, durch den Algorithmus beschränkt. Für eine Szene ist die Menge im vorhinein durch Parameterübergabe festgelegt und wird je nach Lichtstrom auf die Lichtquellen aufgeteilt. Allerdings lässt sich dieser Wert beliebig erhöhen.

Um die Photonen zu generieren, werden Methoden des Monte-Carlo-Sampling verwendet. Jeder Lichtquellentypus nutzt dabei eine andere Strategie, die im folgenden beschrieben werden sollen.

3.3.1.1 Punktlichtquellen

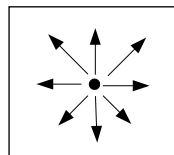


Abb. 3.1: Uniform samplende Punktlichtquelle

Punktlichtquellen sind die einfachste Form von Lichtquellen. Die Photonen werden uniform in alle Richtungen ausgesendet. Zwei Methoden können angewandt werden, um die Richtungen zu

bestimmen: *explicit sampling* und *rejection sampling*. Bei *explicit sampling* wird um die Position der Lichtquelle eine Einheitskugel aufgebaut und die zufallsbestimmten Zahlen auf die Oberfläche der Kugel projiziert. Bei *rejection sampling* wird überprüft, ob die innerhalb des Einheitswürfels durch die Zufallszahlen bestimmte Richtung auch innerhalb der Einheitskugel liegt. Alle übrigen Richtungen werden abgewiesen. Der Pseudocode zu letzterem – und in dieser Studienarbeit angewandten – Verfahren ist in Algorithmus 3.1 abgebildet.

```
sample_photon_directions_for_pointlight
{
    generated_photons = 0;
    while(generated_photons < photons_for_pointlight)
    {
        do{
            x_dir = 2*((Real)rand() / RAND_MAX) - 1;
            y_dir = 2*((Real)rand() / RAND_MAX) - 1;
            z_dir = 2*((Real)rand() / RAND_MAX) - 1;
        }
        while(x_dir * x_dir + y_dir * y_dir + z_dir * z_dir > 1);
        save_sample(pointlight_origin, direction(x_dir, y_dir, z_dir);
        generated_photons++;
    }
}
```

Algorithmus 3.1: Pseudocode zu *rejection sampling* für Punktlichtquellen

3.3.1.2 Spotlights

Im Grunde sind Spotlights gerichtete Punktlichtquellen mit einem beschränkten Öffnungswinkel und einem Richtungsvektor. Die Photonen werden somit nicht mehr uniform in alle Richtungen emittiert. Der Richtungsvektor beschreibt die mittlere Achse des Lichtkegels entlang derer die meisten Photonen ausgesendet werden, denn der Lichtstrom nimmt zum äußeren Rand des Kegels hin ab. Der Öffnungswinkel ist an dieser mittleren Achse anzusetzen, mit der Position der Lichtquelle als Scheitelpunkt. Für die auszusendenden Photonen muss nun gewährleistet sein, dass sie innerhalb des Kegels die Lichtquelle verlassen.

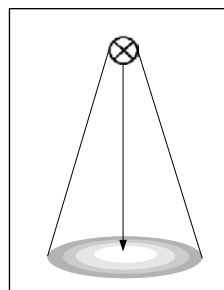


Abb. 3.2: Spotlight

Für diese Art des Sampling existieren zwei Ansätze: Eine direkte und eine indirekte Berechnung der Ausgangsrichtung. Die indirekte basiert auf dem *rejection sampling* der Punktlichtquelle und überprüft zusätzlich, ob der gefundene Vektor auch innerhalb des Lichtkegels liegt. Dies führt natürlich dazu,

dass umso mehr Zufallsberechnungen notwendig sind, je kleiner der Öffnungswinkel ist, da viele Berechnungen verworfen werden müssen. Der direkte Ansatz basiert auf dem *explicit sampling* und nimmt Öffnungswinkel und Richtungsvektor als bestimmende Größen mit in die Berechnung auf [Weizel03]. Für den ersten Ansatz benötigt man zwei Abfragen: Zum einen muss der Winkel zwischen Zufallsrichtung und Richtungsvektor der Lichtquelle bestimmt werden und zum anderen müssen dieser Winkel und der Öffnungswinkel der Lichtquelle miteinander verglichen werden. Da der zweite Ansatz effizienter ist, wird dieser im folgenden näher beschrieben.

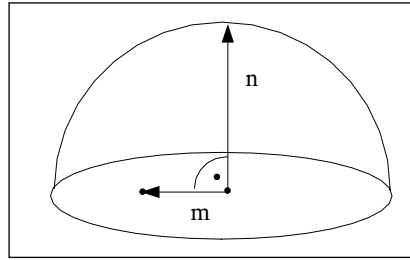


Abb. 3.3: Sampling einer Orthogonalen

$$\text{mit } \langle \vec{n}, \vec{m} \rangle = 0$$

Der Richtungsvektor $\vec{v}$ dient als Normale $\vec{n}$ einer gedachten Kreisscheibe mit Einheitsradius, $\vec{n}$ muss allerdings normalisiert werden. Der Startpunkt der Normalen ist zugleich Position der Lichtquelle und Mittelpunkt der Kreisscheibe. Somit bauen Kreisscheibe und Normale eine Hemisphäre (Halbkugel) rund um das Spotlight auf. Als erster Schritt wird für $\vec{n}$ eine Orthogonale $\vec{m}$ mit zufälliger Länge $[0, 1]$ auf der Kreisscheibe bestimmt. Dabei wird sich die Bedingung $\langle \vec{n}, \vec{m} \rangle = 0$ zu Nutze gemacht: ist eine der xyz -Koordinaten von $\vec{n} \neq 0$, dann werden die übrigen zwei korrespondierenden Komponenten von $\vec{m}$ zufällig im Intervall $[0, 1]$ gewählt und mit diesen beiden nach der fehlenden letzten Koordinate aufgelöst, so dass am Ende die gewünschte Bedingung erfüllt ist. Als nächstes wird ein zufälliger Winkel θ im Intervall $[0, \alpha]$ gewählt – mit α als bekanntem Öffnungswinkel –, der an $\vec{n}$ aufgetragen einen weiteren Vektor $\vec{p}$ aufspannt. $\vec{p}$ wiederum zeigt auf die Oberfläche der Hemisphäre und projiziert dort einen Punkt p . Mit Hilfe des Winkels θ lassen sich nun zum einen die Gegenkathete $\vec{x}$ und die Ankathete $\vec{y}$ zu $\vec{p}$ und $\vec{n}$ über $\sin$ und $\cos$ berechnen. Die Länge von $\vec{x}$ entspricht dem Radius, den die Unterseite des Kegels mit normalisierter Richtung $[0, 1]$ haben würde. Durch $\vec{x}$ kann p im folgenden auf die Normale $\vec{n}$ projiziert werden. Daraufhin werden $\vec{m}$ mittels der Länge von $\vec{x}$ und $\vec{n}$ mittels der Länge von $\vec{y}$ skaliert. p ist folglich das gesuchte Sample auf der durch den Lichtkegel beschränkten Hemisphäre.

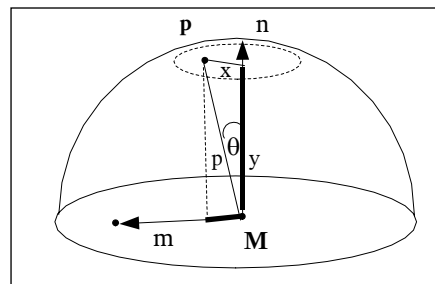


Abb. 3.4: Sampling der Spotlighthemisphäre

3.3.1.3 Flächenlichtquellen

Flächenlichtquellen sind nicht durch einen eindeutigen Ursprung beschrieben, sondern werden von einer Reihe von Punkten aufgespannt. Daher müssen zuerst zufällige Punkte auf deren Oberfläche bestimmt werden, von denen aus wiederum zufällige Richtungen gewählt werden. Das Sampling der Oberflächenpunkte ist somit abhängig von der Art der Grundfläche. Im einfachsten Fall ist dies ein Dreieck, dass durch seine Eckpunkte und seine Normale beschrieben wird. Die im folgenden verwendeten Flächenlichtquellen sind Lambert-Strahler.

Shirley schlägt zum Sampling von Punkten auf einer Dreiecksfläche folgendes Vorgehen vor [Shirley03]: Zwei Zufallszahlen ξ_1 und ξ_2 werden mittels Monte-Carlo-Methoden in die Koeffizienten u und v transformiert und in die Ebenengleichung des Dreiecks eingesetzt. Es resultiert der Punkt p , der sich immer noch auf dieser Ebene befindet:

$$p = p_0 + u * (p_1 - p_0) + v * (p_2 - p_0) \quad (3.6)$$

mit

- p_n Eckpunkte des Dreiecks
- $u = 1 - \sqrt{1 - \xi_1}$
- $v = (1 - u) * \xi_2$

Ist ein zufälliger Punkt auf der Oberfläche gefunden, muss noch die Richtung bestimmt werden. Die Wahrscheinlichkeit der Richtungen soll proportional zum Cosinus des Ausgangswinkels zwischen Oberflächennormalen und Ausgangsrichtung sein und ergibt sich nach Transformation mit COS-Verteilung gesamelter Polarkoordinaten in karthesische Koordinaten:

$$\begin{aligned} x &= \sin(\theta) * \cos(\phi) \\ y &= \cos(\theta) \\ z &= \sin(\theta) * \sin(\phi) \end{aligned} \quad (3.7)$$

mit

- $\theta = \cos^{-1}(\sqrt{1 - \xi})$
- $\phi = 2\pi * \xi$
- ξ Zufallszahl im Intervall $[0, 1]$

Die Photonen werden folglich mit höherer Wahrscheinlichkeit in Richtung der Normalen ausgesendet. Der Richtungsvektor ergibt sich schließlich aus dem Produkt von Oberflächensample und dem normalisierten Vektor aus (x, y, z) .

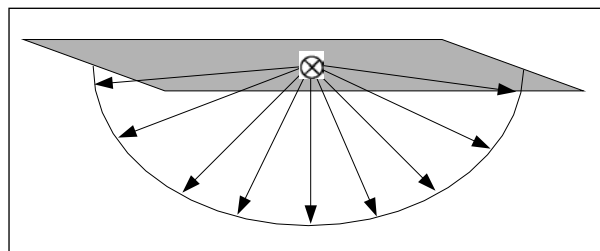


Abb. 3.5: Punkte- und Richtungssampling einer Flächenlichtquelle (hier: nur ein Punktesample)

3.3.2 Scattering

Photonen werden durch die Szene verfolgt und genauso behandelt wie rudimentäre Sehstrahlen. Sie werden reflektiert, refraktiert oder absorbiert – mit einem wesentlichen Unterschied: Anstatt Leuchtdichte aufzusammeln wird der Lichtstrom der Lichtquellen verteilt. Ihre Energie wird von den Reflexionsgraden der Materialien bei jeder Reflexion manipuliert. Auf ihrer Flugbahn können sie höchsten Energie verlieren, aber nie welche dazugewinnen.

Nun stellt sich die Frage, wie der Typ der Reflexion bestimmt werden kann, da ein Material schließlich auch unterschiedliche Parameter besitzen kann und sein Typus nicht eindeutig bestimmt ist. Die Methode hierfür nennt sich *Russisches Roulette*: Die Reflexion wird zufällig anhand der Grade als Intervallbegrenzungen ausgewählt – je höher ein Parameterwert, desto wahrscheinlicher dieser Typ der Reflexion. In den nächsten Absätzen werden die verschiedenen Reflexionsarten für Photonen vorgestellt.

3.3.2.1 Russisches Roulette

Russisches Roulette ist eine Monte-Carlo-Technik, die insbesondere beim *Path Tracing* verwendet wird. Nichts anderes passiert bei der Auswahl der Reflexionsart für Photonen. Ein Material ist zu bestimmten, festgelegten Prozentsätzen diffus, spekulare und refraktierend. Nach den Regeln der Stochastik werden die Photonen zu diesen Prozentsätzen auch reflektiert. Die *wichtigen* Reflexionstypen werden somit bevorzugt. Diese Technik nennt man auch *importance-sampling*. Abbildung 3.6 verdeutlicht dieses Vorgehen.

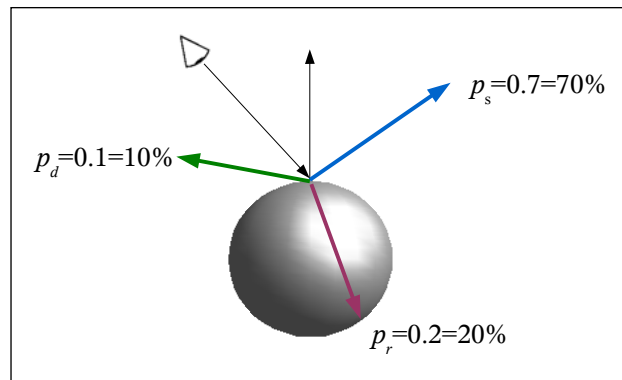


Abb. 3.6: Roulettewerte für Reflexion

Seien nun der diffuse Reflexionsgrad p_d , der spekulare Reflexionsgrad p_s und der refraktierende Reflexionsgrad p_r für das Material festgelegt. Dann ergibt sich die Art der Reflexion anhand der uniform bestimmten Zufallsvariable ξ mit $\xi \in [0,1]$ aus

$$\begin{array}{ll}
 \xi \in [0, p_d] & \Rightarrow \text{diffuse Reflexion} \\
 \xi \in]p_d, p_d + p_s] & \Rightarrow \text{spekulare Reflexion} \\
 \xi \in]p_d + p_s, p_d + p_s + p_r] & \Rightarrow \text{Refraktion} \\
 \xi \in]p_d + p_s + p_r, 1] & \Rightarrow \text{Absorption}
 \end{array} \tag{3.8}$$

Durch die Verfolgung der Photonen nach Wichtigkeit des Reflexionstyps ergibt sich stochastisch gesehen immer das korrekte Resultat. Nichtsdestotrotz erhöht sich die Korrektheit der Wahrscheinlichkeitsfunktion, umso mehr Photonen das Material treffen. Für den *Second Pass* ist dieser Sachverhalt besonders wichtig: Bei zu wenigen Strahlen pro Pixel entsteht trotz der Leuchtdichten der PhotonMap immer noch Rauschen, allerdings verringern sich die Fehler rapide bei leichter Erhöhung der Strahlenanzahl, da die *Radiance Estimate* diesen Nachteil ausgleicht.

3.3.2.2 Diffuse Reflexion

Trifft ein Photon auf eine diffuse Fläche, so wird es zunächst in der PhotonMap gespeichert und anschließend reflektiert. Die Richtung ergibt sich durch die Projektion eines zufällig gefundenen Vektors auf die Hemisphäre rund um die Schnittpunktnormale. Zwei Varianten sind möglich, um diese Richtung zu bestimmen:

Polarkoordinaten θ und ϕ

Seien ξ_1 und ξ_2 uniform verteilte Zufallsvariablen, dann ergibt sich die Ausfallsrichtung $\vec{w}_o$ aus dem Raumwinkel θ zur Schnittpunktnormalen und dem Raumwinkel ϕ als Rotationswinkel um die Normale mit

$$\vec{w}_o = (\theta, \phi) = \left(\cos^{-1} \left(\sqrt{1 - \xi_1} \right), 2\pi\xi_2 \right) \quad (3.9)$$

Orthogonalität und Vektoraddition

Dieser Ansatz ähnelt dem des Sampling zufälliger Richtungen für Spotlights. Zu der Normalen $\vec{n}$ des Schnittpunktes wird eine Orthogonale $\vec{m}$ anhand des Skalarproduktgesetzes $\langle \vec{n}, \vec{m} \rangle = 0$ gesucht. Durch Vektoraddition von $\vec{n}$ und $\vec{m}$ ergibt sich die auf die Hemisphäre zu dem Ursprung von $\vec{n}$ projizierte Ausfallsrichtung $\vec{w}_o$.

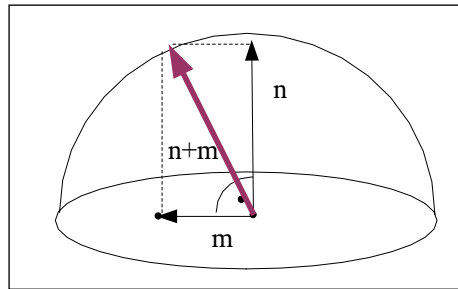


Abb. 3.7: Sampling der Schnittpunkthemisphäre

Da Cosinus-Berechnungen und Wurfelfunktionen sehr teuer sind, ist der zweite Ansatz intuitiver. Die Wahrscheinlichkeit der Ausfallsrichtung $\vec{w}_o$ ist proportional zu dem Cosinus zwischen der Normalen $\vec{n}$ und der gesampleten Richtung $\vec{w}_o$. Mit den Polarkoordinaten wird diese Wahrscheinlichkeit implizit mit eingebunden, im zweiten Ansatz muss die von $\vec{w}_o$ aufgenommene Leuchtdichte im nachhinein mit einem Cosinus-Faktor gewichtet werden.

Die Energie des reflektierten Photons ergibt sich aus der Leuchtdichte des einfallenden Photons, skaliert mit dem diffusen Reflexionsgrad des Materials.

3.3.2.3 Spekulare Reflexion

Trifft ein Photon auf eine spekulare – d.h. spiegelnde – Fläche, so gilt das Gleichheitsprinzip Eingangsrichtung gleich Ausgangsrichtung. Die Ausgangsrichtung $\vec{w}_o$ eines Photons ergibt sich durch die *perfekte Spiegelung* der von der Oberfläche wegzeigenden Einfallsrichtung $\vec{w}_i$ an der Schnittpunktnormalen $\vec{n}$:

$$\vec{w}_o = 2 * (\vec{n} \cdot \vec{w}_i) * \vec{n} - \vec{w}_i \quad (3.10)$$

Die Energie des reflektierten Photons ergibt sich aus der Leuchtdichte des einfallenden Photons, skaliert mit dem spekularen Reflexionsgrad des Materials.

3.3.2.4 Transmission

Trifft ein Photon auf eine das Licht brechende Fläche, so ist der Grad der Refraktion hauptsächlich abhängig von dem Refraktionsindex η_1 des Eintrittsmaterials und dem Refraktionsindex η_2 des Austrittsmaterials. In der Literatur finden sich diverse unterschiedliche Herleitungen der Gleichung zur Berechnung des refraktierten Photons. Nach Jensen ergibt sich die gesuchte Richtung $\vec{w}_r$ aus Abbildung 3.8 und folgender Herleitung:

$$\vec{w}_r = -\frac{\eta_1}{\eta_2} (\vec{w}_i - (\vec{w}_i \cdot \vec{n}) \vec{n}) - \left(\sqrt{1 - \left(\frac{\eta_1}{\eta_2} \right)^2 (1 - (\vec{w}_i \cdot \vec{n})^2)} \right) \vec{n} \quad (3.11)$$

mit

- $\vec{n}$ Normale am Schnittpunkt
- $\vec{w}_i$ von der Oberfläche wegzeigende Eintrittsrichtung

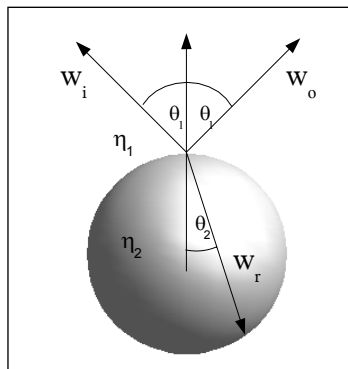


Abb. 3.8: Refraktion, berechnet mit Snell's Gesetz $\eta_1 \sin \theta_1 = \eta_2 \sin \theta_2$

Die Energie des refraktierten Photons ergibt sich aus der Leuchtdichte des einfallenden Photons, skaliert entweder mit dem refraktierenden Reflexionsgrad des Materials, oder mit dem Reflexionsgrad $1 - F_r$, mit F_r als der Fresnel-Approximation.

3.3.3 Abspeicherung in der Photon Map

Jedes Mal wenn ein Photon eine diffuse Fläche trifft, wird es mit seiner Energie – der Anteil des Strahlungsflusses der Lichtquelle – in der Photon Map gespeichert und ein neues Photon mit manipuliertem Energiewert wird erzeugt und in die reflektierte Richtung gesendet. Photonen, die spekulär reflektiert oder refraktiert werden, werden nicht abgelegt. Der Grund dafür ist, dass – wenn man sich einmal diese Kategorie von Materialien näher betrachtet – die Beleuchtung der Umgebung sich auf und in den Objekten dieser Materialien widerspiegelt. Folglich nehmen diese die Informationen der Szene bereits beim herkömmlichen Raytracing mit auf.

Bevor ein Photon in die Photon Map aufgenommen wird, kann es verschiedene Lichtpfade verfolgt haben. Zum einen gibt es solche Photonen, die von der Lichtquelle ausgesendet werden, direkt auf diffusen Flächen landen und sich dort der Energieanteil am Lichtstrom gemerkt wird, und zum anderen jene, die vorher ein- oder mehrfach von spekulären Flächen reflektiert oder refraktiert wurden. Beide Arten zusammengefasst nennt man *Global Photons*, letztere allein nennt man *Caustic Photons*. Wie der Name schon sagt, erzeugen die *Caustic Photons* die für diese Studienarbeit interessanten Kaustiken. Die Konzentration der Photonen an dieser Stelle ist dabei besonders hoch. Nun ist es aber leider sehr unwahrscheinlich, dass beispielsweise von einer Punktlichtquelle uniform versendete Photonen hauptsächlich in die Richtung spekulärer Materialien wandern und nach ihrer Reflexion oder Refraktion dichte Ansammlungen bilden. Um den Effekt zu forcieren, muss die Emission der Photonen also manipuliert werden. Das kann durch die Verwendung entweder richtungsorientierter Spotlights oder präferierter Photonen, den schon beschriebenen *Caustic Photons* geschehen. Da beide Photonenarten aber eine Schnittmenge bilden, die beim Rendering zu einer fehlerhaften Darstellung des Bildes führen kann, müssen sie separat emittiert und gespeichert werden. Dadurch entstehen zwei Arten von Photon Maps: die *Global Photon Map* und die *Caustics Photon Map*.

3.3.3.1 Global Photon Map

Die *Global Photon Map* speichert alle Photonen, die auf diffusen Flächen auftreffen. Sie repräsentiert jegliche Art von Beleuchtung: Direkte, indirekte und die Kaustiken erzeugende indirekte Beleuchtung.

$$L(S|D) * DA$$

Ein Photon wird nach jedem Treffer reflektiert.

3.3.3.2 Caustic Photon Map

Die *Caustic Photon Map* speichert alle Photonen, die auf diffusen Flächen auftreffen und zuvor von mindestens einer spekulären Fläche reflektiert worden sind. Im Gegensatz zu den *Global Photons* werden die *Caustic Photons* von der Lichtquelle nicht in alle Richtungen emittiert, sondern nur in die Richtung spekulärer Flächen. Es gibt mehrere Überlegungen, die Richtungen dieser Photonen ausfindig zu machen. In Kapitel 5 und 7 werden die Ideen näher erläutert.

$$LS+D$$

Nachdem ein Photon gespeichert wurde, wird es absorbiert und gelöscht.

3.1 Architektur der Photon Map

Wie eigentlich jedes auf Geschwindigkeit bedachte System muss die Photon Map zwei wichtige Kriterien bewerkstelligen können: Effizienz und Speicherbedarf. Gewünscht ist ebenso eine korrekte Darstellung des gerenderten Bildes, welche sich proportional zu der Anzahl emittierter und gespeicherter Photonen verhält. Die Größe eines Photons ist also wesentlich für die Größe der Photon Map. Daher ist die Struktur eines Photons wie in Algorithmus 3.1 skizziert aufgebaut.

```
struct Photon
{
    float pos[3];
    short plane;
    unsigned char theta, phi;
    float power[3];
} Photon;
```

Algorithmus 3.2: Aufbau eines Photons

Ein Photon ist idealerweise insgesamt 27 Byte groß und besteht aus seiner Trefferposition, seiner Raumachse im – noch vorzustellenden – *kd-Tree*, zwei komprimierten Raumwinkeln θ und ϕ für seine Einfallsrichtung und seinem Anteil am Strahlungsfluss der Lichtquelle (*Energie*). Bei einer angemessenen Photonenanzahl von etwa 1 Mio. wäre der Speicherbedarf der Photon Map dann etwa 27 MByte.

Da die Photonen mehr oder weniger uniform und zufällig verteilt durch die Szene wandern, aber – wie man später sehen wird – sequentiell nach ihrer Immission gespeichert werden, muss die Speicherarchitektur der Photon Map die Photonen effizient verwalten können. Denn zum einen sollen die Photonen schnell in der Photon Map abgelegt werden, und zum anderen soll später schnell nach bestimmten Photonenpositionen gesucht werden. Für den ersten Teil sollten die Photonen daher sequentiell ohne weiteren Such- oder Sortieraufwand in einem eindimensionalen Array gespeichert werden, und für den zweiten muss eine Architektur gefunden werden, die speichereffizient und einfach zu durchsuchen ist und mit dreidimensionalen Punkten umgehen kann. Am besten eignet sich dafür der *kd-Tree*.

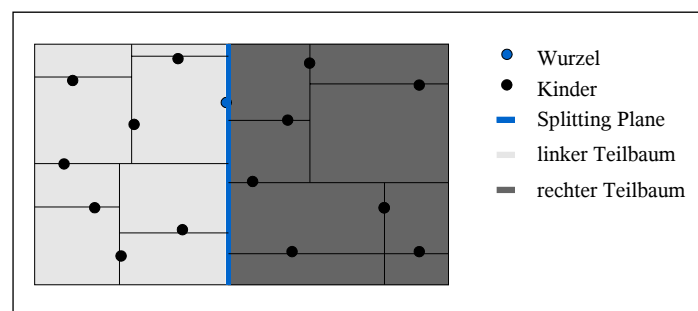


Abb. 3.9: Aufbau eines 2D-Kd-Tree

Der *kd-Tree* ist ein binärer Baum in k Dimensionen. Jeder Knoten des Baums besitzt jeweils einen Zeiger auf den linken und den rechten Teilbaum. Alle Knoten außer den Blättern besitzen zudem eine achsen-orthogonale Raumachse zur Aufteilung einer Dimension (x , y oder z) in zwei separate Halbräume. Alle Photonen des linken Teilbaums sind unterhalb, alle des rechten befinden sich

oberhalb der Trennebene. Bevor das Photonarray also durchsucht werden kann, wird es in einen *kd-Tree* umgewandelt. Hierzu wird das kleinste Volumen gesucht, das alle Photonen einschließt (*Bounding Box*) und dessen Ausdehnung in allen Dimensionen bestimmt. Die Dimension mit der größten Ausdehnung wird diejenige Raumachse, nach welcher die Photonen sortiert werden. Aus dem Array wird das im Median liegende Photon ausgewählt und zum ersten Wurzelknoten gemacht. Die Sortierung startet nun dadurch, dass alle Photonen mit kleinerem Wert in der Dimension in den linken Teilbaum und diejenigen mit größerem in den rechten Teilbaum einsortiert werden. Beide Teilbäume werden rekursiv weiter gesplittet und sortiert, bis alle Photonen behandelt worden sind. Da man davon ausgehen muss, dass der Wurzelknoten nicht der Median aller Photonen ist, muss der Baum aus Effizienzgründen für die Suche noch balanciert werden.

Der Aufwand für die Suche eines Photons in einem balancierten *kd-Tree* mit n Photonen beträgt $O(\log n)$, in einem unbalancierten $O(n)$. Um die k nächsten Nachbarn eines Photons zu finden, wird ein Aufwand von $O(k + \log n)$ benötigt.

3.4 Second Pass: Rendering

Nachdem alle Photonen emittiert worden sind, wird die Szene per Raytracer gerendert. Jensen schlägt dazu einen *Distribution Raytracer* [Cook84] vor, der für die Abschätzung der Leuchtdichte eines Pixels mehrere Sehstrahlen von der Kamera aus versendet und für die indirekte diffuse Beleuchtung von der Schnittpunktposition aus auch mehrere Sekundärstrahlen losschickt, oder – falls die Sekundärstrahlen auf diffuse Flächen treffen – die Abschätzung der Leuchtdichte aus der Photon Map verwendet. Der Aufwand ist – wie man vermuten kann – allerdings sehr hoch. Ähnlich verfährt auch ein *Path Tracer*, mit dem Unterschied, dass von einem Schnittpunkt aus nur ein weiterer Sekundärstrahl verfolgt wird. Das erhöht zwar die Fehlerquote, ist aber wesentlich effizienter in der Geschwindigkeit. Zudem sind durch die Leuchtdichten in der Photon Map viele Störungen schon bei einer leicht gehobenen Anzahl von Strahlen pro Pixel zu beheben.

3.4.1 Direkte Beleuchtung

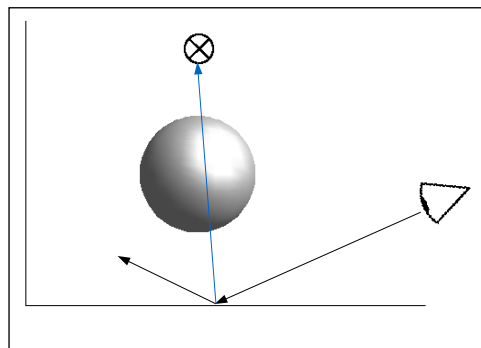


Abb. 3.10: Direkte Beleuchtung mit Tracing von Schattenfühlern

Die direkte Beleuchtung wird durch Einsammlung der Lichtströme aller Lichtquellen der Szene auf den Schnittpunkt bestimmt. Der Grad der Stärke ist abhängig vom Einfallswinkel, dem Typus der

Lichtquelle, der lokalen BRDF und dem Ergebnis des Schattenföhlers. Sie wird bestimmt durch folgenden Teil der Rendering Equation:

$$\int_{\Omega_p} f_r(p, \bar{w}_i, \bar{w}_o) L_{i,l}(p, \bar{w}_i) (\bar{w}_i, \bar{n}) d\bar{w}_i \quad (3.12)$$

3.4.2 Indirekte diffuse Beleuchtung

Indirekte diffuse Beleuchtung wird durch die Einsammlung der Leuchtdichten rekursiv versendeter Sekundärstrahlen von Schnittpunkt p und durch die Abschätzung der Leuchtdichte aus der *Global Photon Map* bestimmt. Sie ist zudem abhängig von der lokalen BRDF. Der entsprechende Term der Rendering Equation ist

$$\int_{\Omega_p} f_{r,D}(p, \bar{w}_i, \bar{w}_o) L_{i,d}(p, \bar{w}_i) (\bar{w}_i, \bar{n}) d\bar{w}_i \quad (3.13)$$

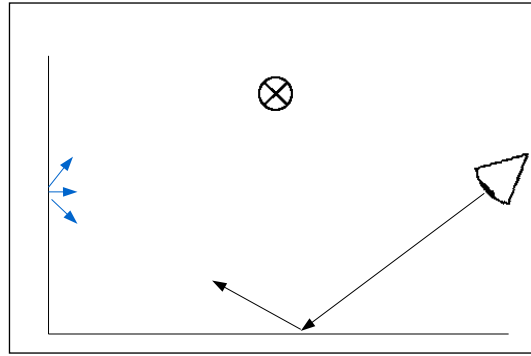


Abb. 3.11: Indirekte diffuse Beleuchtung

3.4.3 Indirekte spekulare Beleuchtung und Glanzreflexion

Indirekte spekulare Beleuchtung und Glanzreflexionen werden durch die Einsammlung der Leuchtdichten rekursiv versendeter Sekundärstrahlen von Schnittpunkt p bestimmt. Sie sind abhängig von der lokalen BRDF und alle Arten von spekularen Materialien werden berücksichtigt. Die Rendering Equation wird um einen weitere Teil vervollständigt:

$$\int_{\Omega_p} f_{r,S}(p, \bar{w}_i, \bar{w}_o) (L_{i,c}(p, \bar{w}_i) + L_{i,d}(p, \bar{w}_i)) (\bar{w}_i, \bar{n}) d\bar{w}_i \quad (3.14)$$

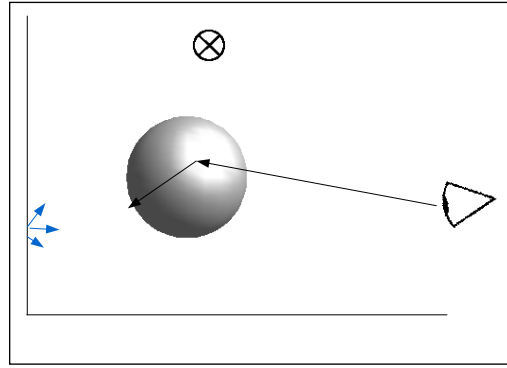


Abb. 3.12: Indirekte spekulare Beleuchtung

3.4.4 Kaustiken

Kaustiken werden in die Berechnung der Rendering Equation gesondert aufgenommen. Sie werden durch die Abschätzung der Leuchtdichte aus der *Caustic Photon Map* sichtbar. Da sie von spekularen Materialien verursacht werden und auf diffusen Flächen vom Betrachter aus direkt sichtbar sind, wird ihre Berechnung mit in die der indirekten diffusen Beleuchtung aufgenommen. Gültig ist hier der Term:

$$\int_{\Omega_p} f_{r,D}(p, \bar{w}_i, \bar{w}_o) L_{i,c}(p, \bar{w}_i) (\bar{w}_i, \bar{n}) d\bar{w}_i \quad (3.15)$$

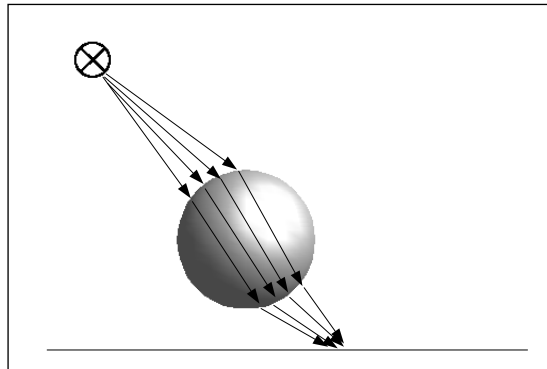


Abb. 3.13: Lichtpfade für Kaustiken

3.4.5 Radiance Estimate – Abschätzung der Leuchtdichte

Betrachtet man sich ein einzelnes Photon, so sieht man, dass seine Energie nahezu keine Auswirkung auf die Leuchtdichte eines Schnittpunktes haben würde. Aus diesem Grund liefern die Photon Maps während des *Second Pass* die *Dichte* der Photonen innerhalb einer festgelegten Region rund um den Schnittpunkt und somit auch eine Abschätzung der vorhandenen Leuchtdichte. Die Einfallsrichtung der Photonen ist eine bestimmende Größe bei der Einsammlung. Je ähnlicher sich die Photonen in dieser Hinsicht sind, desto eher werden sie in die Abschätzung mit aufgenommen.

Die Dichte der Photonen in einer Region ist bestimmt durch das Verhältnis von dem Strahlungsfluss der Lichtquelle zu der bestrahlten Fläche:

$$\frac{d\phi}{dA} \quad (3.16)$$

Die Idee des Radiance Estimate besteht nun darin, diesen Quotienten in die Berechnung der reflektierten Leuchtdichte L_r der Rendering Equation am Punkt p_x einzubinden. Der dementsprechende Term lautet

$$L_r(p_x, \bar{w}_o) = \int_{\Omega_{p_x}} f_r(p_x, \bar{w}_i, \bar{w}_o) L_i(p_x, \bar{w}_i) (\bar{w}_i \cdot \bar{n}_{p_x}) d\bar{w}_i \quad (3.17)$$

mit

- Ω_{p_x} die Hemisphäre aller Einfallrichtungen
- f_r BRDF
- L_i einfallende Leuchtdichte

Die Photon Map liefert allerdings den Anteil des Strahlungsflusses der Lichtquellen, den die Photonen durch die Szene getragen haben. Um diese Informationen für den Term nutzen zu können, muss die ungeklärte Größe L_i umgeformt werden:

$$L_i(p_x, \bar{w}_i) = \frac{d^2\phi_i(p_x, \bar{w}_i)}{(\bar{w}_i \cdot \bar{n}_{p_x}) d\bar{w}_i dA_i} \quad (3.18)$$

Das Verhältnis (3.18) ergibt sich aus der photometrischen Definition von L mit dem Verhältnis von Strahlungsfluss zu projizierten Fläche. Gleichung (3.18) in (3.17) eingesetzt ergibt nun

$$\begin{aligned} L_r(p_x, \bar{w}_o) &= \int_{\Omega_{p_x}} f_r(p_x, \bar{w}_i, \bar{w}_o) \frac{d^2\phi_i(p_x, \bar{w}_i)}{(\bar{w}_i \cdot \bar{n}_{p_x}) d\bar{w}_i dA_i} (\bar{w}_i \cdot \bar{n}_{p_x}) d\bar{w}_i \\ &= \int_{\Omega_{p_x}} f_r(p_x, \bar{w}_i, \bar{w}_o) \frac{d^2\phi_i(p_x, \bar{w}_i)}{dA_i} \end{aligned} \quad (3.19)$$

In (3.19) können nun tatsächlich schon die Daten der Photon Map eingetragen werden, allerdings nur als angenäherte Größe. Denn der einfallende Strahlungsfluss ϕ_i wird von denjenigen n Photonen repräsentiert, die sich am nächsten zur Position des Punktes p_x befinden. Der Anteil eines Photons am Strahlungsfluss – im folgenden Energie genannt – ist definiert als $\Delta\phi(\bar{w}_p)$, ist also abhängig von seiner Einfallrichtung $\bar{w}$. Geht man davon aus, dass nun jedes beliebige Photon p eine Fläche A am Punkt p_x trifft, so ergibt als Annäherung für die gesuchte Leuchtdichte L_r :

$$L_r(p_x, \bar{w}_p) \approx \sum_{p=1}^n f_r(p_x, \bar{w}_p, \bar{w}_o) \frac{\Delta\phi_p(p_x, \bar{w}_p)}{\Delta A} \quad (3.20)$$

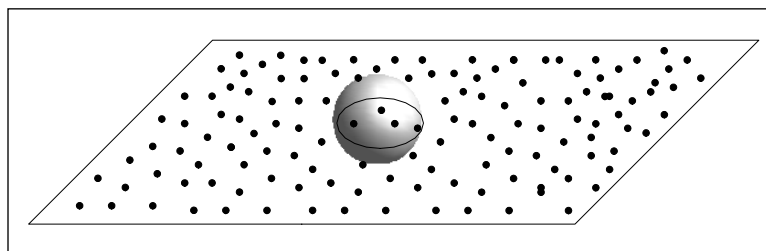


Abb. 3.14: Radiance Estimate mit Kugel als „Sammelkörper“ und projizierter Kreisfläche

Geht man nun davon aus, dass man innerhalb eines festgelegten Radius r nach den nächstgelegenen n Photonen sucht, um die Dichte zu bestimmen, so entsteht aus dem Punkt p_x als Mittelpunkt und r eine Kugel. Diese Kugel ist nun der sogenannte „Sammelkörper“. Allerdings bleiben Photonen nicht einfach in der Luft hängen, sondern landen auf ebenen, diffusen Flächen. Die abzuschätzende Dichte wäre also gering im Hinblick auf das Volumen einer Kugel. Aus diesem Grund wird sie auf eine Kreisfläche in Richtung der ebenen Fläche projiziert. Der sich verändernde Flächeninhalt ΔA lautet daher:

$$\Delta A = \pi r^2 \quad (3.21)$$

Aus den evaluierten Größen ergibt sich schließlich eine Gleichung, die die Daten der Photon Maps vollkommen ausnutzen kann und die reflektierte Leuchtdichte L_r adäquat berechnet:

$$L_r(p_x, \bar{w}_p) \approx \frac{1}{\pi r^2} \sum_{p=1}^n f_r(p_x, \bar{w}_p, \bar{w}_o) \Delta \phi_p(p_x, \bar{w}_p) \quad (3.22)$$

4. Dokumentation des Raytracers

Das in der AG Computergraphik der Universität Koblenz-Landau innerhalb eines Projektpraktikums im WS 2002 / 2003 entstandene Raytracing-System – bekannt unter dem Namen „Reitratze“ – ist das grundlegende System, auf dem diese Studienarbeit aufbaut. Es enthält neben rudimentären Raytracingalgorithmen bereits auch Methoden, um das Verfahren zu beschleunigen und den photorealistischen Eindruck der gerenderten Szenen zu erhöhen. Im folgenden eine Liste der wesentlichen Charakteristika des Systems:

- Kameramodelle *ThinLensCamera* und *PinholeCamera*
- Supersampling mit radialer und Poisson-Verteilung
- Einlesen von vormodellierten und in VRML konvertierten Szenen
- Beschleunigungsverfahren *UniformGrid*, *Octree*, *BSPTree* und *Bounding Volume Hierarchies*
- Parallelisierung
- Beleuchtungsmodelle nach Phong, Schlick und Blinn mit Einbindung des Fresnel-Terms
- Speicherung des gerenderten Bildes im PPM- oder PNG-Format

4.1 Modularisierung

Zur Gewährleistung späterer Erweiterungsmöglichkeiten ist das System objektorientiert programmiert und somit auch modularisiert. Um einen groben Überblick über die Bestandteile des Systems zu erhalten, werden die einzelnen Module im folgenden kurz charakterisiert. Eine ausführlichere Dokumentation der einzelnen Routinen erfolgt in Kapitel 4.2.

Main

Die Implementations-Datei *main.cpp* ist diejenige, die das Programm startet. Es werden globale Variablen initialisiert, die in der Kommandozeile übergebenen Parametern ausgelesen, die Szene geladen und der Raytracing-Algorithmus angestoßen. Zwei Hauptmethoden sorgen für eine getrennte Behandlung der standardmäßigen *seriellen* Verarbeitung der Routinen und der *parallelen* Version, in welcher die einzelnen *Worker* (Clients) eines zusammengeschalteten Clusters das in mehrere Pixelreihen aufgeteilte Bild getrennt rendern und ein *Manager* (Server) diese Partitionen wieder zu einem Gesamtbild zusammenfügt. Die *parallele* Komponente ist eines der wichtigsten Beschleunigungsverfahren, das im System implementiert ist, und führt bei großen Auflösungen zu einem hohen Performancegewinn.

Loader

Daten geometrischer Primitive und deren Materialeigenschaften, Kameramodelle und Lichtquellen für eine Szene werden in dieser Klasse durch Aufruf der jeweiligen Konstruktoren und Methoden der einzelnen Objekte definiert. Der größte Nachteil dieser Variante der Szenenrepräsentation ist, dass die Szenen *immer* im Quellcode festgehalten werden müssen und nicht dynamisch zur Laufzeit dem System mitgegeben werden können. Eine Alternative bietet das Modul *VPars* respektive *Vload*, welches eine in *VRML 97* definierte Szenendatei einlesen und in die interne Datenstruktur des Systems konvertieren kann. Da das Parser-Modul im Rahmen der Studienarbeit nicht an Photon Mapping

angepasst worden ist, wird es im folgenden nicht näher behandelt (hierzu gehören auch separat benötigte Klassen wie *TexJpgTga*, *PointEx* und *TriangleEx*).

Raytracer

Sobald die Szene anhand der Daten des Loader-Moduls definiert und generiert worden ist und alle notwendigen Daten aus der Kommandozeile ausgelesen sind, wird die eigentliche *Tracing*-Schleife angestoßen. Die Klasse *Raytracer* bietet Methoden zum Rendern des Gesamtbildes und partieller Teilbilder sowie zum Verfolgen von Primär- und Sekundärstrahlen durch die Szene.

Camera

Die Klasse *Camera* ist die Oberklasse zu allen im System vorhandenen Kameramodellen. Klassische Eigenschaften einer Instanz dieser Klasse sind die Definitionen von Up-Vektor, Look-At-Vektor, Position, Öffnungswinkel und *ImagePlane*, auf welche die Kamera ausgerichtet ist. Als zusätzliche Information für *Supersampling* definiert diese Klasse die Gewichtung der einzelnen Strahlen. Unterschieden werden die Kamerateypen in den Unterklassen *PinholeCamera* und *ThinLensCamera*.

SceneItem

Die abstrakte Klasse *SceneItem* bietet ein Interface für (fast) alle Bestandteile der Szene. Es werden diejenigen Methoden und Membervariablen festgelegt, die von allen Unterklassen unbedingt implementiert werden müssen.

GeomObject

Die Klasse *GeomObject* ist wie *SceneItem* abstrakt und erweitert jene um weitere Methoden und Eigenschaften zur Repräsentation geometrischer Primitive. Als Unterklassen repräsentieren *Sphere* Kugeln mit Mittelpunkt und Radius, *Triangle* Dreiecke, *Box* achsenparallele Quader und *Polygon* Polygone mit mindestens 3 Eckpunkten. Da sich die Klassen *Box* und *Polygon* zum Zeitpunkt der Beendigung des Projektpraktikums noch im Entwicklungsstadium befanden, werden sie im folgenden nicht näher beschrieben.

Ein besonderes Highlight ist die Klasse *Sphereflake*, die allerdings weder von *SceneItem* noch von *GeomObject* abgeleitet ist. Sie dient zur Darstellung von rekursiv aufgebauten „Kugelflocken“, die zum Testen der Effektivität der Beschleuniger aufgrund ihrer hohen Polygonanzahl sehr gut geeignet sind (siehe Abb. 4.1).

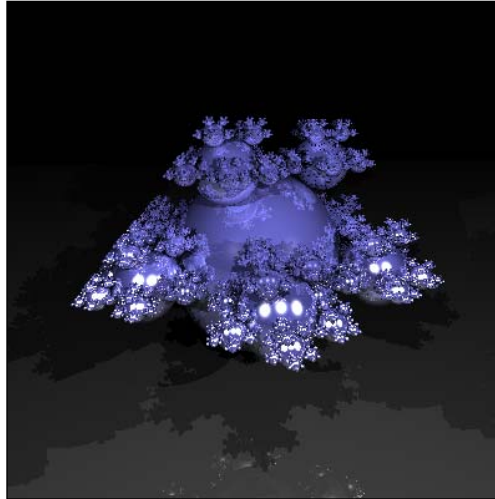


Abb. 4.1: Sphereflake mit Rekursionstiefe 4 und beginnendem Radius 2.0

Container

Da für das System anfangs eine *hybride* Rendering-Architektur gedacht und geplant war, werden alle geometrischen Primitive der Szene in einem sogenannten Container abgelegt. Der Container selbst kann wiederum weitere Container enthalten. Alle Beschleunigungsverfahren des Raytracers sind vom Typ Container. Zu diesen Beschleunigungsverfahren gehören im einzelnen *BSPTree*, *UniformGrid*, *Bounding Volume Hierarchies* und *Octree*. Die Verfahren werden im folgenden nicht näher erläutert, da sie für das Verständnis des Systems nicht weiter von Belang sind.

Image

Die Klasse *Image* ist die Oberklasse zu allen verwendeten Bildtypen des Systems. Hauptsächlich wird im Raytracer der Typ *PPMImage* verwendet, der ein zweidimensionales Array der Größe `width x height` von `unsigned char's` aufbaut. Es werden Methoden zum Setzen und Auslesen von Pixeln und zum Speichern in binärem und ASCII-Format bereitgestellt. Zur direkten Darstellung des Ergebnisses nach dem Rendervorgang dient die Klasse *GLImage*, welche mit Hilfe von OpenGL-Routinen die Daten eines Bildes im PPM-Format einliest und auf dem Bildschirm anzeigt.

Light

Neben Kamera und geometrischen Primitiven mit Materialeigenschaften gehören Lichtquellen zu den wesentlichen Szenenbestandteilen. Die Klasse *Light* definiert Punktlichtquellen. Der Parameter für den Lichtstrom ist allerdings nur ein auf den Bereich $[0, 1]$ beschränkter Farbwert und somit photometrisch nicht konsistent. Bei der Spezifikation des Systems wurde dieser Sachverhalt nicht mit einbezogen, womit sich eine Anpassung im Nachhinein auch als schwierig erwiesen hat.

Als weiterer Lichtquellentyp im System existiert die Klasse *Spotlight*, die die Punktlichtquelle um einen Öffnungswinkel erweitert, in welchem das Licht ausgestrahlt wird.

Material

Jedes geometrische Primitiv der Szene kann nur dann gerendert werden, wenn ihm ein Material zugewiesen wurde. Die Oberklasse *Material* definiert grundlegende Eigenschaften wie jeweils eine *ambient*, *diffuse* und *spekulare TextureMap* und einen Glanzkoeffizienten. In dieser Klasse wird das Shading des gefundenen Schnittpunktes übernommen, welches allerdings erst nach Überschreibung und Implementation in den Unterklassen *HallMaterial*, *PathTracMaterial* und *PathAndHallMaterial* zu einem wünschenswerten Ergebnis führt.

TextureMap

Die abstrakte Klasse *TextureMap* repräsentiert alle Formen von Texturinformationen, die ein Material definieren. Ein Material kann entweder durch den Typ *ColorMap* oder durch den Typ *TexJpgTga* bestimmt werden. Eine *ColorMap* besitzt als Attribut lediglich einen Farbwert *Color*, während eine *TextureMap* vom Typ *TexJpgTga* Texturen vom Typ JPEG oder TGA laden kann.

Mathematische Grund- und Helferklassen

Zur Repräsentation wesentlicher, abstrakter Merkmale eines Raytracing-Systems werden verschiedene Informationen zu konkreten Objekten zusammengefasst. Zu diesen Klassen gehören im einzelnen:

- *Point* Punkt im dreidimensionalen Raum mit (x, y, z) -Koordinaten
- *Vector3D* Vektor im dreidimensionalen Raum mit (x, y, z) -Koordinaten
- *Ray* Sehstrahl mit Punkt als Ursprung und Vektor als Richtung
- *Matrix* Matrizenrepräsentation und -operationen
- *Color* Farbwert im RGB-Format, skaliert auf den Bereich $[0, 1]$
- *LocalGeometry* Repräsentation von Schnittpunktinformationen
- *Timer* Zeitberechnung zur Statistikerstellung

Konfigurationsdateien

Sämtliche Klassen des Systems greifen auf bestimmte Konfigurationsparameter und globale Variablen zurück, die in den Headern *config.h* und *globals.h* vordefiniert sind. Neben global bekannten Structs, Variablen und neuen Typdefinitionen werden per Kommandozeile geforderte optionale Module durch Bitmasken bekannt gegeben.

4.2 Spezifikation

Nachdem nun ein grober Überblick über die einzelnen Module des Raytracers gegeben worden ist, soll in diesem Abschnitt ein detaillierterer Einblick in dessen Funktionsweise gegeben werden. Um gleichzeitig vorzustellen, wie der Ablauf vom Auslesen der Szene bis zur Ausgabe des gerenderten Bildes aussieht, werden die einzelnen Klassen, deren Attribute und Methoden erst dann erwähnt, wenn das System sie zum ersten Mal benötigt.

4.2.1 Auslesen der Kommandozeile

```
parseCommandLine(argc, (char **)argv);
```

Algorithmus 4.1: Parsen der Kommandozeile

Zu Beginn werden die bei der Ausführung in der Kommandozeile angegebenen Parameter mit Hilfe der GNU-Bibliothek *Popt*¹ geparkt. Die zur Verfügung stehenden Optionen sind im einzelnen:

Kurzbeschreibung	Parameterkürzel	Typ	Bedeutung
width	w	INT	Breite des Bildes (Default: 400)
height	h	INT	Höhe des Bildes (Default: 400)
input	i	STRING	Name der Eingabedatei
output	o	STRING	Name der Ausgabedatei für PPM-Format (Default: outfile.ppm)
use-convert	O	STRING	Name der Ausgabedatei für PNG-Format
display	d	NULL	Bildschirm Ausgabe mittels OpenGL (Default: nicht gesetzt)
no_saving	n	NULL	Bild wird nicht abgespeichert (Default: nicht gesetzt)
no_bboxes	b	NULL	Bounding Boxes werden nicht verwendet (Default: nicht gesetzt)
no_shadows	c	NULL	Keine Berechnung von Schatten (Default: nicht gesetzt)
accelerator	a	INT	Beschleunigertyp (0 = kein Beschleuniger; 1 = BVH; 2 = Octree; 3 = Ugrid; 4 = BSPTree, Default= 0)
hierarchies	e	INT	Anzahl der zu erstellenden Hierarchien für BVH (Default: 1)
octree-rec	t	INT	Rekursionstiefe für Octree (Default: 1)
ugrid-density	g	INT	Anzahl der Voxel im Ugrid (Default: 20)
bsp-maxobj	m	INT	Maximale Anzahl der Objekte pro BSP-Blatt (Default: 5)
recdepth	r	INT	Rekursionstiefe (Default: 5)

¹ Parse Command Line Options (<http://gnuwin32.sourceforge.net/packages/popt.htm>)

supersampling	u	INT	Supersampling & Strahlen pro Pixel (Default: 1)
poisson_dist	p	NULL	Poisson-Verteilung für Supersampling (Default: radial)
verbose	v	INT	Verbositylevel [0-3] (Default: 0)
light-model	l	INT	Beleuchtungsmodell (0: Phong; 1: Schlick, 2: Blinn; Default: 0)
partition-type	P	INT	Für Parallelisierung: Partitionierung (0: region; 1: reihenweise)
area	A	INT	Für Parallelisierung: Regionengröße $n \times n$
returntype	T	INT	Für Parallelisierung: Rückgabetyt (0: gesammelt, 1: auf einmal)
compress	C	NULL	Für Parallelisierung: Rückgabe komprimierter Daten
credits	X	NULL	Reitratze-Entwicklerteam

Tab. 4.1: Kommandozeilenoptionen

Die ausgelesenen Werte werden verarbeitet und je nach Typ an globale Variablen, Bitmasken und Parameter für weitere aufzurufende Konstruktoren und Methoden verteilt.

4.2.2 Konstruktion von Szene und Raytracer

```
scene=loader->getScene();
raytracer=new Raytracer(scene,width,height,recDepth,sSampling,lightModel);
PPMImage* img=(PPMImage *) (raytracer->getImage());
```

Algorithmus 4.2: Konstruktion von Szene und Raytracer

Je nach Loadertyp (hartcodierte Szenen oder VRML) wird die Szene mit ihren geometrischen Primitiven, Materialien, Lichtquellen und Kamera geladen. Eine Szene ist folgendermaßen konstruiert:

```
//Konstruktor
Scene(int accel, int accelOpts); //Beschleunigertyp und -optionen
//Membervariablen
Camera* m_camera; //Kameramodell
Container *m_container; //Container mit SceneItems der Szene
vector<Light*> *m_lightList; //Vektor mit Lichtquellen
vector<Material*> *m_materialList; //Liste mit Materialien
Color m_ambientColor; //ambiente, globale Farbigkeit
```

Algorithmus 4.3: Konstruktor und Membervariablen von *Scene*

Da der Raytracer an sich auch in einer separaten Klasse definiert und somit zum Objekt wird, muss dieses Objekt nun erzeugt werden. Als Parameter werden dem Konstruktor die Szene, Höhe und Breite des Bildes, Rekursionstiefe, Anzahl der Strahlen pro Pixel (Supersampling) und der Typ des Beleuchtungsmodells mitgegeben.

```
//Konstruktor
Raytracer(Scene *s,int w,int h,int recDepth,int sSampling,int lightModel);
//Membervariablen
Scene *m_scene;                //Szene
int m_height;                  //Höhe des Bildes
int m_width;                   //Breite des Bildes
int m_recDepth;                //Rekursionstiefe
int m_sSampling;               //Anzahl der Strahlen pro Pixel
int m_light;                   //Beleuchtungsmodell
Camera *m_camera;              //Kameramodell
ImagePlane* m_ip;              //Rastergitter
vector <Ray*> m_raylist;        //Liste mit Primärstrahlen
vector <RayInfo> m_infolist;    //Liste mit Gewichtungen zu den
                                //Primärstrahlen
```

Algorithmus 4.4: Konstruktor und Membervariablen von *Raytracer*

Nach dem Aufruf des Konstruktors wird ein neues Imageobjekt , welchem die aus der Methode `getImage()` in *Raytracer* gewonnenen Bilddaten zugewiesen werden.

4.2.3 Verfolgung von Sehstrahlen

Innerhalb der `getImage()`-Methode wird noch einmal unterschieden, ob seriell oder parallel gerendert wird, da die parallele Version mehrere, verschiedene Bildbereiche mit mehreren Clients berechnet. An dieser Stelle sei nur die serielle Variante erwähnt, die im Aufruf des Algorithmus 4.5 die Koordinaten der Region eines einzigen Pixels mitgeteilt bekommt.

```
getPartOfImage(x0,y0,x1,y1,img);
```

Algorithmus 4.5: Raytracer fordert Teilbereich des Gesamtbildes

Nun wird von der Kamera eine Liste mit allen Primärstrahlen angefordert, die von ihr für den spezifizierten Bildbereich ausgesendet werden, inklusive der zusätzlichen Strahlen bei Verwendung von Supersampling. Zudem wird eine Liste zu den Primärstrahlen geschrieben, die Structs mit Informationen zu ImagePlane – d.h. Pixelkoordinaten und Gewichtung – eines einzelnen Strahls beinhaltet.

```
m_raylist = m_camera->getPrimaryRays(m_sSampling,x0,y0,x1,y1);
m_infolist=m_camera->getPrimaryRayInfos();
```

Algorithmus 4.6: Primärstrahlen und Zusatzinformationen

Die von der Kamera angeforderte Primärstrahlliste wird danach abgearbeitet und die berechneten Strahlen in die Szene verfolgt.

```
for(rayIt=m_raylist.begin();rayIt!=m_raylist.end();rayIt++){
    ...
    colTmp=raytrace(*rayIt, m_recDepth);
    (*col) += (*colTmp) * rayweight;
    ...
}
```

Algorithmus 4.7: Schleife über alle Strahlen eines Pixels – Tracing durch die Szene

```
Color * Raytracer::raytrace(Ray * ray, int recursionDepth){
    LocalGeometry *lg=m_scene->intersect(ray);
    if (lg == NULL){
        ...
        return new Color(0,0,0);
    }
    GeomObject* go= (GeomObject*) lg->getSceneItem();
    Color* col;
    Material* mat = go->getMatPointer();
    col = mat->getColor(ray, lg, recursionDepth);
    ...
}
```

Algorithmus 4.8: Methode zur Strahlverfolgung mit Daten zum Sehstrahl und Rekursionstiefe

Die Methode `raytrace()` verfolgt nun den Strahl und sucht nach dem vordersten Schnittpunkt. Hierzu wird ein Schnittpunkttest mit der gesamten Szene durchgeführt. Innerhalb dieses Tests reicht die Szene den Sehstrahl an ihren Root-Container weiter, der wiederum in seiner eigenen `SceneItem`-Liste dasjenige Objekt sucht, welches am nächsten zum Betrachter liegt. Das zurückgelieferte Objekt `lg` ist vom Typ *LocalGeometry* und kann mit einem der in Algorithmus 4.9 aufgeführten Konstruktoren erzeugt werden:

```
//Konstruktoren
LocalGeometry (Color *color, SceneItem *item, Real dist, Point *point,
Vector3D *normal, Ray *ray);
LocalGeometry (Color *color, SceneItem *item, Point *point, Vector3D
*normal);
//Membervariablen
Color * mColor;                //Zeiger auf eine Farbe
SceneItem * mItem;             //Zeiger auf das geschnittene SceneItem
Real mDistance;                //Abstand zum Schnittpunkt
Point * iPoint;                //Schnittpunkt
Vector3D * mNormal;            //Normale am SP
Ray * r;                       //Strahl, der Schnittpunkt verursacht
//hat
```

Algorithmus 4.9: Konstruktoren und Membervariablen von *LocalGeometry*

Falls der Strahl ein Objekt getroffen hat, liefert der Schnittpunkttest allerdings noch keine Farbe zurück, da bis hierhin nur die geometrischen Primitive getestet wurden. Daher muss das zum Objekt zugehörige Material ermittelt und die am Schnittpunkt vorhandene Farbe im Material selbst berechnet werden. Die Oberklasse *Material* bietet dazu folgende Methoden und Eigenschaften:

```
//Konstruktor
Material( TextureMap* ambient, TextureMap* diffuse, TextureMap* specular,
          Real glossiness);
//Membervariablen
TextureMap* m_ambientColor;
TextureMap* m_diffuseColor;
TextureMap* m_specularColor;
Real m_glossiness;
int type;
//Methoden
Color* getColor (Ray* R, LocalGeometry* LG, int recursionDepth);
```

Algorithmus 4.10: Konstruktoren , Membervariablen und Methoden von *Material*

Ein klassisches Material besitzt eine ambiente, eine diffuse und eine spekulare *TextureMap*, die später unterschiedlich verrechnet werden. Ein Glanzfaktor beeinflusst die spekulare Reflexion des Materials. Um den Typus des Materials zu unterscheiden, muss jede Unterklasse einen Integerwert vergeben.

4.2.4 Shading

Das eigentliche Shading übernimmt die Methode `getColor()`, die in dieser Oberklasse allerdings nur testweise implementiert ist und daher nur den Wert der diffusen *TextureMap* zurückliefert. Berechnungen erfolgen in den Implementierungen von *HallMaterial* und *PathTracMaterial*. Daher wird der weitere Ablauf am Beispiel von *HallMaterial* erläutert, welches im Wesentlichen der Struktur von *Material* entspricht, zusätzlich aber noch zwei *TextureMaps* für den refraktierenden und reflektierenden Farbanteil und einen Refraktionsindex als Attribute besitzt.

```
//Zusätzliche Membervariablen
TextureMap* m_reflectionColor;
TextureMap* m_refractionColor;
Real m_refractionIndex;
```

Algorithmus 4.11: Zusätzliche Membervariablen von *HallMaterial*

Zuerst wird beim Sprung in `getColor()` der ambiente Anteil der Leuchtdichte am Schnittpunkt berechnet. Er ergibt sich aus dem globalen ambienten Licht der Szene skaliert mit dem ambienten Reflexionsgrad des Materials als angenäherte BRDF.

```
result = m_ambientColor->getColor() * scene->getAmbientColor();
```

Algorithmus 4.12: Ambienter Term

Zur weiteren Berechnung werden die bei der Schnittpunktberechnung ermittelten Werte von Intersektionspunkt, der von dem Objekt in Richtung Betrachter zeigende Intersektionsstrahl und die Normale am Schnittpunkt benötigt. Die Normale muss hierzu in Richtung des Betrachters zeigen.

```
if (viewVec.dotprod(normal) < 0.0) normal *= -1;
```

Algorithmus 4.13: Normale umkehren, wenn sie vom Betrachter wegzeigt

Es erfolgt die Berechnung der *direkten* Beleuchtung, indem über alle Lichtquellen der Szene iteriert, und der *diffuse* und *spekulare* Anteil an der Gesamtfarbe für jede Lichtquelle ermittelt und aufsummiert wird. In der Spezifikation existieren zwei Arten von Lichtquellen: Punktlichtquellen und Spotlights. Punktlichtquellen werden durch ihre Position bestimmt, Spotlights besitzen zusätzlich noch einen Öffnungsradius und eine Beleuchtungsrichtung. Die Klasse *Light* der Punktlichtquellen bildet die Basisklasse.

Da eine Typüberprüfung in C++ zur Laufzeit sehr aufwendig ist, wird eine boolsche Variable verwendet, um den Typ der Lichtquelle zu bestimmen.

```
class Light
{
    protected:
        Color m_Intensity;
        Point m_Position;
        bool m_spotlight;
};

class Spotlight : public Light
{
    protected:
        Real m_angle;
        Vector3D m_direction;

    public:
        Spotlight(Point pos, Color intens, Real angle, Vector3D dir);
};
```

Algorithmus 4.14: Die Lichtklassen *Light* und *Spotlight* (Lese- und Schreiboperationen nicht aufgeführt)

Um festzustellen, ob ein Schnittpunkt überhaupt für diese Berechnung in Frage kommt und nicht im Schatten eines Objekts liegt, wird ein *Schattenfühler* vom Schnittpunkt in Richtung Lichtquelle gesendet, der *false* zurückliefert, wenn sich kein Objekt auf diesem Weg befindet. Liegt der Punkt nicht im Schatten, so wird der Lichtstrom der Lichtquelle abgefragt, die – im Falle dessen, dass es sich um ein Spotlight handelt – noch mit einem Abweichungsgrad gewichtet werden muss, denn die Intensität von Spotlights nimmt im Lichtkegel von innen nach außen ab. Die Abweichung definiert sich aus dem Verhältnis des Öffnungswinkels des Spotlights und des Winkels zwischen Richtungsvektor des Spotlights und dem Schattenfühler.

```
//Überprüfung, ob Schnittpunkt im Lichtkegel liegt
bool HallMaterial::inCone(Light* l, Vector3D lightVec)
{
    Vector3D v=l->getDirection();
    Real dev = getDeviation(v, lightVec);
    if(dev < l->getAngle())
        return true;
    else return false;
}
//Winkelabweichung zwischen Richtung des Spotlights und Vektor von
//Schnittpunkt zu Spotlight
Real HallMaterial::getDeviation(Vector3D v, Vector3D lightVec)
{
    lightVec*=-1;
    Real dev = acos(lightVec.dotprod(v) / (v.getLength() *
                                         lightVec.getLength()));
    lightVec*=-1;
    return dev;
}
```

Algorithmus 4.15: Abfragen für *Spotlights*

Ist im folgenden der Cosinus zwischen der Normalen am Schnittpunkt und dem Vektor in Richtung Lichtquelle kleiner 0, dann existiert an diesem Punkt keine *direkte* Beleuchtung und es wird mit der eventuellen rekursiven Verfolgung des Strahls fortgefahren. Andernfalls wird der erste *diffuse* Anteil berechnet durch Algorithmus 4.16 nach *Phong*, nämlich die Gesamtintensität, gewichtet mit dem soeben ermittelten Cosinus und der diffusen Farbe des Materials als angenäherte *BRDF*:

```
*result += m_diffuseColor->getColor() * intens * angle;
```

Algorithmus 4.16: Diffuser Anteil nach *Lambert*

Jedes Material besitzt einen Glanzkoeffizienten. Dieser Koeffizient bestimmt das Gradmaß des *spekularen* Reflexionsanteils des Materials und ist zugleich der *Phong-Koeffizient*. Ist der Koeffizient >1, wird der spekulare Anteil berücksichtigt und das in der Kommandozeile angegebene Beleuchtungsmodell wird angewandt. Für jedes Modell muss zusätzlich ein weiterer Winkel berücksichtigt werden:

- **Phong**
der Winkel zwischen dem perfekt gespiegelten Reflexionsstrahl und dem Sehstrahl

```
rLightVec = (normal * (lightVec.dotprod(normal) * 2)) - lightVec;
```

- **Blinn**
der Winkel zwischen der Schnittpunktnormalen und dem Vektor, der den Winkel zwischen Sehstrahl und Schattenfühler halbiert (= *Bisector*)

```
bisector = (viewVec + lightVec)*0.5;
```

- Schlick
der Winkel zwischen der Schnittpunktnormalen und dem Vektor, der den Winkel zwischen Sehstrahl und Schattenfühler halbiert (= *Bisector*), allerdings anders berechnet wird als bei Blinn

```
help = lightVec + viewVec;
bisector = help * (1/(sqrt(help.dotprod(help))));
```

Erst wenn auch dieser Cosinus > 0 ist, kann die *spekulare* Farbe berechnet werden. Im Programmcode ergibt sich dann für die einzelnen Beleuchtungsmodelle:

- Phong

```
*result += m_specularColor-&gtgetColor() * intensity * pow(rAngle,
m_glossiness);
```

- Blinn

```
*result += m_specularColor-&gtgetColor() * intensity * pow(rAngle,
m_glossiness);
```

- Schlick

```
*result += m_specularColor-&gtgetColor() * intensity * (rAngle /
(m_glossiness-(m_glossiness*rAngle)+rAngle));
```

Blinn und Schlick's Ansätze werden allerdings nur angenähert und nicht ausführlich berechnet. In [Schlick93] ist der hier verwendete und berechnete Faktor $G(v)$ beschrieben als der Anteil des nicht beschatteten, reflektierten Lichts.

Nachdem die von allen Lichtquellen ausgehende *direkte* Beleuchtung eingesammelt worden und das Abbruchskriterium der maximal erreichten Rekursionstiefe nicht eingetroffen ist, werden die *Sekundärstrahlen* erzeugt und versendet. Dies geschieht nur im Falle dessen, dass das Material auch spekulare Eigenschaften besitzt, die mittels `m_reflectionColor` und `m_refractionColor` festgelegt wurden. Die Berechnung dieser Anteile erfolgt entweder mittels der Fresnel-Approximation, wenn ein Brechungsindex >1 gegeben ist, oder es wird nur der Reflexionsgrad (`m_reflectionColor`) verwendet, der bei der Erzeugung des Materials angegeben wurde.

```
if (m_refractionIndex > 1.0)
{
    Vector3D partOne = viewVec * (1/m_refractionIndex);
    Real partTwo = (viewVec.dotprod(normal) * (1/m_refractionIndex));
    Real partThree = 1-(((1/m_refractionIndex) * (1/m_refractionIndex)) *
        (1-viewVec.dotprod(normal) *
        (viewVec.dotprod(normal)))));
    Vector3D transmittVec = partOne - (normal*(partTwo + sqrt(partThree)));
    viewVec *= (-1);
    Vector3D normalN = normal * (-1);
    Real term = 0;
    Real V1 = viewVec.dotprod(viewVec);
    Real N1 = normal.dotprod(normal);
    Real N2 = normalN.dotprod(normalN);
    Real T2 = transmittVec.dotprod(transmittVec);
```

```

if (V1 > 0.0 && N1 > 0.0 && N2 > 0.0 && T2 > 0.0 )
{
    Real laengeV1 = sqrt(V1);
    Real laengeN1 = sqrt(N1);
    Real laengeN2 = sqrt(N2);
    Real laengeT2 = sqrt(T2);

    Real Skalar1 = viewVec.dotprod(normal);
    Real Skalar2 = transmittVec.dotprod(normalN);

    if (Skalar1 > 0.0 && Skalar2 > 0.0)
    {
        Real Winkel1 = Skalar1 / (laengeV1 * laengeN1);
        Real Winkel2 = Skalar2 / (laengeN2 * laengeT2);
        Real I2 = m_refractionIndex;
        Real I1 = 1;
        Real parallel = ((I2*Winkel1) - (I1*Winkel2)) / ((I2*Winkel1) +
            (I1*Winkel2));
        Real senkrecht = ((I1*Winkel1) - (I2*Winkel2)) / ((I2*Winkel1) +
            (I1*Winkel2));
        Real licht = (parallel*parallel) + (senkrecht*senkrecht);

        term = 0.5 * (licht);
    }
    if (term >= 1.0)
    {
        term = 1.0;
    }
    setReflectionColor(new ColorMap(Color(term,term,term)));
    setRefractionColor(new ColorMap(Color(1-term,1-term,1-term)));
}
viewVec *= (-1);
}

```

Algorithmus 4.17: Berechnung des Fresnel-Terms

Sollte der Reflexionsgrad des Materials ungleich 0 sein, wird ein Reflexionsstrahl erzeugt und in die Szene geschickt. Die Richtung entspricht der eines perfekt gespiegelten Strahls. Die von diesem Sekundärstrahl rekursiv eingesammelte Leuchtdichte wird mit dem Reflexionsgrad skaliert und zum Ergebnis der *direkten* Beleuchtung addiert. An dieser Stelle hat sich bei der Implementation des Raytracers ein kleinerer Irrtum eingeschlichen. Die direkte Beleuchtung wird hier mit dem Koeffizienten des Komplements zum Reflexionsgrad skaliert, sie muss allerdings vollständig verrechnet werden.

```

// Reflexionsstrahl erzeugen und losschicken
if ((m_reflectionColor->getColor().getR() +
    m_reflectionColor->getColor().getG() +
    m_reflectionColor->getColor().getB()) > 0.01)
{
    Vector3D reflectVec=viewVec-(normal*( viewVec.dotprod(normal))) * 2;
    Ray* reflectedRay = new Ray (IPoint, reflectVec);
    Color* reflecColor = raytracer->raytrace(reflectedRay,

```

```

--recursionDepth);
Color koef = Color(1,1,1) - m_reflectionColor->getColor();
*result = (*result) * koef + (*reflecColor) * m_reflectionColor->getColor();
delete reflecColor;
}

```

Algorithmus 4.18: spekulare Reflexion und Versendung eines Sekundärstrahls

Gleiches Verfahren wird auf die Sekundärstrahlen für refraktierende Materialien angewandt. Es gilt, dass der Refraktionsgrad – nicht zu verwechseln mit dem Refraktionsindex – ungleich 0 sein muss, damit der Primärstrahl rekursiv verfolgt wird. Es wird das gleiche Transmissionsmodell verwendet wie in Kapitel 3.3.2.4 beschrieben, aber mit unterschiedlicher Herleitung:

$$\vec{w}_r = \vec{w} * \frac{\eta_1}{\eta_2} - \left(\vec{n} * \left((\vec{w} \cdot \vec{n}) * \frac{\eta_1}{\eta_2} + \sqrt{1 - \left(\left(\frac{\eta_1}{\eta_2} \right)^2 * (1 - (\vec{w} \cdot \vec{n})^2)} \right)} \right) \right) \quad (4.1)$$

mit

- $\vec{w}_r$ refrakter Transmissionsstrahl
- $\vec{w}$ Einfallsvektor (von Oberfläche des Objektes abgeneigt)
- $\vec{n}$ Normale am Schnittpunkt
- η_1 Refraktionsindex des Eintrittsmaterials
- η_2 Refraktionsindex des Austrittsmaterials

In der Implementation wird lediglich der Übergang eines Strahls von Vakuum / Luft in das Material des Objekts berücksichtigt. Allerdings ist diese Abfrage auch dann gültig, wenn ein Strahl von einem Material wieder in Vakuum / Luft übergeht, was zu einem inkorrekten Resultat führt. Die Berechnung für Austrittsstrahlen ist nicht vorgesehen und ist erst während der Studienarbeit eingebaut worden.

Ist der Refraktionsstrahl gefunden, wird er weiter verfolgt und die ermittelte Leuchtdichte zurückgeliefert. Die resultierende Farbe am Schnittpunkt ergibt sich nun aus Addition von *direkter* Beleuchtung und jener Leuchtdichte, skaliert mit der angenäherten BRDF (`m_refractionColor`).

```

if ((m_refractionColor->getColor().getR() +
    m_refractionColor->getColor().getG() +
    m_refractionColor->getColor().getB()) > 0.01)
{
    Vector3D partOne = viewVec * (1/m_refractionIndex);
    Real partTwo = (viewVec.dotprod(normal) * (1/m_refractionIndex));
    Real partThree = 1 - (((1/m_refractionIndex) * (1/m_refractionIndex)) *
        (1 - (viewVec.dotprod(normal) * (viewVec.dotprod(normal)))));
    Vector3D transmittVec = partOne - (normal * (partTwo + part(partThree)));
    Ray* transmittRay = new Ray(IPoint, transmittVec);
    Color* refractColor = raytracer -> raytrace(transmittRay,
        --recursionDepth);
    *result = *result + m_refractionColor->getColor() * (*refractColor);
    delete refractColor;
}

```

Algorithmus 4.19: transmittierende Reflexion und Versendung eines Sekundärstrahls

Zusammengefasst werden alle Arten spekularer Reflexionen berücksichtigt und in jedem Fall weiter verfolgt. *Diffuse* Reflexionen werden allerdings nicht berücksichtigt und absorbiert. Nachdem somit die Leuchtdichte am Schnittpunkt vollständig ermittelt wurde, muss diese noch auf den definierten Farbbereich $[0, 1]$ skaliert werden, um zum einen nicht existente Werte im Farbraum zu eliminieren und zum anderen spätere Fehler beim Setzen des Pixelwerts zu vermeiden.

Der an *Raytracer* zurückgelieferte Wert wird bei eventuellem Supersampling gewichtet und auf den bisher ermittelten Wert für das Pixel addiert. Nachdem alle Strahlen für Supersampling verschossen wurden, wird der Mittelwert an die zugehörige Pixelposition in *PPMImage* gesetzt und die Versendung weiterer Primärstrahlen fortgesetzt. Je nach Kommandozeilenoptionen wird das Resultat nun im PPM- oder PNG-Format gespeichert oder am Bildschirm angezeigt. Der Raytracer terminiert mit der Ausgabe von Zeitstatistikangaben.

5. Integration

Wie schon in Kapitel 4.1 beschrieben, ist der Raytracer modular aufgebaut. Die wichtigsten Instanzen des Systems wie Punkte, Vektoren, Strahlen, Materialien oder geometrische Primitive sind in separaten Klassen implementiert und können daher nahezu trivial erweitert und angepasst werden. In der Spezifikationsphase der Studienarbeit war dies auch noch unumstößlich. Vorgesehen war die Implementation eines zusätzlichen Materials, dass sowohl mit Sehstrahlen als auch mit Photonen umgehen kann und diese entsprechend der Konventionen aus [Jensen01] behandelt, der in [Jensen01] beschrieben und im vollständigen Quellcode abgedruckten Photon Map, einer Klasse zur Simulation von Flächenlichtquellen, um Halbschatten erzeugen und Kaustiken großflächig rendern zu können, und eine Erweiterung der Implementationsdatei *main.cpp*, die schließlich das Photon Mapping anzustoßen hat. In der Implementationsphase zeigte sich aber, dass eine Integration in ein bestehendes System genauso komplex ist wie die Entwicklung eines komplett neuen. Es ist aufgefallen, dass viele weitere Klassen an Photon Mapping angepasst werden mussten als vorgesehen. Der Überblick über bereits modifizierte und noch anzupassende Klassen konnte dabei schon einmal verloren gehen. Ebenso mussten bestehende Methoden einiger Klassen neu geschrieben werden, da gewisse Funktionalitäten nicht vollständig integriert waren. Nichtsdestotrotz waren die Modifikationen nicht so umfangreich, wie man vielleicht vermuten mag. Es wurde zum Ziel gesetzt, die Funktionalität des bestehenden Raytracers durch die Modifikation von Klassen und Methoden nicht einzuschränken. Alle Verfahren des Systems können somit auch weiterhin verwendet werden.

5.1 Basiskonfiguration

In der Basisdatei *main.cpp* werden systemweit geltende globale Variablen initialisiert, die Kommandozeile ausgelesen und das eigentliche Raytracing angestoßen. Globale Variablen sind für ein System wegen ihrer dauerhaften Speicherreservierung bis zur Terminierung eigentlich zu vermeiden, dennoch mussten einige Objekte hinzugefügt werden, da sie von mehreren Klassen ständig benötigt werden. Im Header *globals.h* sind diese Variablen als `extern` deklariert:

Globale Variable	Bedeutung
<code>PhotonMap *photonmap</code>	globale Photonmap
<code>PhotonMap *causticphotonmap</code>	spekulare Photonmap zur Abspeicherung der Photonen für die Kaustiken
<code>Light *actuellight</code>	die zum aktuellen Zeitpunkt Photonen emittierende Lichtquelle
<code>PhotOpts ph_opts</code>	struct mit Optionen für Photon Mapping, ausgelesen aus der Kommandozeile
<code>int MAX_REC_DEPTH</code>	Maximale Rekursionstiefe (Default: 5)
<code>bool INSIDE_CAUSTIC_TRACING</code>	boolscher Wert zur Trennung des Radiance Estimate von globalen und spekularen Photonen (Default: FALSE)
<code>bool DIRECT_LIGHTING</code>	boolscher Wert zum Abfangen von Überbleuchtung (Default: nicht gesetzt, bei erster Verwendung auf TRUE))

Tab. 5.1: Globale Variablen

Photon Mapping benötigt des weiteren zusätzliche, variable Parameter, die der Kommandozeile entnommen werden können, wenn nicht die Standardwerte genutzt werden sollen. Es soll möglich sein, alternativ nur globale, spekulare oder beide Arten von Photonen von den Lichtquellen zu versenden. Eine Schwierigkeit des Radiance Estimate kann an dieser Stelle schon erwähnt werden: die Trennung der Leuchtdichte globaler und spekulärer Photonen. Da globale Photonen in Richtung aller Objekte versendet werden, ist es unumgänglich, dass die Leuchtdichten beider Photon Maps rund um einen Schnittpunkt eingesammelt werden, um alle berechneten Werte zu berücksichtigen. An den Punkten, an welchen Kaustiken entstehen bzw. die Photonen der `causticphotonmap` eingesammelt werden, akkumuliert sich so die Leuchtdichte und das Ergebnis entspricht nicht mehr den realen Beleuchtungsverhältnissen.

Die speziell für Photon Mapping hinzugefügten Parameter bestehen nun aus:

Parameter	Bedeutung
<code>int globalphotons</code>	globale Photonen (Default: 10.000)
<code>int causticphotons</code>	spekulare Photonen zur Darstellung von Kaustiken (Default: 1.000.000)
<code>bool photonmapping</code>	Trigger zum Aktivieren von Photon Mapping (Default: FALSE)
<code>bool displaypm</code>	Trigger zur Darstellung der Photon Map (Default: FALSE)
<code>float maxdistance</code>	Maximaler Radius, in welchem sich die Photonen während des Radiance Estimate befinden dürfen (Default: 0.15)
<code>int nearestphot</code>	Anzahl der einzusammelnden Photonen rund um den Schnittpunkt (Default: 50)
<code>int lightrays</code>	Anzahl der Samples, die von einer Flächenlichtquelle ausgesendet werden, um die direkte Beleuchtung zu ermitteln (Default: 20)

Tab. 5.2: Zusätzliche Parameter aus der Kommandozeile für Photon Mapping

Eine Besonderheit ist das in der Konfigurationsdatei `config.h` neu definierte `struct PhotOpts`, welches die Parameter `maxdistance`, `nearestphot` und `lightrays` zu einer Datenstruktur zusammenfasst. Dies vermeidet Overhead bei der Übergabe der Werte, zumal die Instanz `ph_opts` global definiert ist.

Start des Photon Mapping-Moduls

Wie der Standard-Raytracer des Projektpraktikums wird Photon Mapping in der Basismethode `main()` angestoßen. Nachdem `scene` und `raytracer` initialisiert sind, werden die `globalphotonmap` und die `causticphotonmap` mit ihrer Anzahl an maximalen Photonen initialisiert. Die `scene` teilt die ihr zugewiesene Photonenanzahl auf die Lichtquellen auf. Im Anschluss beginnt der *First Pass* – das Photon Tracing: Es wird über alle Lichtquellen iteriert und für beide Photon Maps separat die Photonenamples mit zufälligen Ursprüngen auf der Lichtquelle und zufälligen Richtungen erzeugt. Jedem Photon wird sein Anteil am Lichtstrom und sein Ursprung zugewiesen und schließlich mit einer der passenden Methoden aus *Raytracer* – entweder `photontrace()` oder `causticPhotonTrace()` – übergeben. Nach der Emission aller

Photonen werden die Leuchtdichten der jeweiligen Photon Map mit `scale_photon_power()` gewichtet – warum dies erst nach der Emission passiert, wird später noch erläutert. Damit die Leuchtdichte später effizient abgeschätzt werden kann, müssen die Maps nun noch balanciert werden. Der weitere Ablauf für den *Second Pass* – das Rendern der Szene mit ihren Photonen – folgt in etwa dem des Standard-Systems: Je nach Trigger kann nun entweder die Photon Map angezeigt werden (die Photonen werden pixelweise mit voller Intensität gesetzt) oder das Bild gerendert werden – die speziellen Methoden hierfür ruft *Raytracer* auf. Das Resultat wird gespeichert und kann am Bildschirm angezeigt werden. Es ist allerdings zu beachten, dass wahlweise nur die Photon Map oder das gerenderte Bild betrachtet und gespeichert werden kann. Nach Erzeugung und Terminierung eines OpenGL-Fensters terminiert auch das Programm.

5.2 Klassen und Objekte

Die Funktionsweise des Raytracers wurde bereits in Kapitel 4 beschrieben. Da das Photon Mapping-Verfahren sich prinzipiell nicht in großem Maße von dessen Ablauf unterscheidet, werden an dieser Stelle zunächst die hinzugefügten Klassen und Objekte erklärt. Dennoch wird in einem der folgenden Unterkapitel auf die Besonderheiten des *First* und *Second Pass* eingegangen.

5.2.1 PhotonMap

Die Photon Map ist die wohl wichtigste Datenstruktur des Photon Mapping und wurde daher komplett aus [Jensen01] übernommen. Jensen hat das Verfahren selbst entwickelt, den Quellcode optimiert und zur Verfügung gestellt. Daher wird auf die nähere Dokumentation des Quellcodes verzichtet und lediglich auf die Beschreibung von Datenstrukturen und Operationen Wert gelegt.

Das Photon

```
typedef struct Photon
{
    float pos[3];      /**< Photonenposition */
    short plane;       /**< Trennungsebene für den kd-Tree */
    unsigned char theta, phi; /**< einfallende Richtung */
    float power[3];    /**< unkomprimierte Photonenenergie */
} Photon;
```

Algorithmus 5.1: Aufbau eines Photons

Ein Photon wird komprimiert in der Photon Map abgelegt. Es ist ein 27 Bytes großes Struct, dass sich aus einem 3x4 Byte float-Array für die Trefferposition, einem 1 Byte short-Flag für die Teilungsebene im kd-Tree, zwei 1 Byte unsigned char's für die Polarkoordinaten der Einfallsrichtung und ein weiteres 3x4 Byte float-Array für die Energie (*Lichtstrom*) zusammensetzt. Die Komprimierung erfolgt für die Einfallsrichtung des Photons und besteht aus einem Mapping der Polarkoordinaten auf 65536 mögliche Richtungen. Wenn ein Photon in der Map gespeichert wird, wird die Einfallsrichtung (3 float-Werte) anhand des Algorithmus 5.2 in die Polarkoordinaten `theta` und `phi` transformiert. Um die spätere Rücktransformation während des Raytracing zu beschleunigen, wird vorab im Konstruktor eine Look-Up-Tabelle erzeugt, in welcher

die Daten von `costheta`, `sintheta`, `cosphi` und `sinphi` jeder möglichen Richtung abgelegt werden. Jede Richtung ergibt sich dann aus der Umrechnung der Polarkoordinaten in kartesischen Koordinaten:

$$\begin{aligned}x &= \sin(\theta) * \cos(\phi) \\y &= \sin(\theta) * \sin(\phi) \\z &= \cos(\theta)\end{aligned}\tag{5.1}$$

```
theta = acos(dir_z) * (256.0/PI);
phi = atan2(dir_y, dir_x) * (256.0 / (2.0*PI));
```

Algorithmus 5.2: Konvertierung der Einfallsrichtung zu Polarkoordinaten

Während des Radiance Estimate wird zusätzlich eine besondere Art Photonen benötigt: die *Nearest Photons*. Dieses Struct charakterisiert diejenigen Photonen, die innerhalb des vorgegebenen Radius des Sammelkörpers – in diesem Fall eine Kugel – eingesammelt werden sollen. Es besteht aus einem 4 Byte `int`-Wert für die maximale Anzahl einzusammelnder Photonen, einem 4 Byte `int`-Wert für die Position im Array innerhalb der bisher gefundenen Photonen, einen weiteren 2 Byte `int`-Wert als Wahrheitswert für den Aufbau eines Heap, einem 3x4 Byte `float`-Array für die Schnittpunktposition, für welche die Leuchtdichte bestimmt werden soll, einem 4 Byte `float` für die Distanzwerte der Photonen und einem 27 Byte `x (int max)` großen Array für die Speicherung aller gefundenen Photonen. Es beinhaltet somit die Daten aller während des Radiance Estimate gefundenen Photonen. Der Aufbau des Struct ist in Algorithmus 5.3 abgebildet.

```
typedef struct NearestPhotons
{
    int max;      /**< maximale Anzahl der einzusammelnden Photonen */
    int found;    /**< Position im Array innerhalb der bisher gefundenen
                    Photonen*/
    int got_heap; /**< Wahrheitswert fuer Speicherung auf dem Heap */
    float pos[3]; /**< Schnittpunktposition */
    float *dist2; /**< Distanzwerte */
    const Photon **index; /**< Array für die Speicherung aller gefundenen
                    Photonen */
} NearestPhotons;
```

Algorithmus 5.3: Aufbau der *NearestPhotons*

Der Konstruktor

Der Konstruktor bekommt als Parameter die (maximale) Anzahl der zu emittierenden (und zu speichernden) Photonen übergeben und reserviert in dessen Größe Speicher für ein eindimensionales Array von Photonen. Die Anzahl der bereits abgespeicherten Photonen wird auf 0 gesetzt. Für das Wiederauffinden der Photonen im später generierten kd-Tree und dessen Balancierung wird zusätzlich eine Bounding Box verwendet, die die Ausmaße von maximalster und minimalster Position der Photonen hat. Die bereits erwähnte Look-Up-Tabelle für die Photoneneinfallsrichtungen wird hier initialisiert.

```

class PhotonMap
{
public:
    PhotonMap(int max_phot);
    ~PhotonMap();
    void scale_photon_power(const float scale);
    void store(const float power[3], const float pos[3], const
               float dir[3]);
    void balance( void );
    void irradiance_estimate(float irrad[3], const float pos[3],
                           const float normal[3], const float
                           max_dist, const int nphotons) const;
    void locate_photons(NearestPhotons *const np, const int index)
                       const;
    void photon_dir(float *dir, const Photon *p) const;
    PPMImage* visualize(int pm_width, int pm_height);
    vector<Ray*>* collectSpecularReflections(vector <Ray*> &rays,
                                           Light *l);

private:
    void balance_segment(Photon **pbal, Photon **porg, const int
                        index, const int start, const int end);
    void median_split(Photon **p, const int start, const int end,
                     const int median, const int axis);
    bool saveDebug(char *filename);
    Photon *photons;
    int stored_photons
    int half_stored_photons;
    int max_photons;
    int prev_scale;
    float costheta[256];
    float sintheta[256];
    float cosphi[256];
    float sinphi[256];
    float bbox_min[3];
    float bbox_max[3];
};

```

Algorithmus 5.4: Konstruktoren, Methoden und Membervariablen der Klasse *PhotonMap*

Die Membervariablen

Die Photon Map ist ein Objekt bestehend aus einem Array von Photonen, die der Struktur des Struct Photon entsprechen. Zusätzlich werden die Anzahl der gespeicherten Photonen, die Hälfte der Anzahl der gespeicherten Photonen (*root* im Heap des kd-Tree) und die maximale Anzahl zu speichernder Photonen vermerkt. Die Variable *prev_scale* merkt sich die Anzahl bisher gespeicherter Photonen, um bei einer erneuten Skalierung den von einer vorherigen Lichtquelle ausgesendeten Lichtstrom nicht noch einmal zu modifizieren. *Costheta*, *sintheta*, *cosphi* und *sinphi* dienen als Arrays für die Ablage vorberechneter Polarkoordinaten in einer Look-Up-Tabelle. Und zuletzt bestimmen die Koordinaten in *bbox_min* und *bbox_max* die minimale und maximale Ausdehnung des Volumens aller Photonen in der Szene.

Die Methoden

```
void store(const float power[3], const float pos[3], const float dir[3]);
```

Hat während des *First Pass* ein Photon eine diffuse Fläche getroffen, wird es mit seiner Position, Einfallrichtung und Leuchtdichte in der Photon Map abgespeichert. Die Bounding Box wird den neuen Daten entsprechend angepasst. In dieser Methode erfolgt auch die Konvertierung der in `float` angegebenen Einfallrichtung zu Polarkoordinaten nach Algorithmus 5.2.

```
void scale_photon_power(const float scale);
```

In der Theorie werden die Photonen mit einem Bruchteil des Lichtstroms der Lichtquelle ausgesendet. Jedes Photon trägt diese Energie weiter und verteilt sie. Trifft es auf eine Fläche, wird die Energie mit dem jeweiligen Reflektionsgrad des Materials skaliert. Das Photon verliert somit an Energie. Mathematisch gesehen ist dies korrekt, in der Implementierung ist diese Form der Verrechnung allerdings nicht wünschenswert. Angenommen, es gäbe eine Lichtquelle in der Szene mit hellstem Lichtstrom (`Color(1.0, 1.0, 1.0)`), die 1 Mio. Photonen aussendet, so wäre die Energie eines jeden Photons auf $1e-06$ beschränkt. Wird die Energie nun skaliert, kann es leicht zu Rundungsfehlern kommen, die sich später negativ auswirken würden. Aus diesem Grund werden die einzelnen Photonen mit vollem Lichtstrom der Lichtquelle versandt. Erst *nach* dem Tracing werden die gespeicherten Photonen mit der für eine Lichtquelle versendeten Photonenanzahl skaliert:

```
energy of photon = energy of photon / emitted photons from lightsource
```

Diese Methode wird jedes Mal aufgerufen, wenn eine Lichtquelle ihre Photonen versendet hat. Es werden nur diejenigen Photonen skaliert, die von dieser Lichtquelle emittiert wurden.

```
void balance( void );
```

Nachdem alle für die Szene bestimmten Photonen versendet worden sind, muss die Struktur der Photon Map zunächst noch balanciert werden, da die Lichtquellen iterativ ihre Photonen versendet und abgespeichert haben.

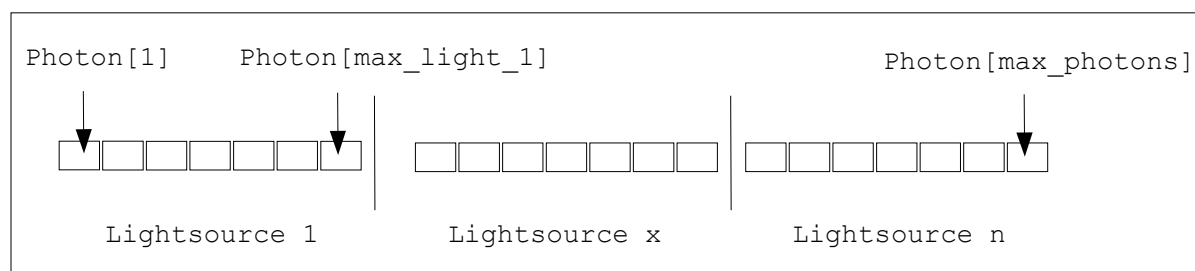


Abb. 5.1: Aufbau des Arrays der Photon Map *vor* der Balancierung

`Balance()` kopiert das Photonenarray in ein temporäres zweites Array, das später das balancierte sein wird. Diese Arrays zuzüglich einem Index, dem Start- und Endindex (=Anzahl der gespeicherten Photonen) des ursprünglichen Arrays werden an die Methode `balance_segment()` weitergereicht. Es wird nun der Median und innerhalb der Bounding Box die Achse zum Splitten für den kd-Tree gesucht. Die Ergebnisse erhält die Methode `median_split()` samt Start- und Endindex des zu ordnenden Arrayteils, um den Photonenblock rund um den Median zu partitionieren.

Die beiden aufgeteilten Blöcke werden nun rekursiv weiter aufgeteilt und sortiert, bis das gesamte Array verarbeitet ist. `Balance()` sorgt nun für eine Restrukturierung des kd-Trees und baut einen Heap auf, mit dem später während des Radiance Estimate gerechnet wird.

```
void irradiance_estimate(float irrad[3], const float pos[3],
                        const float normal[3], const float max_dist,
                        const int nphotons) const;
```

Im *Second Pass* wird beim Verfolgen der Strahlen an den Schnittpunkten der von den Photonen verteilte Lichtstrom eingesammelt. Der Lichtstrom kommt als Empfängergröße *Beleuchtungsstärke* am Schnittpunkt an und wird als *Leuchtdichte* weitergegeben. Für das Radiance Estimate werden als Parameter die Schnittpunktposition, die Normale am Schnittpunkt, der Radius des „Sammelkörpers“ und die maximale Anzahl der einzusammelnden Photonen benötigt. Für die Distanzen der Photonen zum Schnittpunkt und die *Nearest Photons* werden zwei Arrays mit der Größe der Anzahl einzusammelnder Photonen initialisiert. Mittels der Methode `locate_photons()` werden auf dem erstellten Heap entweder die negativen Distanzen links vom Index oder der rechte Teil mit positiven Distanzen rechts vom Index rekursiv durchsucht. Die Rekursion dauert solange, bis der Index, ab welchem begonnen wird, größer ist als der Median (`half_stored_photons`). Die Distanzen werden berechnet und quadriert. Ist nun ein Photon gefunden, dass das Kriterium der maximalen Distanz nicht überschreitet, und das Array mit der maximalen Anzahl zu suchender Photonen noch nicht gefüllt, wird es einsortiert. Ansonsten wird der Heap nach dem Photon mit bisher größter Distanz gesucht, verglichen und bei kleinerer Distanz zum Schnittpunkt ersetzt und einsortiert. Ist das Array der *Nearest Photons* (MAXHEAP) noch nicht gefüllt, muss er zuerst aufgebaut werden. Nach dem Rücksprung zu `irradiance_estimate()` wird die Leuchtdichte aller gefundenen Photonen dann aufsummiert, wenn mehr als 8 Photonen gefunden worden sind. Das Resultat muss nun noch mit der abgeschätzten Dichte – in diesem Fall eine auf die Ebene projizierte Kugel – skaliert werden.

```
void photon_dir(float *dir, const Photon *p) const;
```

Diese öffentliche Methode errechnet aus den Polarkoordinaten wieder die Richtung im Weltkoordinatensystem für ein gegebenes Photon `p` und nutzt hierzu die im Konstruktor erstellte Look-Up-Tabelle.

```
PPMImage* visualize(int pm_width, int pm_height);
```

Zur simplen Darstellung der Photon Map in einem OpenGL-Fenster wurde diese Methode implementiert. Sie stammt nicht aus dem Original-Quellcode von Jensen und ist lediglich zu Testzwecken implementiert, da sie mathematisch nicht korrekt ist. Als Korrekturfaktoren für die zweite Dimension werden der Abstand der Kamera zur *ImagePlane* (*Focus*) und der zu betrachtende Bildausschnitt (Mittelpunktbestimmung) bestimmt.

```
vector<Ray*>* collectSpecularReflections(vector<Ray*> &rays, Light *l);
```

Als erster Versuch zum Tracing von Photonen in Richtung spekularer Materialien wurde diese Methode eingebaut. Der Algorithmus ist relativ simpel, dafür allerdings auch sehr teuer im Aufwand.

Es wird ein neuer Beschleuniger erzeugt, in welchem alle von einer Lichtquelle erzeugten Samples auf Intersektion getestet werden. Diejenigen Lichtstrahlen (= Photonenrichtungen), die auf spekulare Materialien treffen, werden in einem neuen Vektor zwischengespeichert. Solange dieser Vektor nicht

die Größe aller von der Lichtquelle auszusendenden *Caustic Photons* besitzt, werden neue Samples erzeugt und getestet. Der Aufwand steigt somit, je weniger spekulare Objekte in der Szene vorhanden sind, exponentiell an und bremst das gesamte System unnötig aus. Ein zweiter Ansatz zur Bestimmung dieses für Kaustiken sehr wichtigen Vorgangs wird in der Dokumentation zu den Lichtquellen in Kapitel 5.2.5 vorgestellt.

```
bool PhotonMap::saveDebug(char *filename)
```

Zur testweisen Ausgabe von gespeicherten Daten in der Photon Map dient die Methode `saveDebug()`. Die Daten werden in einem *.txt-File innerhalb des Arbeitsverzeichnisses abgelegt.

5.2.2 Light

Anders als beim traditionellen Raytracing werden beim Photon Mapping zwei Pfade verfolgt. Der Pfad vom Auge des Betrachters zu den Objekten und deren Beleuchtung in der Szene (*backward-raytracing*) und der direkte Weg des Lichtes zu den Objekten. Damit die Lichtquellen auch aktiv an der Beleuchtung der Szene teilnehmen und somit Photonen aussenden können, mussten sie noch angepasst werden. Die im Raytracer bereits vorhandenen Methoden und Variablen werden nicht näher erläutert.

```
class Light
{
protected:
    Color m_Intensity; /** Intensität der Lichtquelle */
    Point m_Position; /** Position der Lichtquelle */
    bool m_spotlight; /** Spotlight? */
    Sample *m_sample;
    int m_photons; /**< Photonen der Lichtquelle */
    int m_caustic_photons; /**< CausticPhotonen der Lichtquelle */
    Real m_sum_of_intensity; /**< Gesamtintensitaet */
    Color* m_photon_power; /**< einzelne Photonenenergie */
    vector <Ray*> *photondirlist; /**< Liste mit zufaelligen
                                   Photonenrichtungen */
    vector <Ray*> *causticphotondirlist; /**< Liste mit zufaelligen
                                   Photonenrichtungen fuer Kaustiken*/
    bool m_arealight; /**< Switch fuer Flaechenlichtquelle */

public:
    Light(Point pos, Color intens);
    Light();
    virtual ~Light();
    virtual Color getIntensity();
    virtual Point getPosition();
    virtual void setIntensity(Color intens);
    virtual void setPosition(Point pos);
    virtual bool isSpotlight();
    virtual bool isAreaLight();
    virtual Vector3D getDirection() { return Vector3D(0,0,0); };
    virtual Real getAngle() { return 0; };
```

```

    virtual void setPhotons(int photon_power);
    virtual int getPhotons();
    virtual void setCausticPhotons(int caustic_photons);
    virtual int getCausticPhotons();
    virtual Real getIntensitySum();
    virtual vector<Ray*>* samplePhotonDirections();
    virtual vector<Ray*>* sampleCausticPhotonDirections();
    virtual vector<Point*>* sampleSurfacePoints(int samples);
    virtual Color* getPhotonPower();
    virtual Vector3D getNormal();
    virtual vector<Ray*>* getSamples();
    virtual vector<Point*>* getSurfaceSamples();
};

```

Algorithmus 5.5: Aufbau der Klasse *Light*

Der Konstruktor und die Membervariablen

Eine rudimentäre Punktlichtquelle wird bestimmt durch ihre Position und ihren Lichtstrom. Da diese Klasse Basisklasse aller Lichtquellentypen ist und spätere Typüberprüfungen teuer sind, wird der exakte Typ durch zwei boolsche Variablen `m_spotlight` und `m_arealight` bestimmt. Jede Lichtquelle bekommt ihren Anteil an in der Szene versendeten *Global Photons* und *Caustic Photons* zugewiesen. Diese Werte werden in den Variablen `m_photons` und `m_caustic_photons` gespeichert. Zur Bestimmung dieser Anzahl ist der gesamte Lichtstrom notwendig – je heller die Lichtquelle, desto mehr Photonen-, deren Wert in `m_sum_of_intensity` gespeichert wird. Des Weiteren ist die Energie jedes einzelnen Photons durch `m_photon_power` festgelegt. Jeder Farbkanal wird mit der Anzahl auszusendender Photonen skaliert.

Des Weiteren nimmt der STL-Vektor `vector<Ray*> *photondirlist` die gesampleten Richtungen und die Position der Lichtquelle als Ursprung der global auszusendenden Photonen auf. `vector<Ray*> *causticphotondirlist` ist das Äquivalent für die Samples in Richtung spekularer Materialien. Das Objekt `m_sample` dient der Erzeugung spezieller Richtungen für die Emission der *Caustic Photons*. Die Klasse *Sample* wird in Kapitel 5.2.5 vorgestellt.

In den Unterklassen kommen nur wenige neue Membervariablen hinzu. Im Wesentlichen ändern sich dort die Routinen zum Samplen der Photonen.

Die Methoden

```
vector<Ray*>* samplePhotonDirections();
```

Da der Ursprung der Photonen bereits durch die Position der Punktlichtquelle festgelegt ist, müssen lediglich noch zufällige Richtungen gefunden werden. *Monte-Carlo-Sampling* ist hierzu am besten geeignet. Die Implementation richtet sich nach dem in Kapitel 3.3.1.1 vorgestellten Verfahren *rejection sampling*. Position und Richtung eines Photons werden als *Ray* in den Vektor `photondirlist` geschrieben.

```

vector<Ray*>* Light::samplePhotonDirections()
{
    //Zufallskoordinaten im Bereich [-1,1]
    Real x_dir = 0.0; Real y_dir = 0.0; Real z_dir = 0.0;
    //Zufallsvektor (=Richtungsvektor)
    Vector3D *random_direction;
    //Zufallsstrahl (=Richtung und Ursprung eines Photons)
    Ray *photonray;
    //Emittieren der Photonen
    for(int i=1; i<=this->getPhotons(); i++)
    {
        //Wahl zufaelliger Richtung an der Stelle des Ursprungs
        //mittels rejection sampling
        do{
            x_dir = 2*((Real)rand() / RAND_MAX) - 1;
            y_dir = 2*((Real)rand() / RAND_MAX) - 1;
            z_dir = 2*((Real)rand() / RAND_MAX) - 1;
        }
        while(x_dir * x_dir + y_dir * y_dir + z_dir * z_dir > 1);
        delete random_direction;
        random_direction = new Vector3D( x_dir, y_dir , z_dir );
        photonray = new Ray(this->m_Position, *random_direction);
        photondirlist->push_back(photonray);
    }
    return photondirlist;
}

```

Algorithmus 5.6: *rejection sampling*

```
vector<Ray*>* sampleCausticPhotonDirections();
```

Für die Emission von Photonen in Richtung spekulärer Materialien wird ein anderes Verfahren verwendet. Der Ansatz stammt aus [Marlon03] und wird in der Klasse *Sample* implementiert. Es werden Punkte auf einer gedachten Kreisscheibe gesamplet, deren Mittelpunkt durch den Mittelpunkt und deren Radius durch die Ausdehnung des spekulären Objektes bestimmt ist und die zur Lichtquelle hin ausgerichtet ist.

```

vector<Ray*>* Light::sampleCausticPhotonDirections()
{
    ...
    //Samples auf Kreisscheibe erzeugen
    m_sample = new Sample(this->getCausticPhotons(), 0.0, 1.0, 1.0,
                        this->getPosition());
    m_sample->generateSamples();
    vector<Vector3D*>* samples = m_sample->getSampleList();
    ...
}

```

Algorithmus 5.7: Erzeugen eines *Sample*-Objekts


```
vector <Point*>* sampleSurfacePoints(int samples);
```

Die Punktlichtquelle hält für ihre Unterklassen einen Methodenrumpf bereit. Mit dieser Methode soll es möglich sein, Punkte auf der Oberfläche der Lichtquelle zu sampeln. Wichtig ist dies für alle Formen von Flächenlichtquellen, zu denen während des Shading mehrere, unterschiedliche Schattenfühler geschickt werden müssen, um den Grad der Leuchtdichte eindeutig messen zu können. Hierdurch ist es zudem möglich, Halbschatten darzustellen.

Die Methoden für Lese- und Schreiboperationen sind selbsterklärend und werden daher nicht weiter dokumentiert.

5.2.3 AreaLight

Bisher kennt der Raytracer nur zwei Arten von Lichtquellen: Punktlichtquellen und Spotlights. Punktlichtquellen haben den Nachteil, dass sie das Licht uniform in der gesamten Szene verteilen. Der Fluss des Lichtes kann nicht gesteuert werden. Spotlights können durch die Angabe ihres Öffnungswinkels zwar den Fokus auf einen bestimmten Ausschnitt der Szene lenken, sind in der Szene selbst aber wie die Punktlichtquellen nicht sichtbar. Diese beiden Nachteile gleichen die Flächenlichtquellen aus.

In der Spezifikationsphase gab es Überlegungen, die Klasse *AreaLight* polymorph von *Light* und *SceneItem* abzuleiten. Dadurch sollte es möglich sein, eine Flächenlichtquelle ebenso wie ein geometrisches Primitiv zu handhaben und zu rendern. In der Praxis ist dieser Versuch allerdings aus diversen Gründen gescheitert. Daher wird aus den drei Eckpunkten der Dreiecksgrundfläche ein Dreieck als geometrisches Primitiv erzeugt, welches im Loader separat in die Liste der *SceneItems* eingefügt werden muss. Die Photonen emittierende Lichtquelle und deren zu rendernde Fläche werden also getrennt behandelt. Diese Trennung führt allerdings zu einem weiteren Problem, welches in einem der kommenden Unterkapitel behandelt wird: die Korrektheit der Schattenfühler.

Diverse Methoden, die im Release nicht mehr nötig sind, blieben dennoch im Quellcode erhalten.

```
class AreaLight : public Light ,public SceneItem
{
public:
    AreaLight();
    AreaLight( Color intens, Point &v1, Point &v2, Point &v3 );
    AreaLight( Color intens, Vector3D normal, Point &v1, Point &v2,
              Point &v3 );
    virtual ~AreaLight();
    virtual LocalGeometry* intersect(Ray* ray);
    virtual bool shadowIntersect(Ray *ray);
    virtual vector<Ray*>* samplePhotonDirections();
    virtual vector<Ray*>* sampleCausticPhotonDirections();
    virtual vector <Point*>* sampleSurfacePoints(int samples);
    virtual BoundingBox* inquireBounds();
    LocalGeometry* photontrace(Photon ph, Vector3D ph_dir);
    virtual Vector3D getNormal();
    virtual vector<Ray*>* getSamples();
    virtual vector<Point*>* getSurfaceSamples();
    virtual Material* getMatPointer();
```

```

        virtual void setMatPointer(Material *mat);
        virtual Triangle* getPlainObject();

protected:
    BoundingBox *m_box;    /**< BoundingBox der Lichtquelle */
    Point m_vertices[3]; /**< Ortsvektoren der drei Eckpunkte */
    Triangle* m_triangle; /**< Abbildung der Lichtquelle auf ein Dreieck */
    vector <Point*> *samplelist; /**< Liste mit zufaelligen
                                   Oberflaechenpunkten */
    Vector3D m_normal; /**< Flaechennormale */
    Real m_area;         /**< Flaecheninhalt */
    Material* mat_pointer; /**< Zeiger auf Materialtyp der Lichtquelle */
};

```

Algorithmus 5.8: Die Klasse *AreaLight*

Der Konstruktor und die Membervariablen

Der Konstruktor von *AreaLight* bildet aus den drei Eckpunkten der Flächenlichtquelle ein Dreieck als geometrisches Primitiv. Um im Loader den Überblick zu behalten, wurden alle Methoden, die für das Rendern geometrischer Primitive notwendig sind, in die Klasse *AreaLight* übernommen. So besitzt die Flächenlichtquelle einen Zeiger auf das Material und eine Oberflächennormale zur Ausrichtung der Lichtquelle. Zur Emission der Photonen wird die Normale dem Konstruktor mitgegeben und zur Intersektion automatisch in der *intersect()*-Methode des Primitivs berechnet. Ebenso besitzt die Flächenlichtquelle eine Bounding Box für den Intersektionstest. Alle anderen Daten können mit Methoden der Basisklasse übergeben werden.

Die Methoden

```

LocalGeometry* intersect(Ray* ray);
bool shadowIntersect(Ray *ray);
BoundingBox* inquireBounds();

```

Wie bereits erwähnt, werden in dieser Klasse die für die Intersektionstests notwendigen Methoden der Klasse *Triangle* reimplementiert. Allerdings werden hier nur die Methoden des geometrischen Primitivs aufgerufen und die Daten des Dreiecks weitergereicht.

```

vector<Ray*>* samplePhotonDirections();

```

Gegenüber den Punktlichtquellen müssen auf der Oberfläche der Flächenlichtquelle zusätzliche Punkte als Ursprünge für die Photonen ermittelt werden. Als Algorithmus dient das von Peter Shirley auf der SIGGRAPH 2003 vorgestellte Verfahren zum Samplen von zufälligen Punkten auf einer Dreiecksfläche [Shirley03]. Die von Shirley ermittelten Zufallszahlen werden in die Ebenengleichung eingesetzt, woraus sich folgender Term für den gesamplen Punkt ergibt:

$$p_0 = p_0 + u * (p_1 - p_0) + v * (p_2 - p_0) \quad (5.2)$$

mit

- $u = 1 - \sqrt{1 - \xi_1}$
- $v = (1 - u) * \xi_2$

Auf die Transformation der Zufallszahlen $\xi_1 \wedge \xi_2 \in [0,1]$ zu den Koeffizienten u und v geht Shirley nicht näher ein.

Nachdem ein zufälliger Punkt auf der Oberfläche gefunden worden ist, wird mittels zufälligem Sampling der Winkel der Polarkoordinaten die neue Richtung bestimmt (*cos-Verteilung*). Die Wahrscheinlichkeit des Samplens einer bestimmten Richtung ist proportional zu dem Cosinus des Winkels zur Normalen der Fläche und somit wahrscheinlicher für die Richtung der Normalen. Ein Quellcodeauszug zu diesem Verfahren findet sich in Algorithmus 5.10.

```
vector <Ray*>* AreaLight::samplePhotonDirections()
{
    ...
    for(int i=1; i<=this->getPhotons(); i++){
        psi_three = (Real)rand() / RAND_MAX;
        psi_four = (Real)rand() / RAND_MAX;
        theta = acos(sqrt( 1 - psi_three ));
        phi = 2 * PI * psi_four;
        x_dir = sin(theta) * cos(phi);
        y_dir = cos(theta);
        z_dir = sin(theta) * sin(phi);... }
}
```

Algorithmus 5.9: *explicit sampling* mit Cosinus-Verteilung

```
vector<Ray*>* sampleCausticPhotonDirections();
```

Im Unterschied zum Sampling global zu emittierender Photonen werden in dieser Methode nur die Ursprünge der Photonen generiert. Die hierzu passenden Richtungen werden von der Klasse *Sample* ermittelt.

```
vector <Point*>* sampleSurfacePoints(int samples);
```

Um während des Shading eine bestimmte Anzahl an Schattenfühlern zur Lichtquelle zu schicken und die Leuchtdichte am Schnittpunkt exakter zu bestimmen, generiert diese Methode eine gewünschte Anzahl an Zufallspunkten auf der Oberfläche.

```
LocalGeometry* photontrace(Photon ph, Vector3D ph_dir);
```

Der Raytracer behandelt Photonen ähnlich wie Sehstrahlen. Sie werden anhand ihres Ursprungs und ihrer Richtung durch die Szene verfolgt. Zwar haben die Photonen ihre Richtung als Polarkoordinaten gespeichert, doch erschien es einfacher, diese direkt im Weltkoordinatensystem zu verarbeiten, da mit Polarkoordinaten während des Tracing nicht gearbeitet wird. Ein Aufruf der Methode `photon_dir()` aus der Photon Map wäre somit teurer. Daher werden an die Methode zwei Parameter übergeben, die im Anschluss zu einem *Ray*-Objekt konvertiert und an die jeweiligen `intersect()`-Methoden weitergeleitet werden. Zurückgeliefert werden also alle am Schnittpunkt ermittelten Daten. Diese Methode wurde in der abstrakten Klasse *SceneItem* festgelegt und muss von allen geometrischen Primitiven und den Beschleunigern implementiert werden.

```
Triangle* getPlainObject();
```

Diese Leseoperation wird im Loader besonders wichtig. Da Lichtquellen und ihre Primitive getrennt behandelt werden, muss die Grundform der Lichtquelle in der separaten *SceneItem*-Liste abgespeichert werden. Um die Fläche nicht extra als Dreieck definieren zu müssen, liefert `getPlainObject()` jenes zurück.

5.2.4 Spotlight

Das Verhalten von Spotlights während der Emission von Photonen gleicht dem von Punktlichtquellen. Während des *rejection sampling* muss zusätzlich überprüft werden, ob sich die gefundene Richtung auch innerhalb des Öffnungswinkels befindet. Die hierfür verantwortliche Methode arbeitet wie `inCone()` aus Algorithmus 4.15. Ein Nachteil dieses Vorgehens ist, dass der Aufwand proportional steigt, je kleiner der Öffnungswinkel ist.

5.2.5 Sample

Die in der Klasse *PhotonMap* implementierte Methode zum Samplen von Richtungen zu spekularen Objekten ist sehr teuer und war ein erster Versuch. Marlon schlägt dagegen vor, den Mittelpunkt und die maximale Ausdehnung eines zu samplenden Objektes zu bestimmen und mit diesen Daten eine gedachte Kreisscheibe zu generieren, die zur Lichtquelle hin ausgerichtet ist [Marlon03]. Auf dieser Kreisscheibe werden schließlich uniform verteilte Punkte generiert, mit welchen die Richtungen berechnet werden können. Angesichts der Tatsache, dass die Kreisscheibe einen sehr großen Teil des Objektes überdecken wird, werden die meisten Photonen treffen können.

```
class Sample
{
public:
    Sample();
    Sample(const Point &lightsource);
    Sample(int num, const Point &lightsource);
    Sample(int num, Real d, Real s, Real r, const Point &lightsource);
    Sample(int num, Real d, Real s, Real r);
    ~Sample();
    void generateSamples();
    void generateAreaSamples(vector <Point*>* pointlist);
    void setSampleIndices(Real d, Real s, Real r);
    void setSampleNumber(int num);

    vector<GeomObject*>* getSpecularList();
    vector<Vector3D*>* getSampleList();

private:
    vector<SceneItem*>* m_itemlist;
    vector<GeomObject*> m_specularlist;
    vector<Vector3D*>* m_samplelist;
    Real m_diffus;
    Real m_specular;
```

```

Real m_refract;
Vector3D m_span;
Point m_lightsource;
int m_photons;

void defineOrthogonal(const Vector3D &v);
};

```

Algorithmus 5.10: Die Klasse *Sample*

Die Konstruktoren

Jedes *Sample* benötigt die Position der Lichtquelle, von welcher die Photonen emittiert werden, die Anzahl der zu erzeugenden Samples und die maximalen Reflexionsindizes, die Materialien haben dürfen. Eine Ausnahme gilt für Flächenlichtquellen: anstatt einer einzigen Position wird eine Liste mit Oberflächensamples wird an die Methode `generateAreaSamples()` übergeben.

Die Methoden

```
void generateSamples();
```

Zur Analyse der Materialien wird die Liste der *SceneItems* benötigt. In einem iterativen Durchlauf werden diejenigen *GeomObjects* gesucht, deren Materialien als Maxima die geforderten Reflexionsindizes besitzen, und in den neuen STL-Vektor `m_specularlist` eingefügt. Im Anschluss werden die Samples gleichmäßig auf die Objekte der Liste verteilt. Für jedes Objekt wird nun die *BoundingBox* abgefragt, deren Mittelpunkt bestimmt und der Vektor $\overline{ML}$ vom Mittelpunkt zur Lichtquelle errechnet. Der Radius der Kreisscheibe ergibt sich aus der Hälfte der Länge des Vektors von *LLF* (lower left front) zu *URB* (upper right back) der *BoundingBox*. Für jedes Sample wird nun eine Orthogonale zu $\overline{ML}$ gesucht, welche auf der Kreisscheibe liegt. Diese muss noch mit einer Zufallszahl im Bereich $[0, \text{radius}]$ skaliert werden. Die gesuchte Richtung ist der Vektor $\overline{LS}$ von der Lichtquelle zum Punkt auf der Kreisscheibe.

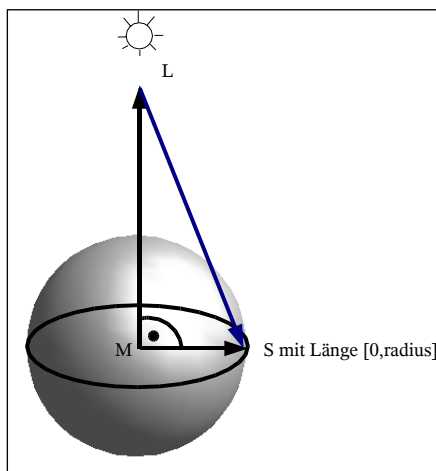


Abb. 5.2: Sampling auf Kreisscheibe

```
void generateAreaSamples(vector <Point*>* pointlist);
```

Diese Methode verhält sich ähnlich wie `generateSamples()`, mit dem Unterschied, dass für eine Liste von Oberflächensamples zufällige Richtungen gefunden werden müssen.

```
void defineOrthogonal(const Vector3D &v);
```

Durch Freistellung des Skalarprodukts $\langle v, w \rangle = 0$ lässt sich zufällig eine Orthogonale zu einem gegebenen Vektor finden. Einzige Bedingung ist, dass mindestens eine Koordinate ungleich 0 ist.

```
void Sample::defineOrthogonal(const Vector3D &v)
{
    //Bedingung fuer orthogonalen Vektor: Skalarprodukt = 0
    //Der Lichtvektor wird als Normale zur Kreisscheibe betrachtet.
    //n1 * x1 + n2 * x2 + n3 * x3 = 0
    Real v1 = 0.0; Real v2 = 0.0; Real v3 = 0.0;
    //Fallunterscheidung - wenn eine Koordinate == 0.0 ist
    if(v.getX() != 0.0)
    {
        v2 = 2.0*(Real) rand() / RAND_MAX - 1;
        v3 = 2.0*(Real) rand() / RAND_MAX - 1;
        v1 = (-v.getY()*v2 - v.getZ()*v3) / v.getX();
    }
    else if(v.getY() != 0.0)
    {
        v1 = 2.0*(Real) rand() / RAND_MAX - 1;
        v3 = 2.0*(Real) rand() / RAND_MAX - 1;
        v2 = (-v.getZ()*v3 - v.getX()*v1) / v.getY();
    }
    else if(v.getZ() != 0.0)
    {
        v1 = 2.0*(Real) rand() / RAND_MAX - 1;
        v2 = 2.0*(Real) rand() / RAND_MAX - 1;
        v3 = (-v.getX()*v1 - v.getY()*v2) / v.getZ();
    }
    ...
}
```

Algorithmus 5.11: Sampling einer Orthogonalen

5.2.6 Scene

An die Szene werden ebenso zusätzliche Anforderungen gestellt. Sie muss die Photonen an ihren Container weiterleiten, die Photonen unter den Lichtquellen aufteilen und während der Intersektionstests zwischen Primär- und Sekundärstrahlen unterscheiden.

```
//Zusätzliche Methoden
vector <SceneItem*>* getLightObjectList();
void addLightObjectList(vector <SceneItem*>* items);
LocalGeometry* primary_intersect(Ray *ray);
void photonScale(int globalphotons);
void causticPhotonScale(int causticphotons);
```

```

        LocalGeometry* photontrace(Photon ph, Vector3D ph_dir);

//Zusätzliche Membervariablen
        vector<SceneItem*> *m_lightObjectList;
        vector<SceneItem*> *m_itemList;
        vector<SceneItem*> *m_globalList;
        Container* m_lightContainer;

```

Algorithmus 5.12: Zusätzliche Methoden und Membervariablen in der Klasse *Scene*

Zusätzliche Methoden

```
LocalGeometry* photontrace(Photon ph, Vector3D ph_dir);
```

Wie jedes *SceneItem* muss *Scene* eine Methode zum Verfolgen von Photonen implementieren, die Parameterdaten an die `intersect()`-Methode des Containers weiterreicht.

```

vector<SceneItem*>* getLightObjectList();
void addLightObjectList(vector<SceneItem*>* items);
LocalGeometry* primary_intersect(Ray *ray);

```

Bei der Ermittlung der direkten Beleuchtung während des Shading entstand das Problem, dass manche Schnittpunkte mehrfach von ein und derselben Flächenlichtquelle beleuchtet wurden, wenn die Sekundärstrahlen das Primitiv der Lichtquelle getroffen und die Leuchtdichte an dessen Schnittpunkten zurückgeliefert haben. Daher mussten diese Primitive bei der Verfolgung der Sekundärstrahlen exkludiert werden. Die Methode `addLightObjectList()` nimmt eine vom Loader generierte Liste mit den Lichtquellenprimitiven auf und baut zwei verschiedene STL-Vektoren auf: einer mit allen Objekten der Szene und einer ohne die Lichtquellen. Der Raytracer kann nun anhand der Rekursionstiefe entscheiden, welche Strahlen gerade verfolgt werden und den passenden Container verwenden. Die Methode `primary_intersect()` dient dem Verfolgen der Primärstrahlen.

```

void photonScale(int globalphotons);
void causticPhotonScale(int causticphotons);

```

Die in der Kommandozeile festgelegten Werte für die Anzahl global und spekulär verteilter Photonen werden von der Szene auf die in ihrer *LightItem*-Liste vorhandenen Lichtquellen prozentual nach Höhe ihres jeweiligen gesamten Lichtstroms verteilt.

5.2.7 Raytracer

Die Klasse *Raytracer* kontrolliert den Durchlauf des *Photon Tracing* und des *Rendering*. Im *First Pass* müssen die Photonen von den Lichtquellen verschossen und verfolgt werden. Im *Second Pass* werden dann die Sehstrahlen verfolgt und sammeln zusätzlich die Leuchtdichte der Photonen ein. Da nur das Material *PhotonMaterial* in der Lage ist, diese Konditionen zu handhaben, darf in einer Szene nur dieses Material spezifiziert sein. Alle anderen Materialien werden ignoriert. Um den korrekten Ablauf zu kontrollieren, wurden daher drei neue Methoden eingeführt.

In der Methode `getImage()` überprüft der Raytracer, in welchem Modus er sich befindet. Ist Photon Mapping als Option eingeschaltet, werden die passenden Methoden aufgerufen. Da nur die Primärstrahlen direkte Beleuchtung einsammeln dürfen, wird der Trigger `DIRECT_LIGHTING` auf `true` gesetzt. Das Material setzt diesen Wert auf `false`, wenn die Leuchtdichte zu Sekundärstrahlen evaluiert wird.

First Pass

```
void photontrace( Photon ph, Vector3D ph_dir, int rec_depth );
```

Ein Photon wird zunächst durch die Szene verfolgt und auf einen Schnittpunkt hin überprüft. Sollte es ein Objekt treffen, wird dessen Material abgefragt. Ist das Material vom Typ *PhotonMaterial* wird das Photon und seine Schnittpunktinformationen zuzüglich Rekursionstiefe an die Methode `getHit()` des Materials weitergeleitet, in welcher das Photon entweder reflektiert oder absorbiert wird. Sollte ein Treffer auf diffusem Material registriert werden, so wird das Photon zurückgeliefert und mit den neu berechneten Daten zu Einfallrichtung und Leuchtdichte in der Photon Map gespeichert.

```
void causticPhotonTrace(Photon ph, Vector3D ph_dir, int rec_depth );
```

Gleiches Vorgehen gilt für das Verschießen von *Caustic Photons*, mit dem Unterschied, dass zum einen die Photonen in der separaten `causticphotonmap` gespeichert und zum anderen der Lichtpfad abgefragt werden muss. *Caustic Photons* müssen vorher mindestens eine spekulare Fläche getroffen haben, bevor sie auf diffuses Material treffen und dort gespeichert werden. Bisher existieren für diese Analyse Zählvariablen `dHits`, `sHits` und `rHits` im Material. Bei jedem Treffer werden diese separat hochgesetzt und können vor der Speicherung abgefragt werden. Anschließend können sie wieder zurückgesetzt werden. In der Praxis geht diese Idee leider nicht immer auf. Durch die zahlreichen Rekursionen können die Treffer keinem Photon eindeutig zugeordnet werden. Besser wäre es, jedem Photon ein zusätzliches `short`-Flag mitzugeben, um spekulare Treffer festzuhalten. Allerdings erhöht dies den Speicheraufwand in der Photon Map. Eine weitere Alternative würden zwei verschiedene *Photonen-Structs* bieten: eines zum Tracen und eines für die Speicherung.

Second Pass

```
Color* photonrender(Ray *ray, int recursionDepth);
```

Eigentlich führt `photonrender()` den gleichen Algorithmus aus wie der Standard-Raytracer. Wie aber schon bemerkt, müssen Primär- und Sekundärstrahlen getrennt behandelt werden. Festzulegen ist dies leicht mit der Rekursionstiefe. Bei der ersten Rekursionsstufe werden die Primärstrahlen verfolgt und `primary_intersect()` aufgerufen, alle übrigen Strahlen wandern durch die Methode `intersect()`. Ebenso kann diese Methode nur dann aufgerufen werden, wenn die Materialien vom Typ *PhotonMaterial* sind.

5.2.8 PhotonMaterial

Diejenige Klasse, die die wichtigsten Änderungen übernimmt, ist die Klasse *PhotonMaterial*. Sie übernimmt die komplette Verwaltung von *First Pass* und *Second Pass*. Ohne die Spezifikation von *PhotonMaterial* für die Objekte im Loader kann *Photon Mapping* nicht durchgeführt werden.

Die Implementation ist bewusst an die des Materials *HallMaterial* angelehnt. Bevor bestimmte Module und Verfahren in *PhotonMaterial* eingebaut wurden, sind sie in *HallMaterial* getestet worden. Unter anderem ist *HallMaterial* nun auch in der Lage, mit Flächenlichtquellen umzugehen. Ursprüngliche Funktionen wurden allerdings nicht modifiziert, so dass der Raytracer immer noch die gleiche Funktionsweise besitzt wie vor der Studienarbeit.

```
class PhotonMaterial : public Material
{
protected:
    TextureMap* m_reflectionColor;
    TextureMap* m_refractionColor;
    Real m_refractionIndex;
    Real m_diffusIndex;
    Real m_specularIndex;
    Real m_RefractIndex;
    Real m_roughness;
    Real m_isotropy;
    bool m_hit;
    int m_diffuse_hits;
    int m_specular_hits;
    int m_refract_hits;

public:
    PhotonMaterial(TextureMap* ambient, TextureMap* diffuse,
                  TextureMap* specular, Real glossiness);
    virtual ~PhotonMaterial();
    virtual Color* getColor(Ray* ray, LocalGeometry* lg,
                          int recursionDepth);
    virtual Photon getHit( Photon ph, LocalGeometry *lg, int rec_depth );
    virtual void setRefractionColor(TextureMap* refract);
    virtual void setDiffusionColor(TextureMap* diffus);
    virtual void setRefractionIndex(Real refractionIndex);
    virtual void setSpecularIndex(Real specIndex);
    virtual void setDiffusIndex(Real diffusionIndex);
    void setDSRRIndices(Real diffusionIndex, Real specIndex,
                      Real refractionIndex, Real R_indice);
    void setRoughness(Real roughness);
    void setIsotropy(Real isotropy);
    bool isHit();
    void getMaterialHits(int &d_hits, int &s_hits, int &r_hits);
    void resetMaterialHits();
    void setTransmissionInside(bool inside);
    void getDSRRIndices(Real &d_indice, Real &s_indice, Real &r_indice,
                      Real &R_indice);
    Real calcSchlickBRDF(Vector3D incoming_dir, Vector3D ref_light_vec,
                      Vector3D intersect_normal);
    Vector3D calcTransVec(Vector3D incoming_dir,
                      Vector3D intersect_normal);
    void sampleHemisphere(Real &v1, Real &v2, Real &v3,
                      const Vector3D &intersect_normal);

private:
    Photon scaleEnergy(Photon ph);
```

```
bool inCone(Light* l, Vector3D lightVec);  
bool getDeviation(Vector3D v, Vector3D lightVec);  
bool m_inside_trans_mat;  
};
```

Algorithmus 5.13: Die Klasse *PhotonMaterial*

Die Membervariablen

PhotonMaterial erbt von der Basisklasse die drei *TextureMaps* *ambient*, *diffuse* und *specular* zur Annäherung der BRDF's bei der direkten Beleuchtung und *glossiness* als Phong-Koeffizienten. Die *TextureMaps* *m_refractionColor* und *m_reflectionColor* werden mit Hilfe des Fresnel-Terms berechnet und bestimmen die Menge des transmittierten und reflektierten Lichts bei der Verfolgung von Sekundärstrahlen.

Bei der Verfolgung der Sehstrahlen wird Path Tracing verwendet, um diffuse Reflexionen zu ermöglichen. Zwar schlägt Jensen einen Distribution Raytracer vor, der bei diffuser Reflexion mehrere Strahlen verfolgt, um Rauschen zu verringern, doch reichen in dieser Version des Path Tracers bereits 10 Strahlen pro Pixel aus, um die Artefakte weitestgehend zu eliminieren. Die Daten aus der Photon Map gleichen die fehlenden Werte auf diffusen Flächen aus. Der Grad der jeweiligen Reflexionsart wird durch die Indizes *m_diffuseIndex*, *m_specularIndex* und *m_RefractIndex* festgelegt. Den Brechungsindex des Materials bei Transmission definiert *m_refractionIndex*. Da in *HallMaterial* nur Refraktionen im Übergang von Vakuum / Luft zu festem Material berechnet werden konnten, ist die boolsche Variable *m_inside_trans_mat* eingeführt worden, die sich den Status des Sehstrahls merkt und somit den reversen Übergang ermöglicht.

Für das lokale Beleuchtungsmodell von Schlick werden zwei weitere Parameter benötigt: *m_roughness* für die Oberflächenbeschaffenheit des Materials (0 = absolut glatt und spekulär, 1 = sehr rau und Lambertstrahler) und *m_isotrophy* für den Isotropiefaktor des Materials (0 = absolut anisotropisch und somit nicht rotationssymmetrisch, 1 = isotropisch und somit rotationssymmetrisch).

Photonen werden bei Treffern mit diffusen Oberflächen gespeichert und mit der boolschen Variable *m_hit* gemerkt. Die Anzahl und die Art der Treffer, die ein Photon auf seinem Weg hatte, werden in den Variablen *m_diffuse_hits*, *m_specular_hits* und *m_refract_hits* festgehalten.

Der Konstruktor

Das Standard-Material ist eine Lambertfläche mit absolut diffusem Reflexionsgrad, absolut rau und isotropisch. Der Materialtyp ist 4. Es wird davon ausgegangen, dass der Sehstrahl außerhalb des Materials startet, weswegen Übergänge von Material zu Material noch nicht möglich sind.

Zusätzliche Methoden

```
Vector3D calcTransVec(Vector3D incoming_dir, Vector3D intersect_normal);
```

Anhand einer Einfallrichtung und einer Schnittpunktnormalen wird mit Hilfe von Snell's Gesetz der transmittierte Strahl eines Materials berechnet. Die Berechnungsvorschrift findet sich in Gleichung (4.1).

Es müssen mehrere Fälle unterschieden werden: Zum einen, ob sich der ankommende Sehstrahl innerhalb oder außerhalb des Materials befindet – ist er innerhalb muss der Kehrwert der Refraktionsindices η_1 und η_2 zur Berechnung genommen werden; zum anderen, ob der Wert unterhalb der Wurzel negativ oder positiv ist – ist der Wert negativ, ist das Ergebnis eine Totalreflexion und der Strahl wird stattdessen reflektiert.

Der Anteil transmittierten Lichts, den der refraktierte Strahl trägt, kann mittels Fresnel-Approximation berechnet werden. Er beträgt $1 - F_r$.

```
void sampleHemisphere(Real &v1, Real &v2, Real &v3,
                      const Vector3D intersect_normal);
```

Diese Methode realisiert dieselbe Berechnung wie `defineOrthogonal()` aus der Klasse *Sample*. Es wird eine Orthogonale zur Schnittpunktnormalen nach dem Zufallsprinzip bestimmt. Zur Projektion des zufälligen Punktes auf die Hemisphäre um die Schnittpunktnormale müssen Normale und Orthogonale noch addiert werden.

```
Real calcSchlickBRDF(Vector3D incoming_dir, Vector3D ref_light_vec,
                    Vector3D intersect_normal);
```

Wie von Jensen vorgeschlagen, wird das lokale Beleuchtungsmodell von Schlick zur Annäherung der BRDF verwendet [Schlick93]. Das Modell wird entgegen der Implementation in *HallMaterial* vollständig realisiert. Dort wurde lediglich der Faktor $G(v)$ berücksichtigt, der eigentlich eine *roughness* im Wertebereich von $[0, 1]$ und den Cosinus zwischen Einfallrichtung $\bar{w}$ und Normale $\bar{n}$ benötigt. Berechnet wurde er allerdings mit dem Phong-Koeffizienten und dem Winkel zwischen dem Halbvektor $\bar{H}$ und der Normalen.

Nach ihrer Definition liegt eine BRDF im Wertebereich $[0, \infty[$. Diese *physikalisch mögliche* Skalierung ist in diesem System allerdings nicht verwendbar, da die Leuchtdichten in dem Wertebereich $[0, 1]$ liegen müssen. Der *Hemisphärische Reflexionsgrad* mit einem Wertebereich $[0, 1]$ ist deshalb besser geeignet. Er wird definiert als

$$\rho(\theta_i) = \frac{\phi}{d^2 \phi_i(\theta_i)} = \int_{2\pi} f_r(d\bar{w}_i, d\bar{w}_o) \cos \theta_o d\bar{w}_o \quad (5.1)$$

Im folgenden wird die Bezeichnung BRDF für alle Arten von Reflexionsgraden als Synonym verwendet.

Während des *First Pass* wird der Wert nicht eingerechnet, denn der Grad der Reflexion ergibt sich aus den im Loader für das Material definierten Roulette-Werten. Alternativ könnten auch die nach Schlick aus Gleichung (2.23) bestimmten Grade d , s und g als Roulette-Werte dienen, doch soll dem Benutzer im Loader möglichst viel Einfluss auf die Bestimmung der Materialien gegeben werden. Im *Second*

Pass dagegen ersetzt diese BRDF die bisherigen, angenäherten Werte der *TextureMaps*, allerdings auch nur bei der Berechnung der direkten Beleuchtung.

Des Weiteren bindet Schlick den Fresnel-Term in seine BRDF ein. Dieser wird extern in `calcFresnel()` berechnet.

```
Photon scaleEnergy(Photon ph);
```

Bei der Feststellung der Leuchtdichten kann es dazu kommen, dass das Resultat das festgelegte Intervall $[0, 1]$ überschreitet. In dieser Methode wird es skaliert. Für den *Second Pass* ist der Quellcode hierzu in die Methode `getColor()` implementiert.

First Pass

```
Photon getHit( Photon ph, LocalGeometry *lg, int rec_depth );
```

In dieser Methode wird der von einem Photon getragene Lichtstrom in die Szene verteilt, nachdem das Photon ein Objekt getroffen hat. Sollte die maximale Rekursionstiefe in diesem Aufruf erreicht sein, erfolgt keine weitere Reflexion und das Material wird auf Diffusivität hin überprüft. In diesem Fall wird das Photon nochmals in der Photon Map ohne Änderung seines Lichtstromanteils gespeichert.

Andernfalls entscheidet *Russisches Roulette* darüber, ob und wie das Photon reflektiert wird. Entscheiden Zufallszahl und diffuser Reflexionsindex, dass das Photon diffus reflektiert und in der Photon Map gespeichert werden soll, wird zuerst eine zufällige neue Richtung bestimmt. Die Energie des demnächst neu verschossenen Photons errechnet sich aus der ankommenden Energie, skaliert mit dem diffusen Reflexionsindex und einer Cosinus-Gewichtung (5.3):

$$\cos_g \alpha = \frac{\langle \vec{v}, \vec{w} \rangle}{\|\vec{w}\|} \quad (5.3)$$

mit

- $\vec{v}$ Schnittpunktnormale
- $\vec{w}$ neuer Richtungsvektor

Es wird ein neues Photon mit aktueller Schnittpunktposition, neuem Richtungsvektor und eben berechneter Energie erzeugt und rekursiv losgeschickt. Das alte, angekommene Photon wird an den Raytracer mit seiner Trefferposition zurückgeliefert, dort in der Photon Map gespeichert und anschließend gelöscht.

Ist die Entscheidung des *Russischen Roulette* auf spekulare Reflexion gefallen, gilt im Prinzip gleiches Vorgehen wie bei der diffusen, mit dem Unterschied, dass die Reflexionsrichtung perfekt gespiegelt ist und das Photon nicht gespeichert, sondern nur reflektiert wird. Die Cosinus-Gewichtung fällt dagegen weg.

Hat sich das *Russische Roulette* für die Transmission entschieden und der Brechungsindex ist größer als der des Vakuums, wird das Photon an der Oberfläche gebrochen und mit neu berechneter Energie weiterverfolgt. Gegenüber herkömmlichem Raytracing wird also nur ein Strahl weiterverfolgt.

Tritt keine dieser Reflexionsarten ein, wird das Photon an der Oberfläche absorbiert.

Second Pass

```
Color* getColor(Ray* ray, LocalGeometry* lg, int recursionDepth);
```

Nachdem alle Photonen verschossen und gespeichert worden sind, sammelt der Raytracer die Leuchtdichten unter Einbeziehung direkter Beleuchtung und reflektierter Werte wieder ein. Im Falle dessen, dass gerade die Schnittpunkte von Primärstrahlen zu untersuchen sind, wird für die direkte Beleuchtung über alle Lichtquellen iteriert. Flächenlichtquellen sind in diesem Zusammenhang ein Sonderfall: Ein Schnittpunkt kann von ihr auch nur partiell beleuchtet werden. Um dies festzustellen, müssen mehrere Teststrahlen zur verschiedenen Punkten auf der Lichtoberfläche geschickt werden. Für das Sampling dieser Punkte wird das gleiche Verfahren angewendet wie für die Photonenursprünge, die Anzahl ist nur deutlich geringer und wird mit `ph_opts.lightrays` festgelegt. Der Lichtstrom der Lichtquelle muss dann mit der Anzahl der Teststrahlen gewichtet werden. Je nach Beschaffenheit der Lichtquelle wird die direkte Beleuchtung also unterschiedlich oft berechnet.

Ebenso gibt es eine Sonderbehandlung für Spotlights: Der Lichtstrom ist abhängig von der Abweichung des Teststrahls innerhalb des Lichtkegels.

Nachdem der Schattenfühler gemeldet hat, dass sich kein weiteres Objekt zwischen Schnittpunkt und Lichtquelle befindet, und festgestellt ist, dass die Normale nicht von der Lichtquelle wegzeigt, werden je nach Beleuchtungsmodell folgende Rechenwege verfolgt:

- Phong

```
*diffus_energy += m_diffuseColor->getColor() * intensity * angle
                  * distance_fac;
angle = ref_light_vec.dotprod(view_vector));
if(m_glossiness > 1.0 angle > 0.0)
{
    *specular_energy += m_specularColor->getColor()*intensity
                       * distance_fac * pow(angle, m_glossiness);
}
```

- Schlick

Der Winkel zwischen Normale und Schattenfühler wird bei der Erstellung der BRDF verrechnet und deswegen an dieser Stelle nicht mit einbezogen.

```
*diffus_energy += m_diffuseColor->getColor() * intensity *
                  distance_fac * brdf;
if(m_glossiness > 1.0 && (halfvector.dotprod(normal))>0.0)
{
    *specular_energy += m_specularColor->getColor()*intensity
                       distance_fac * brdf;
}
```

Die Beleuchtung wird zusätzlich mit einem Distanzfaktor d gewichtet – je weiter die Lichtquelle vom Schnittpunkt entfernt ist, desto dunkler die Beleuchtung:

$$d = \frac{1}{\|\vec{v}\|^2} \quad (5.4)$$

mit

- $\vec{v}$ Schattenfühler

Bei der Implementation hat sich allerdings herausgestellt, dass bessere Ergebnisse erzielt werden, wenn die Länge des Vektors nicht quadriert wird. Jensen definiert diesen Distanzfaktor dagegen nicht, denn eigentlich bleiben die Leuchtdichten entlang des Lichtweges konstant. In der Implementation dagegen musste er wegen Überbeleuchtung eingebaut werden.

Ist die maximale Rekursionstiefe noch nicht erreicht, kann der Strahl weiterverfolgt werden, ansonsten wird abgebrochen und die Leuchtdichte wird – wie im Absatz weiter unten beschrieben – zurückgeliefert, allerdings ohne die aus Reflexion gewonnenen Daten mit einzuberechnen. Danach gelten die Regeln des *Russischen Roulette*. Die angenäherten BRDF für Reflexion und Refraktion werden vorher mittels Fresnel-Approximation berechnet.

Wie im *First Pass* wird für einen diffus reflektierten Sehstrahl die neue Richtung durch die Projektion des aus dem Skalarprodukt gefundenen Vektors auf die Hemisphäre bestimmt. Die nach der Reflexion zurückgelieferte Farbe wird allerdings mit in das Endergebnis aufgenommen, wodurch der Effekt des *Color Bleeding* entsteht. An dieser Stelle wird nun die Leuchtdichte, die Photonen in der Photon Map abgelegt haben, abgeschätzt. Wurden zusätzlich *Caustic Photons* verschossen, so sind deren Leuchtdichten auf die der *Global Photons* aufzurechnen. Wie bereits früher erwähnt, ist es schwierig, die beiden Werte zu trennen, so dass dort, wo Kaustiken entstehen, keine Daten aus der *Global Photon Map* mit einbezogen werden. Die finale Leuchtdichte ergibt sich nun aus der Addition der direkten diffusen und spekularen Beleuchtung, der diffus reflektierten Beleuchtung und der Leuchtdichte aus den Photon Maps. Letztere wird mit dem Cosinus-Faktor aus Gleichung (5.3) und der angenäherten diffusen BRDF gewichtet.

Spekulare Materialien reflektieren wie ein perfekter Spiegel. Die endgültige Leuchtdichte ergibt sich aus der Addition der direkten diffusen und spekularen Beleuchtung und der spekulär reflektierten Beleuchtung, gewichtet mit der angenäherten spekularen BRDF.

Wird der Sehstrahl dagegen an der Oberfläche des Materials gebrochen, ergibt sich die am Schnittpunkt resultierende Farbe aus der Addition der direkten diffusen und spekularen Beleuchtung und der refraktierten Beleuchtung, gewichtet mit der angenäherten BRDF der Fresnel-Approximation.

Die Leuchtdichte wird auf das Intervall $[0, 1]$ beschränkt zurückgeliefert.

6. Resultate

Photon Mapping ist ein sehr mächtiges Verfahren, um globale Beleuchtung effizient zu berechnen und viele Effekte, die aus indirekter Beleuchtung heraus entstehen, darzustellen. Das primäre Ziel dieser Studienarbeit – die Darstellung von Kaustiken – ist für Jensen's Algorithmus ein ideales Anwendungsgebiet. Nichtsdestotrotz muss vorweg gesagt werden, dass zu einem *Raytracer* viele andere Algorithmen zur Verbesserung der Bildqualität und Renderinggeschwindigkeit beitragen. Erst das Zusammenspiel aller Komponenten ermöglicht es, gute Resultate zu erzielen. Im folgenden werden anhand von gerenderten Beispielszenen, insbesondere der Cornell-Box, die Möglichkeiten und die Schwierigkeiten des Systems aufgezeigt.

6.1 Cornell-Box

Die Cornell-Box [Goral83] ist die Standardszene zum Testen von globalen Beleuchtungsalgorithmen. Ursprünglich wurde sie zum für die Evaluation Finites Elemente Techniken (*Radiosity*) und besitzt als Objekten und Materialien zwei diffuse Seitenwände in Rot und Blau, eine Hinterwand, Decke und Boden in mittlerem Grau, zwei diffuse Quader sowie eine Flächenlichtquelle. Später wurden die Quader durch eine Spiegelkugel und eine Glaskugel ersetzt, um spekulare Objekte rendern zu können. Während der Testphase wurden die Parameter immer wieder variiert. Als Referenz dient die Cornell-Box mit kompletter globaler Beleuchtung von Jensen².

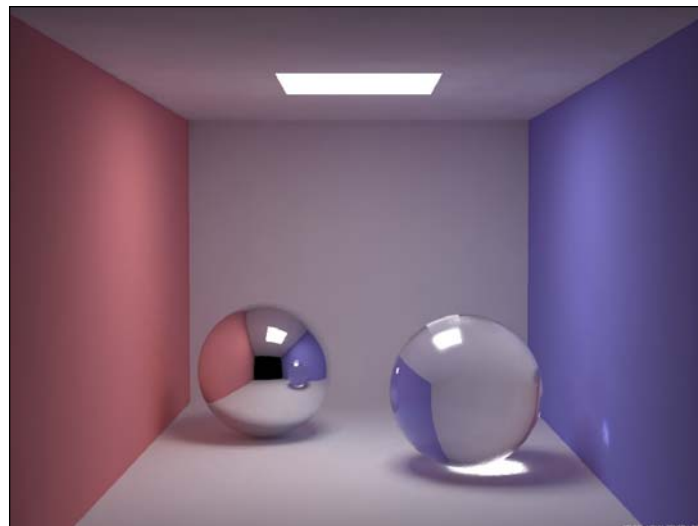


Abb. 6.1: Cornell-Box mit kompletter globaler Beleuchtung [Jensen01]

Das Basis-System aus dem Projektpraktikum lieferte ohne Monte-Carlo-Methoden das Ergebnis aus Abbildung 6.2a. Die Szene wird von einer Punktlichtquelle ausgestrahlt. Der Refraktionsindex der vorderen Glaskugel ist auf 1.6 festgelegt. Die Schatten sind hier noch sehr scharfkantig und die Reflexion in der Glaskugel wird noch nicht sehr deutlich. Der Transmissionsstrahl tritt zudem noch nicht aus dem Material heraus. Dies liegt zum einen daran, dass die Austrittsstrahlen noch nicht

² Jensen's Homepage (<http://graphics.ucsd.edu/~henrik/>)

berücksichtigt werden, und zum anderen, dass die Intersektions-Methode in *Sphere* nicht korrekt implementiert war. Sie wurde später umgeschrieben und folgt nun dem *geometrischen Ansatz* aus [Haines89].

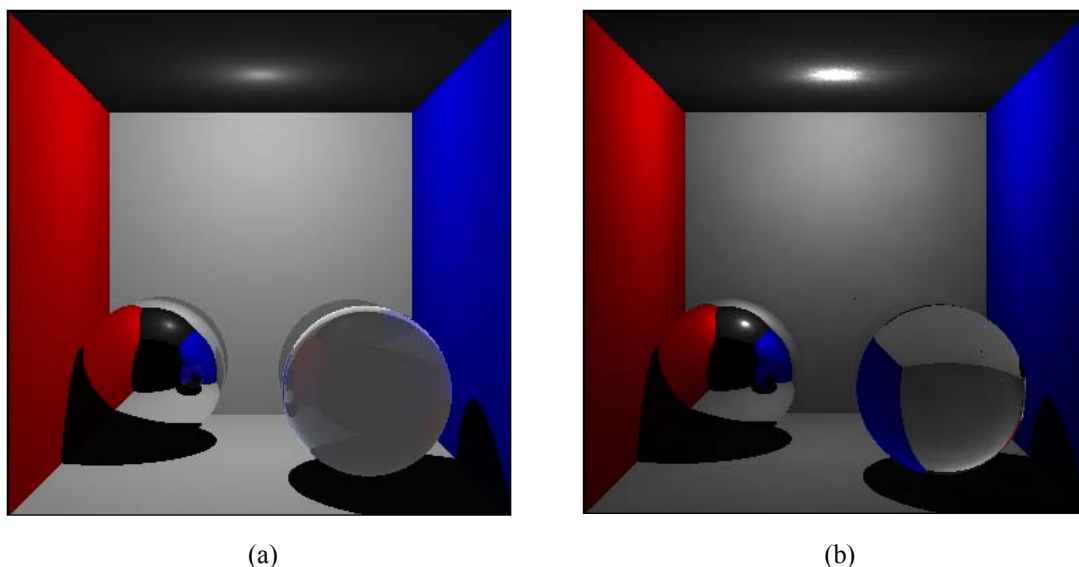
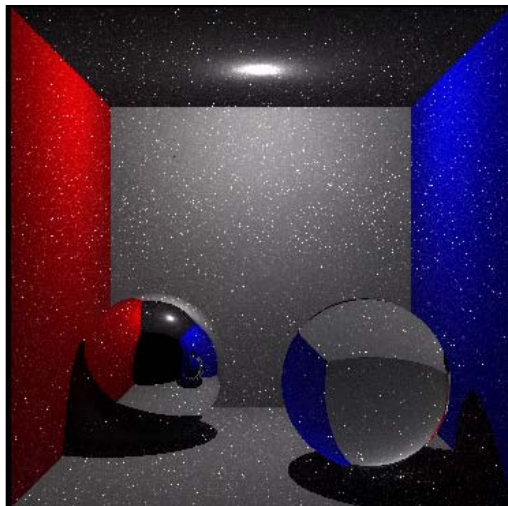


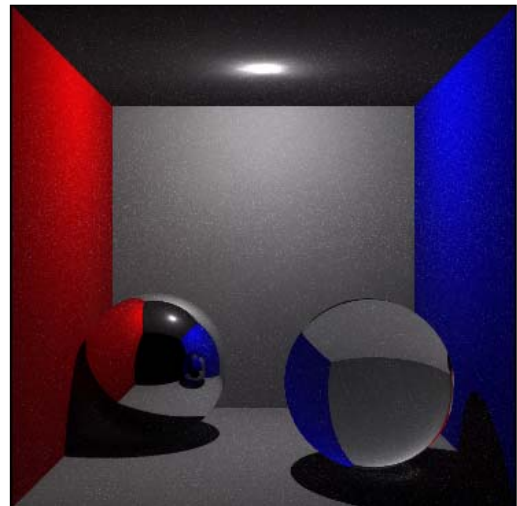
Abb. 6.2: (a) Cornell-Box des Basis-Systems
(b) Photon Mapping (ohne Kaustiken)

Die Szene mit gleichen Parametern – erzeugt mit Photon Mapping – zeigt Abbildung 6.2b. Es werden 10.000 globale Photonen verschossen und in einem Radius von 0.15 – relativ zu einer Szenenausdehnung von 2 – maximal 50 Photonen eingesammelt. Diffuse Reflexionen sind noch nicht enthalten. Wie man sieht, leuchtet die Lichtquelle um ein vielfaches heller. Dies liegt an der uniformen Photonenmission der Punktlichtquelle, die sich knapp unterhalb der Decke befindet, um nicht von jener verdeckt zu werden. Deswegen werden einige Photonen direkt in diese Richtung verschossen und es zeigt sich eine starke Corona. Trotz dieser geringen Photonenanzahl fällt auf, dass der Farbverlauf auf den diffusen Flächen weitaus heterogener ist. Deutlich sind auch die radialen Lichtkegel zu sehen, also dort, wo die meisten Photonen auftreffen. Dies verursacht die *Radiance Estimate* der Photon Map – außerdem wird für die direkte Beleuchtung eine Distanzabweichung zum Licht berechnet. Eine Erhöhung der Photonenanzahl oder der *Nearest Photons* wirkt sich nicht wesentlich auf das Ergebnis aus, da die Photonen sehr gleichmäßig in der Szene verteilt werden und insgesamt gesehen immer die gleiche Energie abgeben.

Diffuse Reflexionen haben sich bei der Implementation als Problem herausgestellt. Trotz mehrfacher Überprüfung der Samplingmethode finden die Sekundärstrahlen nicht ihre korrekte Richtung, um z.B. *Color Bleeding* darzustellen. Ebenso verhält es sich mit der Verwendung des *Path Tracers* im *Second Pass*: Es zeigt sich immer noch das eigentlich unerwünschte Rauschen. Jensen's Vorschlag, einen *Distribution Raytracer* zu verwenden, hätte eventuell eine Verbesserung gebracht. Allerdings kann dieser Mangel durch die Verwendung einer gering erhöhten Supersampling-Rate ausgeglichen werden. Bereits ab 10 Strahlen pro Pixel verringert sich die Varianz spürbar. Die fehlenden Daten – zumindest die auf diffusen Oberflächen – holt sich der *Path Tracer* aus der Photon Map. Die Abbildungen 6.3a und 6.3b zeigen die Unterschiede. Alle weiteren Beispielbilder der Cornell-Box wurden ohne Supersampling und diffuse Reflexion gerendert.



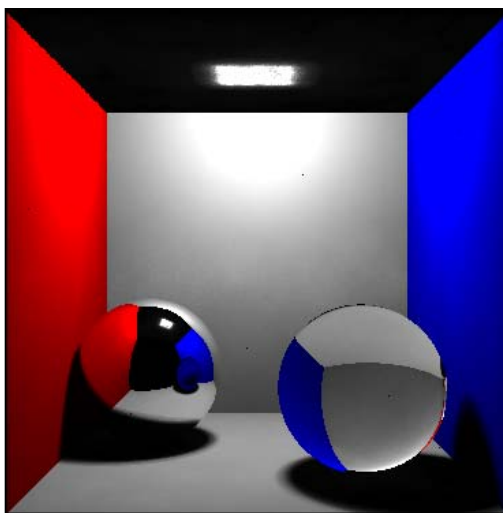
(a)



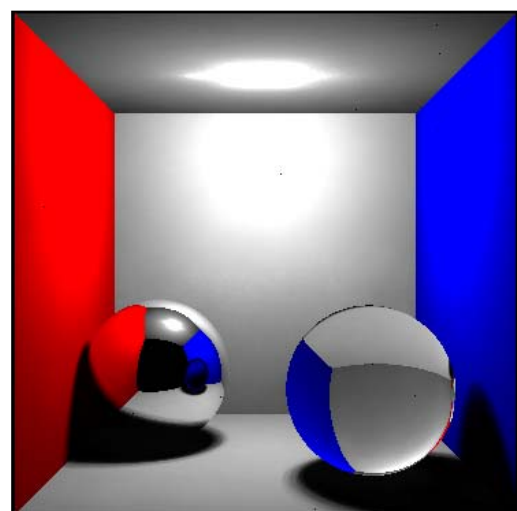
(b)

Abb. 6.3: Noise verursacht durch Path Tracing
(a) kein Supersampling, (b) 10 Strahlen pro Pixel

Mit Flächenlichtquellen lässt sich die Szene gleichmäßiger und heller ausleuchten. Im *Loader* kann festgelegt werden, ob die Lichtquelle sichtbar sein soll oder nicht. In Abbildung 6.4a ist die Lichtquelle eigentlich nicht als zu renderndes Objekt mit aufgenommen, hängt aber unmittelbar unterhalb der Decke (0.01 Abstand), weshalb quasi ihr Abbild an der Decke zu sehen ist. Ein wenig tiefer ist sie in Abbildung 6.4b angebracht. Es sind auch Halbschatten darstellbar, weil die Lichtquelle mit mehreren Schattenfühlern getestet wird und Pixel nur partiell im Dunkeln liegen können.



(a)



(b)

Abb. 6.4: Flächenlichtquellen und Halbschatten
(a) direkt unterhalb der Decke (b) 0.3 Abstand zur Decke

Das Rendern von *Kaustiken* ist etwas aufwendiger. Sie sind deutlich sichtbar, allerdings müssen die Distanz der spekularen Objekte (insbesondere der refraktierenden) zur Lichtquelle und die Anzahl dieser Objekte beachtet werden. Befinden sich zu viele spekulare Objekte in der Szene, erreichen wohlmöglich nicht genügend Photonen die entscheidenden Materialien. Letzteres kann durch

Regulierung der Parameter für die Klasse *Sample* beeinflusst werden. Eine Berücksichtigung in der Kommandozeile bzw. im Loader ist in der Abgabeverision nicht vorgesehen. Des Weiteren erreichen die *Caustic Photons* nicht konzentriert die Stellen, an welchen Kaustiken entstehen. Die entstehende Struktur ähnelt in den meisten Fällen einem Kreuz. Dies könnte durch eine Veränderung der Zufallszahlendichte beim Sampling reguliert werden. In den Abbildungen 6.5, 6.6 und 6.7 werden Kaustiken anhand einer Punkt- und einer Flächenlichtquelle sowie eines Spotlights mit jeweils 1.000.000 *Caustic Photons* und einer *Radiance Estimate* von 100 demonstriert. Die nebenstehenden Bilder zeigen die dazugehörige *Radiance Estimate*.

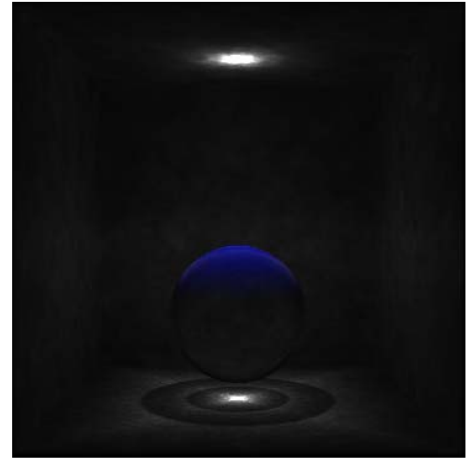
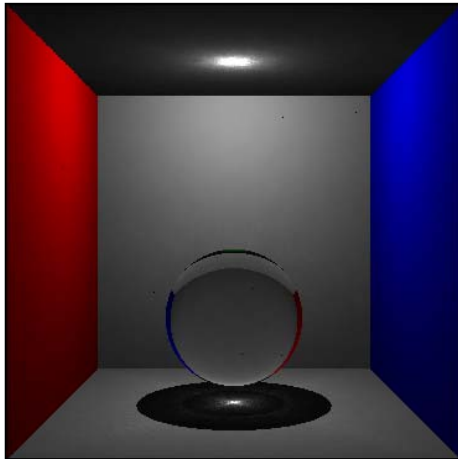


Abb. 6.5: Punktlichtquelle und zugehörige *Radiance Estimate*

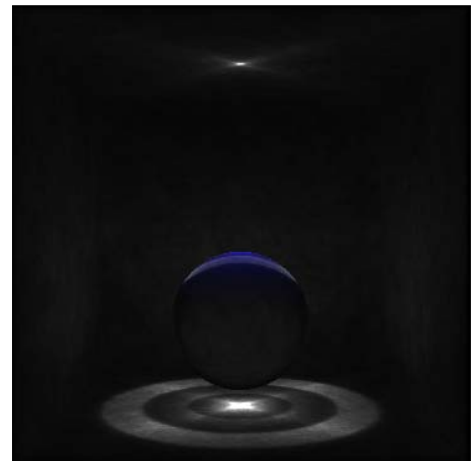
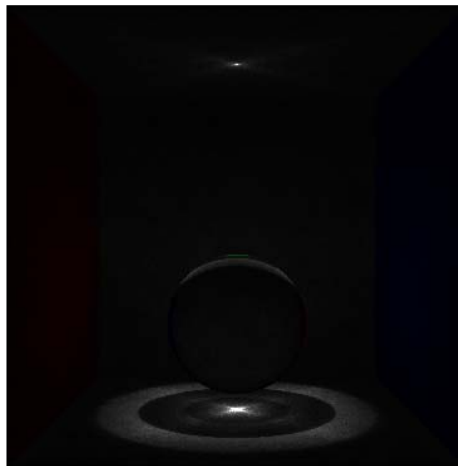


Abb. 6.6: Spotlight und zugehörige *Radiance Estimate*

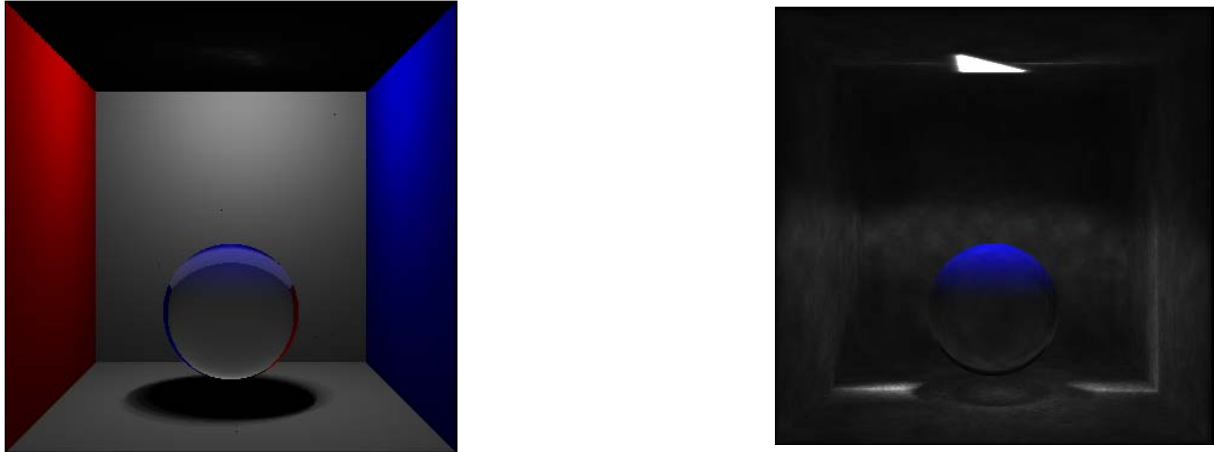
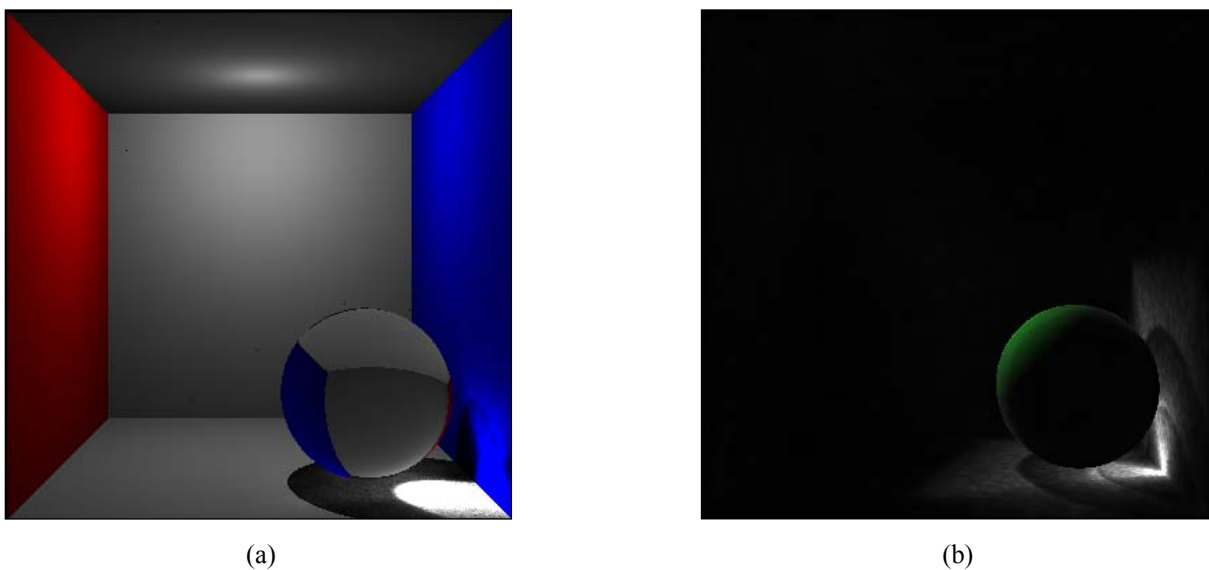


Abb. 6.7: AreaLight und zugehörige *Radiance Estimate*

Die Kaustiken sind bei allen Lichtquellen sehr schwach zu sehen, die Photonen der Flächenlichtquelle landen überhaupt nicht auf der diffusen Fläche unterhalb der Kugel. Das sind Schwachpunkte des Sampling und der Transmission. Die Photonen konzentrieren sich auf einen sehr schmalen Brennpunkt. Außerdem lagert über dem Schatten der Kugel ein weiterer, dessen Verursacher nicht ausfindig gemacht werden konnte.

Trotzdem lassen sich Kaustiken eindrucksvoll demonstrieren. Die Methode ist allerdings nicht ganz korrekt: Positioniert man eine beliebige Lichtquelle innerhalb eines transmittierenden Objektes, so werden die Photonen forciert, refraktiert zu werden. In Abbildung 6.8a befinden sich eine Punktlichtquelle außerhalb der Glaskugel und ein Spotlight mit 0.4 Öffnungswinkel innerhalb. Die Konzentration der Photonen ist hier deutlich verteilt, so dass sich der Brennpunkt vergrößert. Das *Radiance Estimate* der *Caustic Photon Map* mit einer außerhalb liegenden Punktlichtquelle verdeutlicht Abbildung 6.8b. Die Glaskugel ist nicht identisch mit der aus Abbildung 6.8a.



(a)

(b)

Abb. 6.8: (a) Punktlichtquelle außerhalb und Spotlight innerhalb
(b) *Radiance Estimate* mit einer außerhalb liegenden Punktlichtquelle

7. Optimierungsmöglichkeiten

Die Resultate zeigen, dass sich der Algorithmus durchaus noch weiter optimieren lässt. Die Möglichkeiten von Photon Mapping sind bei weitem nicht ausgeschöpft. Allerdings muss man bedenken, dass die Architektur des Basis-Raytracers nicht auf Photon Mapping ausgelegt ist. Um ein wirklich effizientes System zu erhalten, müsste ein grundsätzlich neues Programm entwickelt werden. Die Optimierungen, die der Raytracer von Haus aus mitbringt, beschränken sich nur auf Beschleunigungsmethoden für das Tracing von Sehstrahlen.

In den folgenden Unterkapiteln wird auf einige Optimierungstechniken kurz eingegangen.

7.1 Beschleunigung

Eine Tatsache muss leider direkt festgestellt werden: Die Geschwindigkeit des Raytracers hat sich verringert. Dies wird überwiegend durch die Verwendung eines Path Tracers, durch die hinzugekommenen BRDF-Berechnungen, durch Monte-Carlo-Sampling und die Schattenfühler für die Flächenlichtquellen verursacht. Demgegenüber steht aber eine bessere Qualität der Bilder, selbst wenn diese den Möglichkeiten von Photon Mapping nicht gerecht werden.

Die Beschleuniger des Raytracers zeigen ihr Können erst bei Szenen mit einer großen Anzahl von Objekten. Da Photonen ähnlich wie Sehstrahlen behandelt werden, können die Beschleuniger für das Tracen der Photonen verwendet werden. In der Praxis hat sich nur leider gezeigt, dass sich lediglich die (*Simple*) *Bounding Volume Hierarchies* und der *BSP-Tree* mit allen Arten von Szenen verstehen.

7.1.1 Parallelisierung

Das Modul *Parallelisierung* ist noch nicht an Photon Mapping angepasst worden. Zum einen ist seine Struktur im Quellcode sehr komplex und zum anderen muss sich eine Strategie überlegt werden, *First* und *Second Pass* auf die *Worker* aufzuteilen, da die Photon Map global vorhanden sein muss und nicht trivial aus Einzelteilen zusammengesetzt werden kann. Denkbar wäre daher, *Global Photon Map* und *Caustic Photon Map* von jeweils einem Worker aufbauen zu lassen, sie global verfügbar zu machen und die übrigen Worker die Teilbilder rendern zu lassen. Weitere Ideen wären, dass entweder jeder *Worker* seine eigene Photon Map aufbaut oder die Lichtquellen auf die *Worker* aufgeteilt werden und somit jeder eine Reihe von Photonen verfolgt, die in einer globalen Photon Map abgelegt werden.

7.1.2 Irradiance Caching

Besonders teuer ist es, die indirekte Beleuchtung auf diffusen Oberflächen zu berechnen, weil – für den *Distribution Raytracer* – eine Vielzahl an Sekundärstrahlen pro Pixel verschossen werden müssen und Lambertsche Flächen in vielen Szenen den überwiegenden Anteil ausmachen. Die Idee des Irradiance Caching entstammt nun der Beobachtung, dass die Beleuchtungsstärke auf diffusen Oberflächen sehr homogen ist und sich von Position zu Position nur leicht verändert. Die

Beschleunigung besteht nun darin, nur für ausgewählte Punkte die Beleuchtungsstärke zu berechnen und die übrigen Wert mit den vorberechneten zu interpolieren. Jede im sogenannten *Irradiance Cache* abgelegte Beleuchtungsstärke an einem Punkt bekommt zur Priorisierung bei der Interpolation eine Gewichtung. Die Gewichtung ergibt sich durch eine Abschätzung der Abweichung zweier Oberflächenpositionen: Entweder die Positionsabweichung oder die Abweichung in der Orientierung zur Oberflächennormalen wird als Gewichtungsfaktor genommen, je nachdem welche geringer ist.

Für die Einbindung in Photon Mapping muss berücksichtigt werden, dass *Irradiance Caching* für Kaustiken nicht verwendet werden sollte, sondern nur die Daten der *Caustics Photon Map* ausschlaggebend sind.

7.1.3 Shadow Photons

Beim Einsammeln der direkten Beleuchtung ist die Versendung von Schattenfählern besonders teuer. Insbesondere für Flächenlichtquellen müssen mehrere dieser Strahlen zur Lichtquelle geschickt werden, um Halbschatten darstellen zu können. Eine Überlegung geht nun dahin, sich zusätzlich während des *First Pass* diejenigen Photonen zu merken, die im Schatten eines Objektes liegen: die *Shadow Photons*. Um dieses Verhalten festzustellen, werden sie geradlinig durch die Szene getraced und auf der Fläche hinter dem ersten getroffenen Objekt abgelegt. Die Fläche muss auf jeden Fall von der Lichtquelle bestrahlt werden können, d.h. ihre Normale darf nicht von der Lichtquelle wegzeigen. Andere Flächen, die nämlich von der Lichtquelle abgeneigt sind, können einfach als Schattenflächen identifiziert werden. Sie werden mit *negativer* Energie und einem Flag für direkte Beleuchtung gespeichert.

Eine Möglichkeit, die *Shadow Photons* zur Schattenberechnung zu verwenden, besteht darin, das Verhältnis von *direkt leuchtenden* Photonen und *Shadow Photons* zu bestimmen und dies als Visibilitätsfaktor für die Lichtquelle zu nehmen. Der Wert der direkten Beleuchtung wird dann mit diesem Faktor skaliert.

7.1.4 Projection Maps

Caustic Photons sollen in die Richtung spekulärer Objekte emittiert werden. Es ist daher entscheidend, vor Beginn der Emission bereits Kenntnisse über die Objekte der Szene zu besitzen. Eine *Projection Map* ist eine zweidimensionale Abbildung (mit den Polarkoordinaten (θ, ϕ)) der Hemisphäre an der Position der Lichtquelle. Diese *Map* ist in einzelne Zellen aufgeteilt und speichert dort die Richtungen zu den Objekten der Szene. Da es schwierig ist, die Geometrie der Objekte auf die *Map* zu projizieren, werden zur Vereinfachung *Bounding Spheres* verwendet. Aus der Projektion von Sphäre auf Sphäre ergibt sich daher ein Kreis, anhand dessen Radius die Informationen des Objekts in die Zellen der *Map* eingetragen werden. Nähere Informationen sind in [Jensen95] zu finden.

Fazit

Henrik Wann Jensen's „*Realistic Image Synthesis Using Photon Mapping*“ ist eine Sammlung von Grundlagen zu photorealistischem Rendern und von Ideen und Konzepten rund um Photon Mapping. Auf Grund seiner Kompaktheit war dieses Buch eine ideale Grundlage für die vorliegende Studienarbeit, nicht zuletzt weil Jensen der Entwickler des Verfahrens ist.

Die Integration des Photon Mapping in den Raytracer war indes nicht allzu einfach zu realisieren. Das System war ursprünglich darauf ausgelegt, verschiedene Beschleunigungsalgorithmen miteinander zu vergleichen: *BSPTree*, *Uniform Grid*, *Octree*, *Bounding Volume Hierarchies* und *Parallelisierung*. Als Verfahren zur Darstellung globaler Beleuchtung und anderen Effekten waren hingegen *Path Tracing* und die *ThinsLensCamera* gedacht. Dementsprechend ist die Struktur des Systems auf dieses Ziel hin aufgebaut. Trotz Modularisierung mussten zu den meisten Klassen neue Methoden und Membervariablen hinzugefügt werden, da *Photon Mapping* in jeden Abschnitt – Tracing und Rendering – eingreift. Insbesondere der *First Pass* benötigte Ergänzungen für das Sampling und Tracing von Photonen. Des Weiteren sollte die Funktionalität des bestehenden Systems erhalten bleiben. Wie es sich zeigte, musste Photon Mapping aber eher an den Quellcode angepasst werden als umgekehrt. Um die Effizienz von Photon Mapping vollständig ausnutzen zu können, wäre die Implementation eines neuen Systems daher besser geeignet. Die Integration eines Verfahrens in ein bestehendes System scheint infolgedessen mindestens genauso aufwendig wie die Entwicklung eines neuen. Trotz alledem liegt der größte Vorteil einer Integration darin, dass auf vorhandene Basisklassen und Algorithmen zurückgegriffen werden kann, selbst wenn einige davon mehr oder weniger „sauber“ implementiert worden sind.

Im Bezug auf die Darstellung von *Kaustiken* – das eigentliche Ziel der Studienarbeit – war es am wichtigsten, die *Caustic Photons* in Richtung spekulärer Materialien zu emittieren. Ohne die massive Aussendung von *Caustic Photons* lassen sich Kaustiken nicht realisieren. Zusätzlich sollte also auch das Tracing der Photonen zügig ablaufen. Wie die Ergebnisbilder zeigen, funktionieren nicht beide Methoden so wie gewünscht. Die Photonen werden beispielsweise konzentriert zur Mitte der gedachten Kreisscheibe hin emittiert, so dass sich als Kaustik ein sehr kleiner Brennpunkt ergibt. Um diesen herum sind die Photonen sehr schwach verteilt. An dieser Stelle müsste die Dichte der Samples auf der Kreisscheibe näher untersucht und eventuell angepasst werden. Die Zeiten für das *Photon Tracing* sind hingegen akzeptabel.

Angesichts der Tatsache, dass Kaustiken generell darstellbar sind, kann ich persönlich mit dem Ergebnis der Arbeit zufrieden sein, vor allem, weil es ein langer Weg dahin war, bis sich das erste Erfolgserlebnis zeigte. Zum Beginn war ich der Überzeugung, dass es einfacher ist, ein System zu erweitern, vor allem dann wenn man selbst an seiner Entwicklung beteiligt gewesen ist. Mit der Zeit zeigte sich allerdings, dass der Gesamtüberblick über solch ein komplexes System – inzwischen sind es 55 Klassen – trotz Dokumentation schnell verloren gehen kann. Die Entwicklung eines neuen Systems mag daher den Vorteil haben, dass man von Beginn an auf Photon Mapping hin optimieren kann und dabei auch den Überblick behält. Trotzdem ist Photon Mapping ein Verfahren, um das sich ein Raytracer einfach erweitern lässt. Durch die zahlreichen Erweiterungen dauert es allerdings einige Zeit, bis sich sehenswerte Resultate rendern lassen.

Literatur

Papers & Hardcopy

- [Cook84] Robert L. Cook, Thomas Porter und Loren Carpenter, „Distributed Ray Tracing.“, Computer Graphics (SIGGRAPH '84) 18(3): 137-147, Juli 1984
- [Foley97] James D. Foley, Andries van Dam, Steven K. Feiner und John F. Hughes, „Computer Graphics – Principles and Practice“, Zweite Ausgabe, Addison-Wesley, 1997
- [Geimer00] Markus Geimer, „DREAM – Ein verteiltes Raytracing-System mit Optimierungen“, Diplomarbeit, Universität Koblenz-Landau, Mai 2000
- [Goral84] Cindy M. Goral, Kenneth E. Torrance, Donald P. Greenberg und Bennett Battaile, „Modelling the interaction of light between diffuse surfaces.“, Computer Graphics (SIGGRAPH '84) 18(3): 212-222, Juli 1984
- [Haines89] Eric Haines, „Essential Ray Tracing Algorithms. In: Glassner, Andrew S. (Herausgeber): „An Introduction to Ray Tracing“, 33–77. Academic Press, London, 1989.
- [Heckbert90] Paul S. Heckbert, „Adaptive radiosity textures for bidirectional ray tracing.“, Computer Graphics (SIGGRAPH '91) 24(4): 145-154, August 1990
- [Jensen95] Henrik Wann Jensen und Per H. Christensen, „Photon maps in bidirectional Monte Carlo ray tracing of complex objects.“ Computers & Graphics 19(2): 215-224, März 1995
- [Jensen01] Henrik Wann Jensen, „Realistic Image Synthesis Using Photon Mapping“, A K Peters, Ltd., Natick, Massachusetts, 2001
- [Kajiya86] James T. Kajiya, „The rendering equation.“ Computer Graphics (SIGGRAPH '86) 20(4): 269-278, August 1986
- [Marlon03] John Marlon, „Focus on Photon Mapping“, Premier Press, Boston, Massachusetts, 2003
- [Müller03] Stefan Müller, „Photorealistische Computergraphik.“, Skript zur Vorlesung „Photorealistische Computergraphik“, Universität Koblenz-Landau, 2003
- [Müller03] Stefan Müller, „Computergraphik 2.“, Skript zur Vorlesung „Computergraphik 2“, Universität Koblenz-Landau, 2003
- [Schlick93] Christophe Schlick, „A Customizable Reflectance Model for Everyday Rendering.“, In: Fourth Eurographics Workshop on Rendering, editiert von Michael F. Cohen, Claude Uech und Francois Sillion, 73-84, Eurographics, Juni 1993

- [Shirley03] Peter Shirley, „Fundamentals Of Monte Carlo Integration“, In: Monte Carlo Ray Tracing (SIGGRAPH `03), Kurs 44, 13-28, Juli 2003
- [Ucke98] Christian Ucke und Christoph Engelhardt, „Kaustik in der Kaffeetasse“, In: Physik in unserer Zeit, 29, 120-122, 1998
- [Weizel03] Christian Weizel, „Bidirektionales Path Tracing“, Studienarbeit, Universität Koblenz-Landau, November 2003

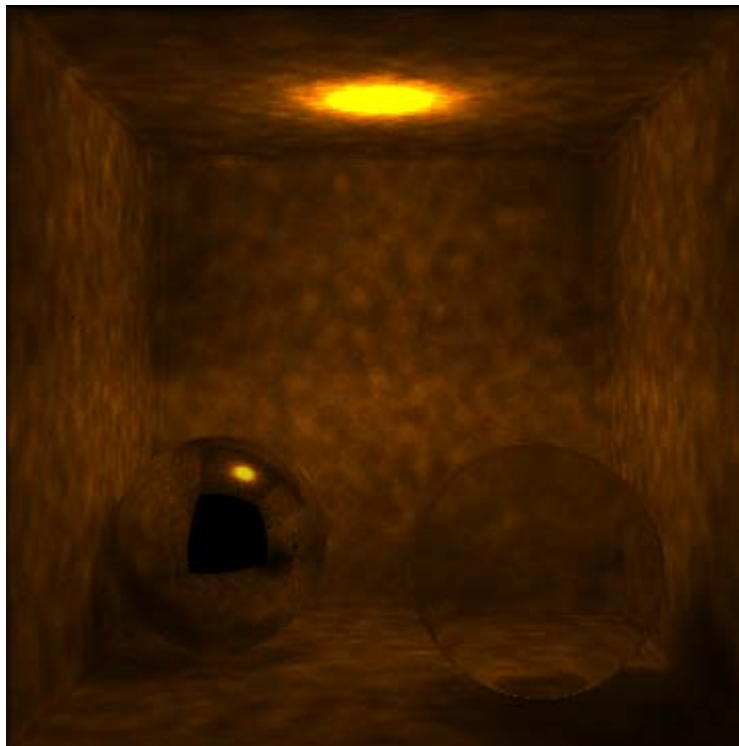
Internetquellen

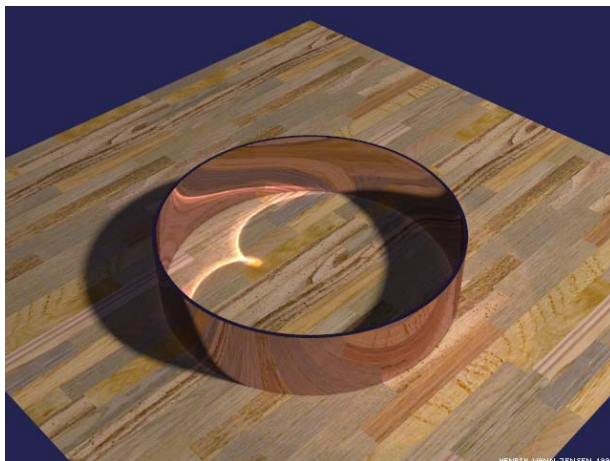
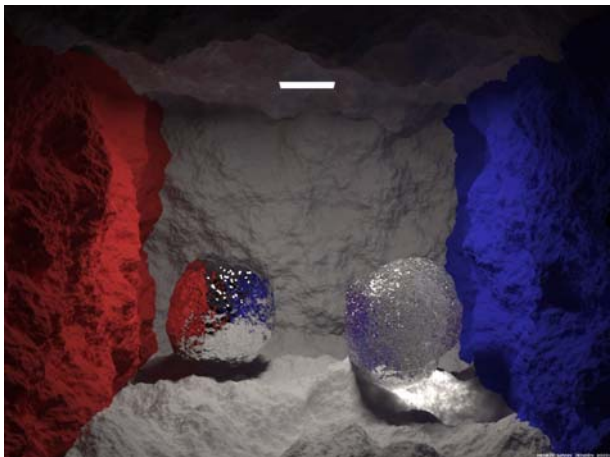
Henrik Wann Jensen's Homepage

USCD Computer Graphics Laboratory, <http://graphics.ucsd.edu/~henrik/>

Anhang

Nebeneffekte aus der ersten Implementationsphase



Beispiele von Henrik Wann Jensen

Quelle: <http://graphics.ucsd.edu/~henrik/>